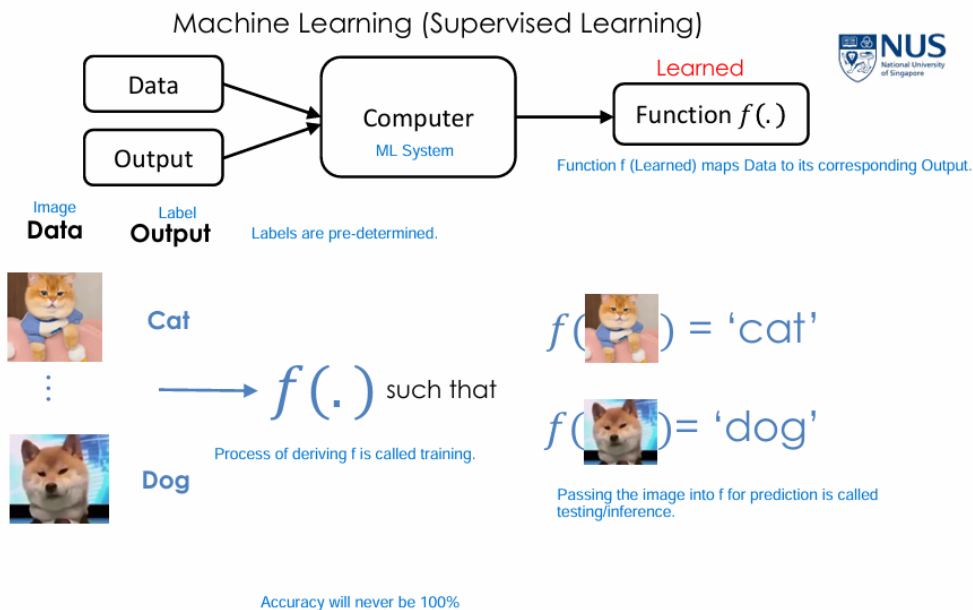


What is Machine Learning?

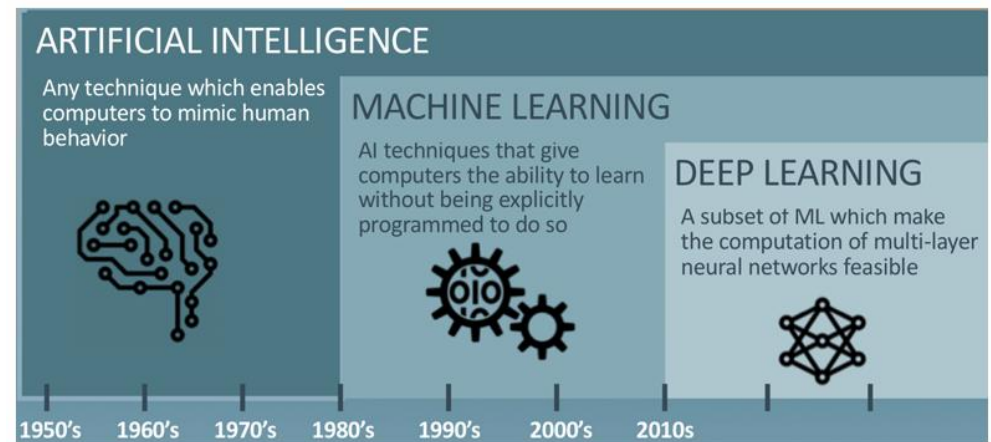
Machine Learning is a computer program that is said to learn:

- from **experience E**
- with respect to some class of **tasks T**
- and **performance measure P**

if its performance at tasks in T, as measured by P, improves with experience E. ~ Tom Mitchell



AI, Machine Learning and Deep Learning



Example of AI but not ML: Deductive Reasoning

NUS is in Singapore; Singapore is in Asia → NUS is in Asia.

When do we need machine learning? (Learning is not always useful)

- No human guidance
- Too much data for human to process

No human guidance
 Lack of human expertise
 (Navigating on Mars)



Too much data for human to process
 Involves huge amount of data
 (Genomics)



Application of Machine Learning

T: Digit Recognition
P: Classification Accuracy
E: Labelled Images

Data/Digits Image

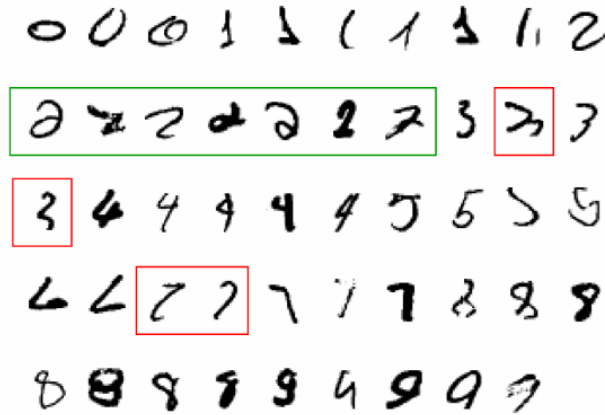
4 "four"

3 "three"

Output/Labels

Labels -> Supervision!

^ Supervised Learning



Task T, Performance P, Experience E

T: Email Categorization
P: Classification Accuracy
E: Email Data, Some Labelled



^ Supervised Learning

Task T, Performance P, Experience E

T: Playing Go Game
P: Chances of Winning
E: Records of Past Games

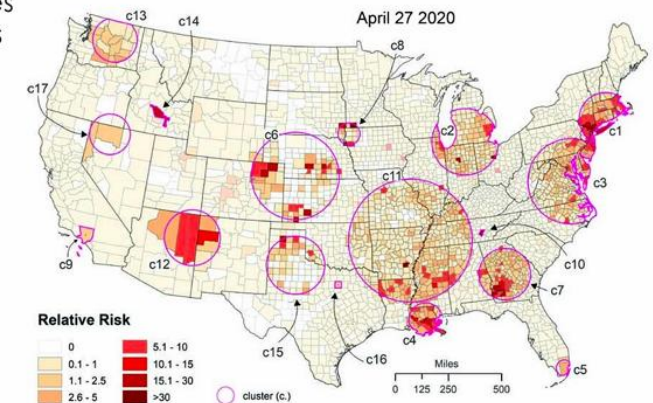


© Copyright EE, NUS. All Rights Reserved.

^ Reinforcement Learning

Task T, Performance P, Experience E

T: Identifying Covid-19 Clusters
P: Small Internal Distances
Larger External Distances
E: Records of Patients



^ Unsupervised Learning

Type of Machine Learning

Supervised Learning

Input:
1) Training Samples,
2) Desired Output
(Teacher/Supervision)

Output:
A rule that maps input to output

Unsupervised Learning

Input:
Samples

Output:
Underlying patterns in data

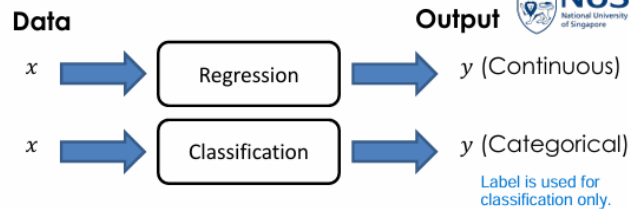
Reinforcement Learning

Input:
Sequence of States,
Actions, and
Delayed Rewards

Output:
Action Strategy: a rule
that maps the
environment to action

Supervised Learning

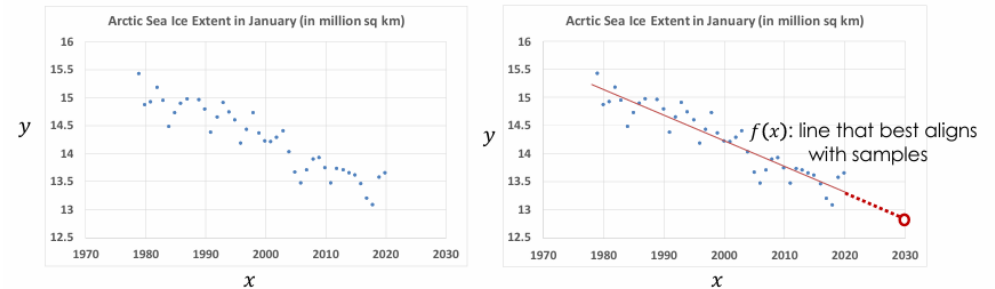
Supervised Learning



Regression

- Given $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$
- Learn a function $f(x)$ to predict real-valued y given x

Regression

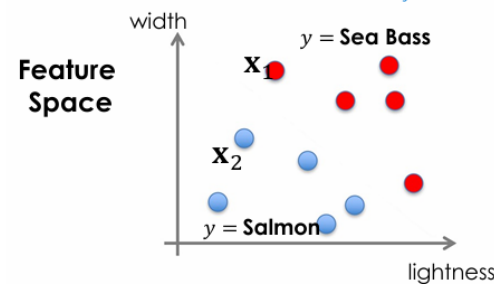


Classification

Classification

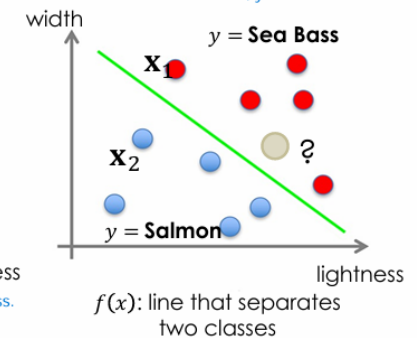
- Given $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$
- Learn a function $f(x)$ to predict categorical y given x

Features and number of features are determined by us



They are categorized using 2 features/attributes, width and lightness. This is known as the feature space.

If it lies on the line, you can take it as either.



$f(x)$: line that separates two classes

- Can handle all kinds of data in general.

Unsupervised Learning (Clustering)

Unsupervised Learning

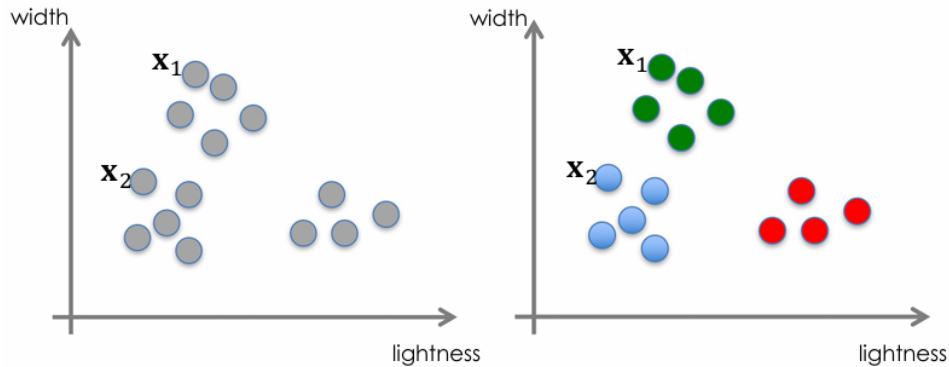


The data given has no labels! They are just data.

Clustering

- Given $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$, without labels
- Output Hidden Structure Behind

Grouping points that are close to each other in the feature space.



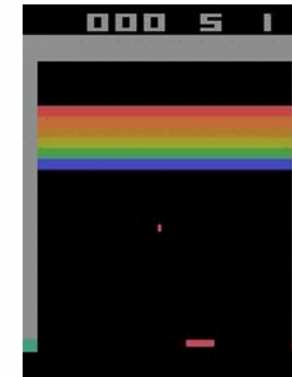
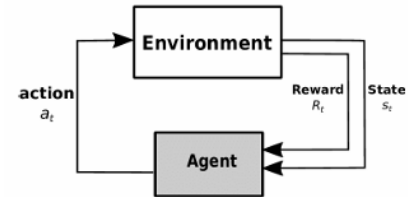
No Label/Supervision is given!

- Important difference than supervised learning: NO LABELS!!
- Works for all data types

Reinforcement Learning

Key Idea: As long there is a sequence of states and actions, it is reinforcement learning.

- Given sequence of states $\mathbf{S}$ and actions $\mathbf{A}$ with (delayed) rewards $\mathbf{R}$
- Output a policy $\pi(a,s)$, to guide us what action a to take in state s



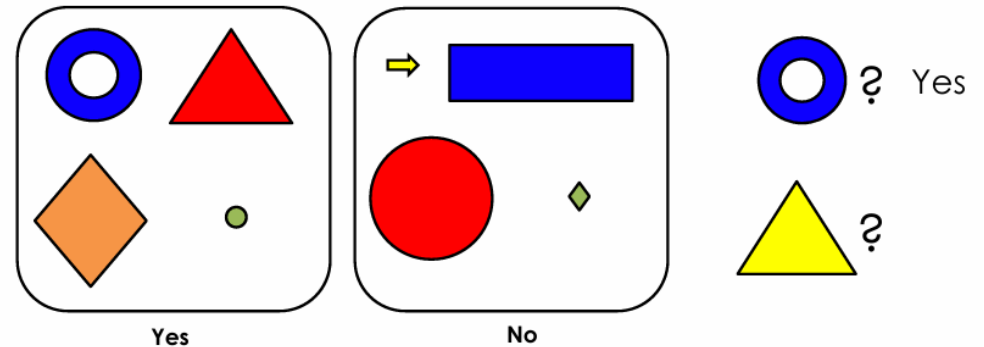
$\mathbf{S}$: Ball Location,
Paddle Location, Bricks
 $\mathbf{A}$: left, right

$\mathbf{R}$:
positive reward
Knocking a brick,
clearing all bricks

negative reward
Missing the ball

zero reward
Cases in between

Toy Example

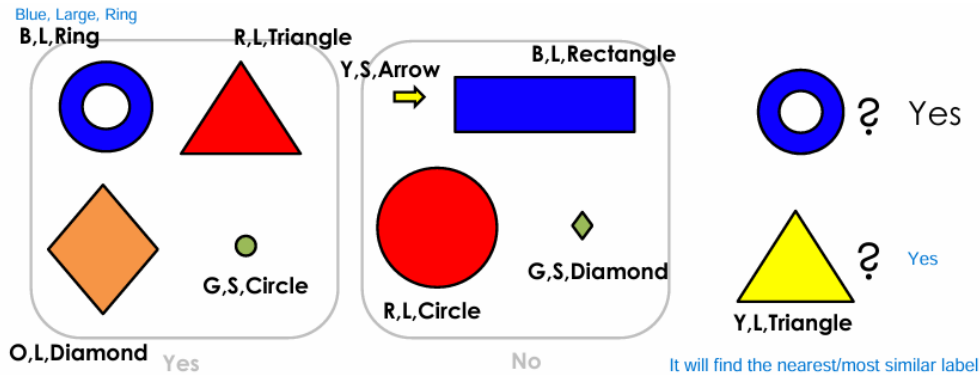


Step1: Feature Extraction
Extract Attributes of Samples

Shape, Color, Size

Step2: Sample Classification
Decide Label for a Sample

Yes and No: 2 labels



Step 1: Feature Extraction
Color, Size, Shape

Table of Similarity

- Determines the testing sample is most similar to which training sample and use that label.

Feature Extraction					Similarity				
	Color	Size	Shape	Label		Color	Size	Shape	Total
	Blue	Large	Ring	Yes		0	1	0	1
	Red	Large	Triangle	Yes		0	1	1	2
	Orange	Large	Diamond	Yes		0	1	0	1
	Green	Small	Circle	Yes		0	0	0	0
	Yellow	Small	Arrow	No		1	0	0	1
	Blue	Large	Rectangle	No		0	1	0	1
	Red	Large	Circle	No		0	1	0	1
	Green	Small	Diamond	No		0	0	0	0

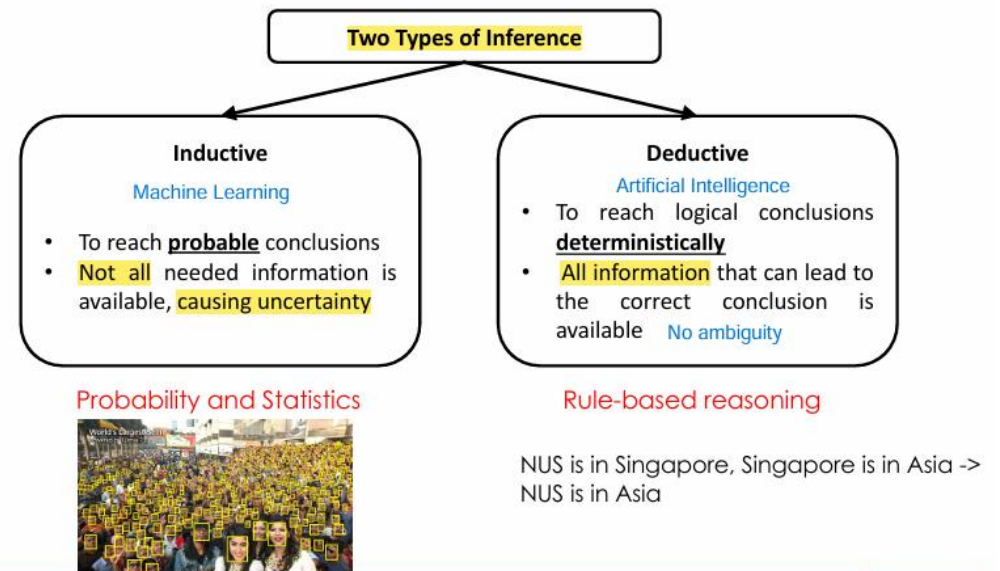
- If you have multiple total that are maximum, you can choose any or choose more features to make them distinct. More features = Lesser coincidence for maximum duplicates.

Also known as **Nearest Neighbour Classifier (Supervised)**:

- 1) Find the “nearest neighbour” of a sample in the feature space
 - 2) Assign the label of the nearest neighbour to the sample
- Usually work on numerical data (E.g. Euclidean) but also works on categorical data if it is properly encoded (Binary or One-Hot)
 - Outputs are nominal/ordinal labels, not ratio/interval data.

Inductive vs. Deductive Reasoning

- Main Task of Machine Learning: To make inference



- All of ML, DL, NN uses inductive reasoning and not deductive.

Inductive Reasoning

- Humans use inductive reasoning all the time and not in a formal way like using probability/statistics.



Quick Summary

Three Components in ML Definition

Task T, Performance P, Experience E

Three Types of in ML

Supervised Learning
Unsupervised Learning
Reinforcement Learning

Inductive and Deductive

Inductive: Probable
Deductive: Rule-based

Two Types of Supervised Learning

Classification, Regression

One Type of Unsupervised Learning

Clustering

Example of a Classifier Model

Nearest Neighbor Classifier

Practice Questions

Practice Question

(Type of Question to Expect in Exams)



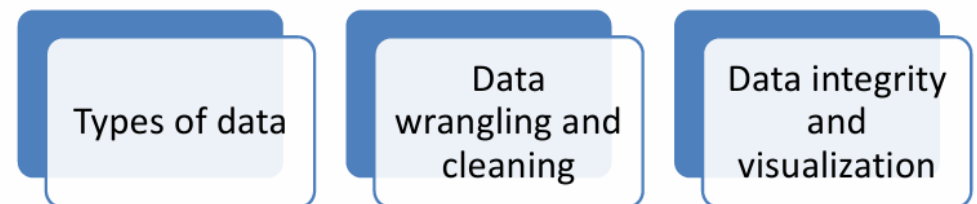
Which of the following statement is true?

- A. Nearest Neighbor Classifier is an example of unsupervised learning *Wrong: It should be supervised learning.*
- B. Nearest Neighbor Classifier is an example of deductive learning *Machine Learning is inductive learning.*
- C. Nearest Neighbor Classifier is an example of feature selection *Feature Extraction is just the first step taken to discover structure within the data. The Nearest Neighbour Classifier is one of the classification algorithms used in the second step (Classification) that is taken based on the first step, Feature Extraction.*
- D. None of the above is correct.**

Different feature extraction method will lead to different classification results.

Both supervised and unsupervised learning involves feature extraction as the first step. But second step for supervised learning is classification while for unsupervised learning is clustering.

Data Engineering



Types of Data

Statistics, Numbers, Text, Figures, Records, Facts

Ways of Viewing Data

1. Based on Levels/Scales of Measurement

- NOIR (Nominal, Ordinal, Interval, Ratio Data)

2. Based on Numerical/Categorical

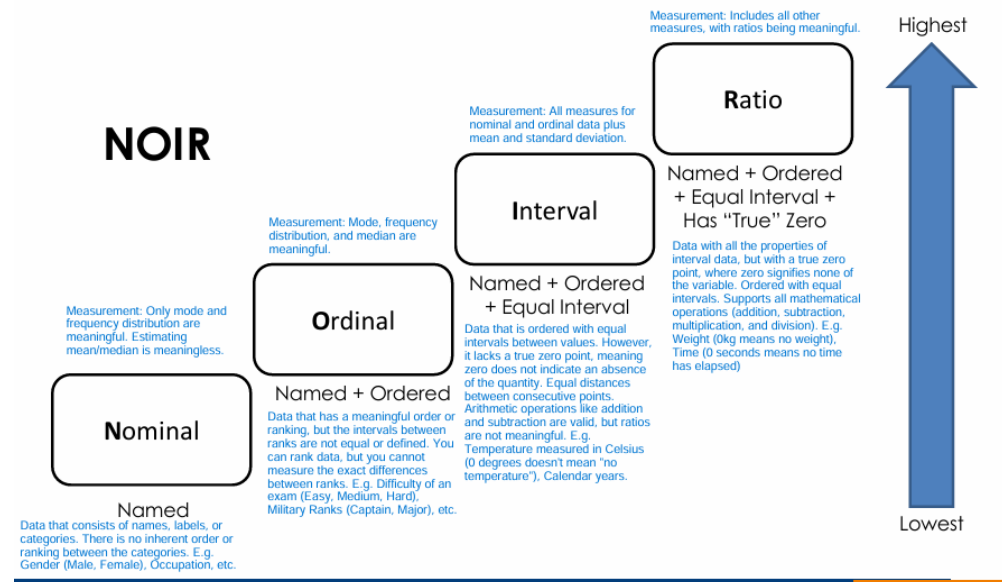
- Numerical, also known as Quantitative

- Categorical, also known as Qualitative

3. Other aspects

- Available or Missing Data

Levels/Scales of Measurement



Mean, Median, Mode

If we are given a sequence of numbers: 1,3,4,6,6,7,8

Mean: Computing the average

$$(1+3+4+6+6+7+8)/7=5$$

Median: Number in the middle (**After Sorting**)

1, 3, 4, 6, 6, 7, 8

*In case of even number of elements

$$1, 3, \underline{4, 6}, 7, 8 \rightarrow (4+6)/2=5$$

Mode: Number with highest frequency

1, 3, 4, 6, 6, 7, 8 → 6 is the number with highest frequency

Nominal Data

- Data that consists of names, labels, or categories. There is no inherent order or ranking between the categories.

- Lowest Level of Measurement

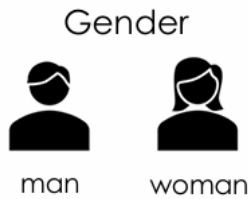
- **Discrete** Categories

- **NO** natural order

- Estimating a mean, median, or standard deviation, would be **meaningless**.

- Possible Measure: mode, frequency distribution

- Example:



Ordinal Data

- Data that has a **meaningful order** or ranking, but the intervals between ranks are **not equal** or defined.
- You can rank data, but you cannot measure the **exact** differences between ranks.
- **Ordered** Categories
- Relative Ranking
- Unknown “distance” between categories: Orders matter but not the difference between values
- Possible Measure: Mode, Frequency Distribution + Median

Example:

Difficulty of an exam (Easy, Medium, Hard)

Military Ranks (Captain, Major), etc.

Interval Data

- Ordered Categories
- Well-defined “unit” measurement: Distances between points on the scale are measurable and well-defined
- Can measure differences!
- **Equal Interval** (between two consecutive unit points)
- **Zero is arbitrary** (not absolute), in many cases human-defined. If the variable equals zero, it does not mean there is none of that variable!
- **Ratio is meaningless**
- Possible Measure: Mode, Frequency Distribution + Median + Mean, Standard Deviation, Addition/Subtraction

Example:

Temperature measured in Celsius: For instance: 10 degrees C, 28 degrees C

Year of someone’s birth: For instance: 1990, 2005, 2010, 2022

Ratio Data

- Most precise and **highest** level of measurement
- Ordered
- Equal Intervals
- Natural Zeros: If the variable equals zero, it means there is none of that variable. Not Arbitrary.

- Possible Measure: Mode, Frequency Distribution + Median + Mean, Standard Deviation, Addition/Subtraction + Multiplication and Division (Ratio)

Example:

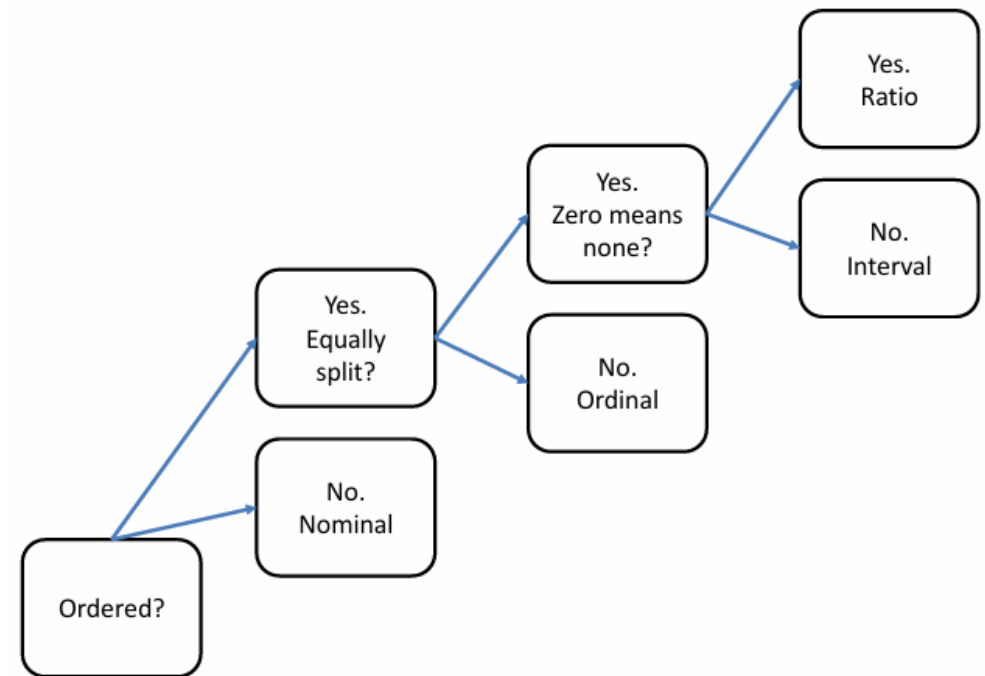
Weights: 10KG, 20 KG, 30 KG

Time: 10 Seconds, 1 Hour, 1 Day

NOIR Table

We can estimate	Nominal	Ordinal	Interval	Ratio
Frequency Distribution	Yes	Yes	Yes	Yes
Median	No	Yes	Yes	Yes
Add or subtract	No	No	Yes	Yes
Mean, standard deviation	No	No	Yes	Yes
Ratios	No	No	No	Yes

NOIR Flowchart



Quick Quiz

- Which level of measurement?

Nominal, Ordinal, Interval, Ratio

1. Favorite Restaurant

- Mcdonald's, Burger King, Subway, KFC, ...

Nominal

2. Weight of luggage measured in KG

Ratio

3. SAT Scores: note that, SAT ranges is [400, 1600]

Interval

4. Size of Packed Eggs in supermarkets

- Small, Medium, Large, Extra Large, ...

Ordinal

5. Military rank

- General, Major, Captain, ...

Ordinal

6. Number of people in a household

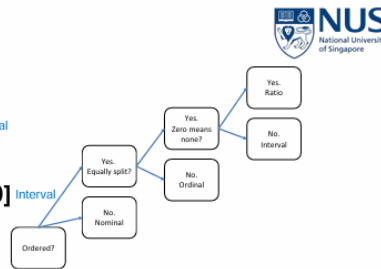
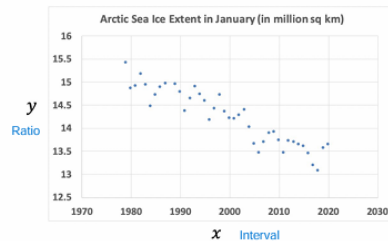
- 1, 2, 3, 4, 5, ...

Ratio

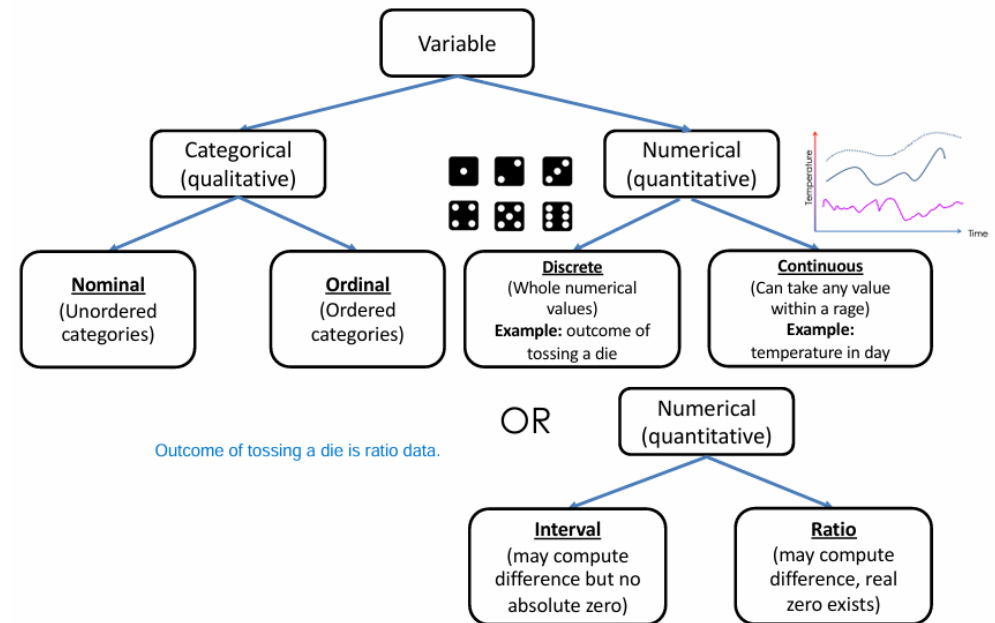
7. Credit Score in United States: the range is [300, 850]

Interval

8.



Numerical or Categorical



Missing Data

Missing Data: Data that is missing and you do not know the mechanism.

- You should use a single common code for all missing values (for example, "NA"), rather than leaving any entries blank.
- Replacing missing values (NA) with an arbitrary value like -1 and **including them in calculations for mean and standard deviation** is NOT a common or statistically sound practice.

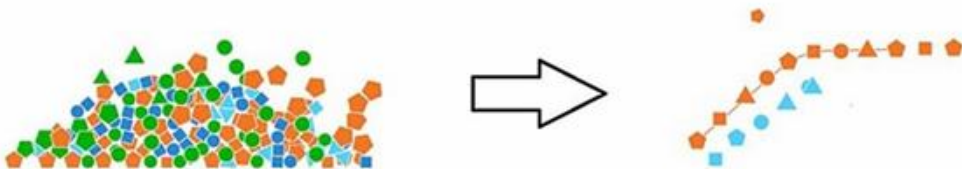
Data Wrangling

Data Wrangling: The process of transforming and mapping data from one "raw" data form into another format with the intent of making it more appropriate and valuable for a variety of downstream purposes such as analytics.

- In short, transforms data to gain insight. Can be done on **any** data!
- It is a general process!

Data wrangling is the process of cleaning, transforming, and organizing raw data into a usable format suitable for analysis or machine learning (ML) tasks. It involves converting raw, messy, or unstructured data into clean, structured, and insightful datasets that can be effectively used in ML models.

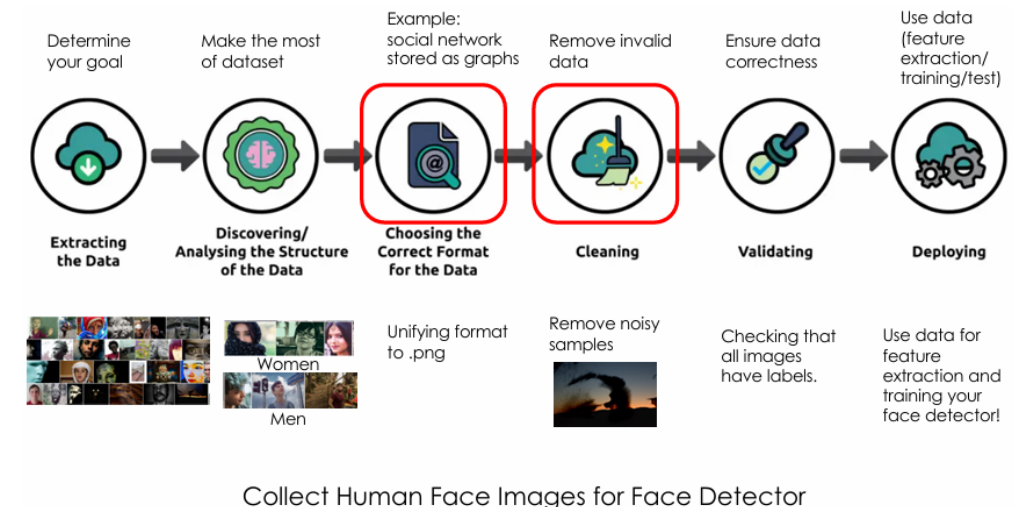
Think of data wrangling as preparing ingredients for cooking – if the raw ingredients (data) are not cleaned or prepped, the final dish (ML model) may fail or produce incorrect results.



When is Data Wrangling Used?

Data wrangling is a critical step in the data preprocessing pipeline and is used:

- Before model training: To ensure the input data is clean, complete, and in a format that the algorithm can process.
- During feature engineering: To create meaningful features that improve model performance.
- When handling messy or incomplete datasets: For example, when working with real-world datasets containing missing values, outliers, or irrelevant information.



Formatting Data

- **Binary Coding** to convert categories into binary form
- One-hot encoding: Unify several entities within one vector

Example: The colour of a pixel can be red, yellow or green

- Very common in classification tasks!

One-hot encoding is a method of converting categorical data (data with discrete categories or labels) into a binary (0s and 1s) format that machine learning algorithms can understand.

Each category is represented as a binary vector, where only ONE position is marked as "1" (indicating the presence of that category) and all other positions are "0".

In the scenario that you need to indicate the presence of 2 or more categories, you will use 2 or more vectors.

E.g. 2 Categories = 2D vector. One vector is for one category. You cannot mark "1" in more than 1 position in a vector!

$$\begin{aligned}\text{red} &= [1, 0, 0] \\ \text{yellow} &= [0, 1, 0] \\ \text{green} &= [0, 0, 1]\end{aligned}$$

Cannot [1, 1, 0]. It is called one-hot encoding! Only 1 position is marked!

Most machine learning models, especially those based on mathematical operations (like linear regression, logistic regression, or neural networks), cannot directly process categorical data.

They require numerical input. One-hot encoding helps translate these categories into numbers in a way that the model can interpret while avoiding the misleading ordinal relationships that numeric labels can imply.

Normalization

Normalization is a data preprocessing technique used to scale numerical data so that it falls within a specific range (commonly [0, 1]).

This ensures that all features contribute equally to a machine learning model, preventing features with larger numerical ranges from dominating the training process.

Linear Scaling: Use formula below to scale data. x_{raw} is the individual raw data value. x_{min} is minimum value out of all values and x_{max} is maximum value out of all values. (Scale each data point to 0-1)

$$x_i = \frac{x_i^{raw} - x^{min}}{x^{max} - x^{min}}, \quad i = 1, 2, \dots, M$$

Z-score standardization: Each independent dimension of data is normally distributed.

$$x_i = \frac{x_i^{raw} - E[X]}{\sigma(X)}, \quad i = 1, 2, \dots, M.$$

This is used when your data is normally distributed or assumed to be, or when you need to ensure zero mean and unit variance (e.g., PCA).

Accounts for standard deviation so that it is easier to compare 2 different data sets.

- Z-score standardization rescales data to have **mean 0** and **standard deviation 1**, but values can be **any real number**, including negatives or numbers > 1.

- It's not bounded between 0 and 1.

Data Cleaning

The process of detecting and correcting (or removing) corrupt or inaccurate records from a record set, table, or database.

Example:

- Clipping Outliers (Extreme values in a dataset are **capped or replaced** by predefined threshold values, thus stabilizing models by reducing the skewness caused by extreme values while retaining the overall structure of the data.)

- Handling Missing Data

Data Cleaning: Handling Missing Features

1. Removing the examples with missing features from the dataset

- Can be done if the dataset is big enough so we can sacrifice some training examples

2. Using a learning algorithm that can deal with missing feature values. E.g. Random forest.

- Doesn't fill up the missing values.

3. Using a data imputation technique

- Data imputation is a data cleaning technique used to handle missing data in a dataset.

- Instead of discarding rows or columns with missing values, imputation involves filling in these gaps with estimated or substituted values.

- The goal is to preserve as much data as possible while maintaining the dataset's statistical integrity.

No right or wrong to choose which method, it depends on the scenario. It is possible to use 2 or more methods at the same time.

Data imputation can apply to mixed-type data (categorical, count, continuous)

Data Cleaning: Handling Missing Features: Imputation

Method 1: Replace the missing value of a feature by an **average** (It can be mean for normally distributed data, mode for categorical data or median for skewed data or data with outliers) value of this feature in the dataset:

$$\hat{x}^{(j)} \leftarrow \frac{1}{N} \sum_{i=1}^N x_i^{(j)}$$

Method 2: Highlight the missing value (Commonly Used/Preferred)

- Replace the missing value with a value outside the normal range of values.
- For example, if the normal range is $[0, 1]$, then you can set the missing value to -1 . Often used to signal that a value is missing, allowing the model to learn patterns involving missingness.
- Enforce the learning algorithm to learn what is best to do when the feature has a value significantly different from regular values.
- There are many more imputation techniques.
- Do not include missing values that are replaced with "NA" or -1 in calculations for mean and STD DEV.

Data Integrity

- Data integrity is the maintenance and the assurance of data accuracy and consistency
- A critical aspect to the **design**, implementation, and usage of any system that stores, processes, or retrieves data.
- Very broad concept!
- Data integrity refers to the accuracy, consistency, and reliability of data throughout its lifecycle.
- Data integrity applies to all data types (nominal, ordinal, interval, ratio).

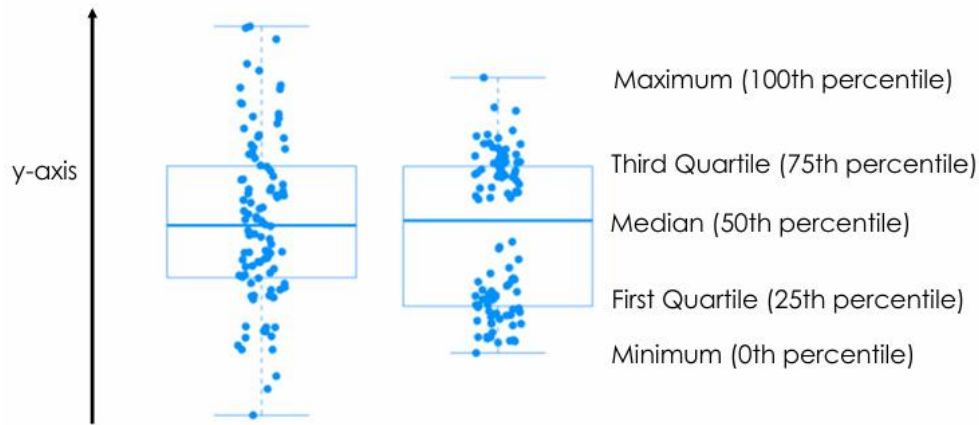
It ensures that the data used in any process—whether for analysis, machine learning (ML), or business decision-making—remains trustworthy, unaltered, and valid.

It is a key aspect of data governance, ensuring that data is not corrupted or modified in unintended ways.

Example:

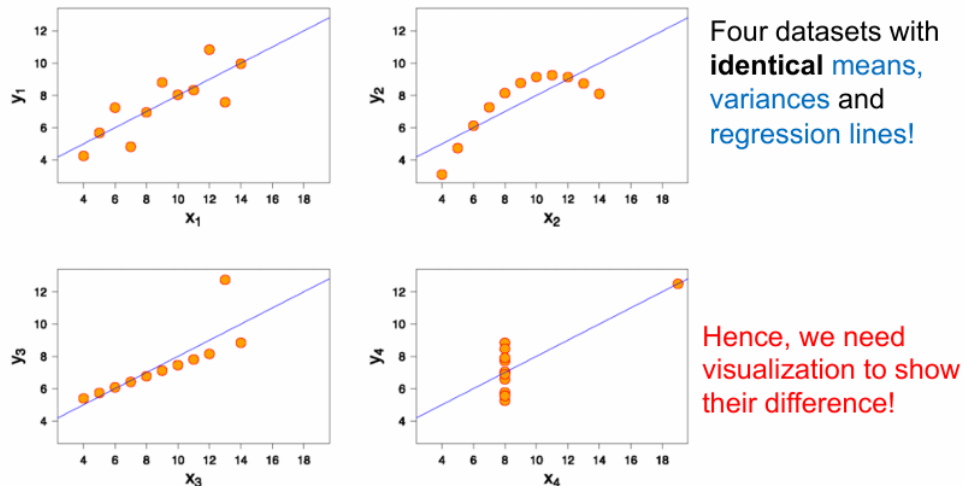
- In a dataset, numeric columns/cells should not accept alphabetic data.
- A binary entry should only allow binary inputs (0 or 1 not both).

Data Visualization: Boxplots (Works for all Data Types)



- The first quartile (Q1) is defined as the middle number between the smallest number (i.e., Minimum) and the median of the data set.
- The third quartile (Q3) is the middle number between the median and the highest value (i.e., Maximum) of the data set.

Why Visualization is Necessary



Practice Questions

	Color	Size	Shape
○	Blue	Large	Ring
▲	Red	Large	Triangle
◇	Orange	Large	Diamond
●	Green	Small	Circle
➡	Yellow	Small	Arrow
■	Blue	Large	Rectangle
●	Red	Large	Circle
◆	Green	Small	Diamond

What are the NOIR data types of *color*, *size*, and *shape* in the table?

Nominal, Ordinal, Nominal

What about their label, *yes/no*?

Yes/No: Nominal

Introduction to Linear Algebra

- A **scalar** is a simple numerical value, like 15 or -3.25 (Focus on real numbers)
- Variables or constants that take scalar values are denoted by an **italic** letter, like x or a
- A **vector** is an ordered list of scalar values, denoted by a **bold** character, e.g. $\mathbf{x}$ or $\mathbf{a}$
- In many books, vectors are written column-wise:

$$\mathbf{a} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} -2 \\ 5 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

- The three vectors above are two-dimensional, or have two elements

- We denote an **entry** or **attribute** of a vector as an italic value with an index, e.g. $a^{(j)}$ or $x^{(j)}$.

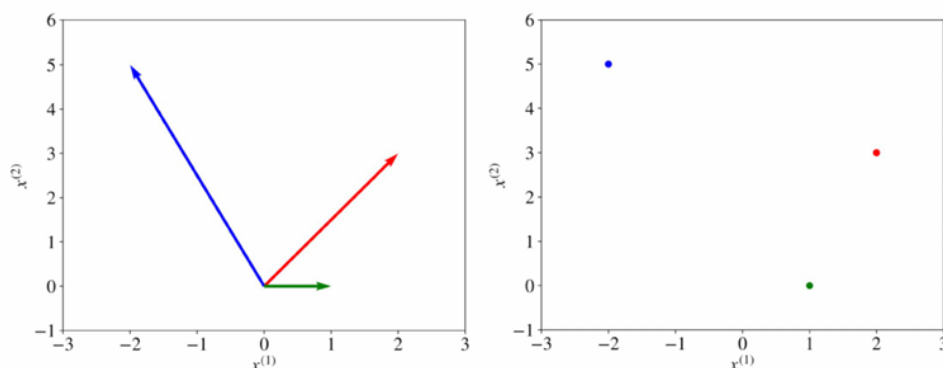
- The index j denotes a specific dimension of the vector, the position of an attribute in the list

$$\mathbf{a} = \begin{bmatrix} a^{(1)} \\ a^{(2)} \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad \text{or more commonly} \quad \mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Note: $x^{(j)}$ is not to be confused with the power operation, e.g., x^2 (squared). Square of an indexed attribute of a vector is denoted as $(x^{(j)})^2$

Vectors can be **visualized** as, in a multi-dimensional space, arrows that point to some directions, or points.

Illustrations of three two-dimensional vectors, $\mathbf{a} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} -2 \\ 5 \end{bmatrix}$, and $\mathbf{c} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$



A **matrix** is a rectangular array of numbers arranged in rows and columns. Denoted with **bold capital** letters, such as **X** or **W**. An example of a matrix with two rows and three columns:

$$\mathbf{X} = \begin{bmatrix} 2 & 4 & -3 \\ 21 & -6 & -1 \end{bmatrix}$$

A **set** is an unordered collection of unique elements. When an element x belongs to a set **S**, we write $x \in \mathbf{S}$. A special set denoted **R** includes all real numbers from minus infinity to plus infinity.

A set $\mathbf{S} \{1,3,5,7,9\}$ is the same as $\{1,5,3,9,7\}$. **Order do not matter**. They just have to have the same unique elements to be identical.

Note: For elements in matrix **X**, we shall use the indexing $x_{1,1}$, where the first and second indices indicate the row and the column position. Usually, for input data, **rows represent samples and columns represent features**. $x_{1,1} = 2$, index don't start from 0.

E.g. Row 1 = Sample 1, Row 2 = Sample 2, Column 1 = Feature 1, Column 2 = Feature 2, Column 3 = Feature 3

Capital Sigma

- **Capital Sigma:** the **summation** over a collection $\{x_1, x_2, x_3, x_4, \dots, x_m\}$ is denoted by:

$$\sum_{i=1}^m x_i = x_1 + x_2 + \dots + x_{m-1} + x_m$$

Capital Pi

- **Capital Pi**: the **product** over a collection $\{x_1, x_2, x_3, x_4, \dots, x_m\}$ is denoted by:

$$\prod_{i=1}^m x_i = x_1 \cdot x_2 \cdot \dots \cdot x_{m-1} \cdot x_m$$

Linear Dependence & Independence

- A collection of d-dimensions d -vectors $\mathbf{x}_1, \dots, \mathbf{x}_m$ (with $m \geq 1$) is called **linearly dependent** if

$$\beta_1 \mathbf{x}_1 + \dots + \beta_m \mathbf{x}_m = 0$$

holds for some $\beta_1, \dots, \beta_m$ that are **not all zero**.

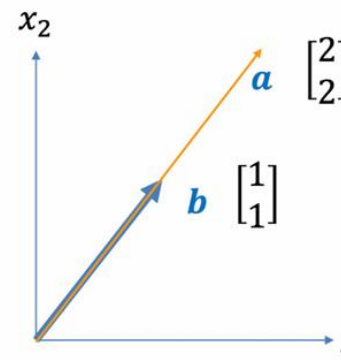
- A collection of d -vectors $\mathbf{x}_1, \dots, \mathbf{x}_m$ (with $m \geq 1$) is called **linearly independent** if it is not linearly dependent, which means that

$$\beta_1 \mathbf{x}_1 + \dots + \beta_m \mathbf{x}_m = 0$$

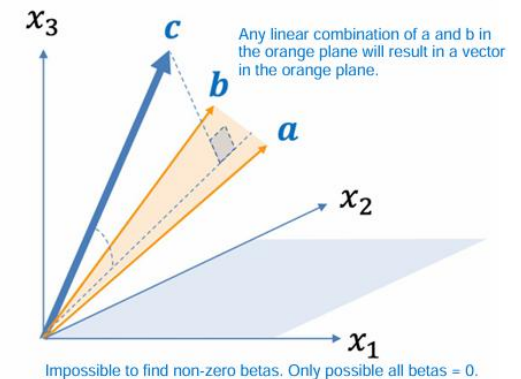
only holds for $\beta_1 = \dots = \beta_m = 0$.

Note: If all rows or columns of a square matrix $\mathbf{X}$ are **linearly independent**, then $\mathbf{X}$ is **invertible**.

Geometry of Dependency and Independence



$$\beta_1 \mathbf{a} + \beta_2 \mathbf{b} = 0$$



Impossible to find non-zero betas. Only possible all betas = 0.

$$\beta_1 \mathbf{a} + \beta_2 \mathbf{b} \neq \beta_3 \mathbf{c}$$

Systems of Linear Equations

These equations can be written compactly in a matrix-vector notation:

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Where

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \dots & x_{m,d} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

Note: Data Matrix = Matrix with rows as samples and columns as features.

- The **data matrix** $\mathbf{X} \in \mathcal{R}^{m \times d}$ and the **target vector** $\mathbf{y} \in \mathcal{R}^m$ are given
- The **unknown vector of parameters** $\mathbf{w} \in \mathcal{R}^d$ is to be learnt
- The $\text{rank}(\mathbf{X})$ corresponds to the maximal number of linearly independent columns/rows of $\mathbf{X}$.

Causality

- Causality, or causation is:
 - The influence by which one event or process (i.e., cause) contributes to another (i.e. effect),
 - The cause is **partly responsible** for the effect, and the effect is **partly dependent** on the cause
- Causality relates to an extremely very wide domain of subjects: philosophy, science, management, humanity.
- Causality research is extremely complex
 - Researcher **can never be completely certain** that there are no other factors influencing the causal relationship.
 - In most cases, we can only say **“probably” causal. Probabilistic.**

Note: Causality refers to a cause-and-effect relationship between two variables. If one variable (the cause) directly influences another variable (the effect), then there is a causal relationship.

- (Probable) causal relations or non-causal?
 - New web design implemented ? Web page traffic increased Probable causal relation.
 - Your height and weight ? Gets A in EE2211 Non-causal relation.
 - Uploaded new app store images ? Downloads increased by 2X Probable causal relation.
 - One works hard and attends lectures/tutorials ? Gets A in EE2211 Probable causal relation.
 - Your favorite color ? Your GPA in NUS Non-causal relation.

How to establish causality?

One popular way to causal data analysis is **Randomized Controlled Trial (RCT)**

- A study design that randomly assigns participants into an experimental group or a control group.
- As the study is conducted, the **only expected difference** between two groups is the outcome variable being studied.
- Example:
 - To decide whether smoking and lung cancer has a causal relation, we put participants into experimental group (people who smoke) and control group (people who don't smoke), and check whether they develop lung cancer eventually.
- RCT is sometimes infeasible to conduct and has moral issues.

Causality is a statistical relationship

- Decades of data show a clear causal relationship between smoking and cancer.
- If one smokes, it is a sure thing that his/her risk of cancer will increase.
- But it is not a sure thing that one will get cancer.
- The relationship is not deterministic.
- The causal effect is real, but it is an effect on your **average risk**.

Correlation

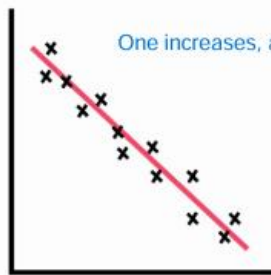
- Correlation measures the degree to which two variables change together.
- If two variables are correlated, it means that they tend to vary in a predictable way, but it **does not imply** that one causes the other.
- Correlation alone cannot prove causation because a third variable (confounding) may influence both correlated variables.
- Even if two variables are related, it may not be clear which one is the cause, and which is the effect.
- In statistics, correlation is **any statistical relationship**, whether causal or not, between two random variables.
- Correlations are useful because they can indicate a predictive relationship that can be exploited in practice.

Both increase/decrease in the same direction.



Positive
Correlation

One increases, and the other decreases.



Negative
Correlation

NOTE: Correlation does NOT imply causation!

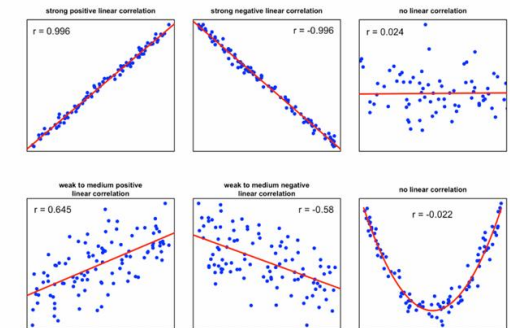
Linear Correlation Coefficient

- Also known as the Pearson Coefficient, computes the degree of correlation.
- It quantifies the **strength** and **direction** of the relationship, ranging from -1 to 1.

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} = \frac{s_{xy}}{s_x s_y}$$

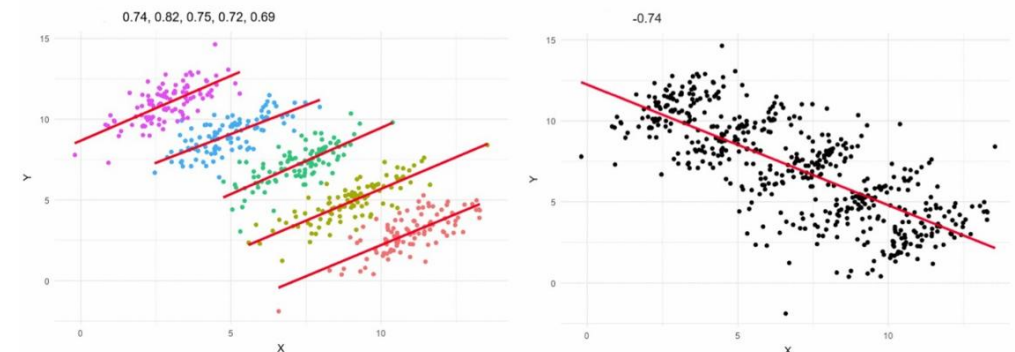
Strong linear relationship	$r > 0.9$
Medium linear relationship	$0.7 < r \leq 0.9$
Weak linear relationship	$0.5 < r \leq 0.7$
No or doubtful linear relationship	$0 < r \leq 0.5$

The same holds for negative values.



Simpson's Paradox

Simpson's paradox is a phenomenon in probability and statistics, in which a trend appears in several different groups of data but **disappears or reverses** when these groups are combined.



The same set of samples!

Example



- Batting Average in professional baseball game
- Two well-known players, Derek Jeter and David Justice

Batter \ Year	1995		1996		Combined	
Derek Jeter	12/48	.250	183/582	.314	195/630	.310
David Justice	104/411	.253	45/140	.321	149/551	.270

of wins # of games

Probability

- We describe a **random experiment** by describing its procedure and observations of its **outcomes**.
- Outcomes are mutual exclusive in the sense that only one outcome occurs in a specific trial of the random experiment.
- This also means an outcome is not decomposable.
- All unique outcomes form a sample space.
- A subset of sample space S , denoted as A , is an **event** in a random experiment $A \subset S$, that is meaningful to an application.
- Example of an event: Faces with numbers no greater than 3.

Axioms of Probability

Assuming events $A \subseteq S$ and $B \subseteq S$, the probabilities of events related with and must satisfy,

1. $Pr(A) \geq 0$
2. $Pr(S) = 1$
3. If $A \cap B = \emptyset$, then $Pr(A \cup B) = Pr(A) + Pr(B)$
 *otherwise, $Pr(A \cup B) = Pr(A) + Pr(B) - Pr(A \cap B)$

Random Variable

- A random variable, usually written as an italic capital letter, like X , is a variable whose possible values are numerical outcomes of a random event.
- There are **two types** of random variables: discrete and continuous.

Notations

- Some books used $P(\cdot)$ and $p(\cdot)$ to distinguish between the probability of discrete random variable and the probability of continuous random variables respectively.
- We shall use $Pr(\cdot)$ for both the above cases

Discrete Random Variable

- A discrete random variable (DRV) takes on only a **countable** number of distinct values such as red, orange, blue or 1, 2, 3.
- The probability distribution of a discrete random variable is described by a list of probabilities associated with each of its possible values.
- This list of probabilities is called a probability mass function (pmf). (Like a histogram, except that here the probabilities sum to 1)

Expectation of DRV

- Let a **discrete random variable** X have k possible values $\{x_i\}_{i=1}^k$.

- The **expectation** of X denoted as $E(x)$ is given by,

$$E(x) \stackrel{\text{def}}{=} \sum_{i=1}^k [x_i \cdot \Pr(X = x_i)] \\ = x_1 \cdot \Pr(X = x_1) + x_2 \cdot \Pr(X = x_2) + \dots + x_k \cdot \Pr(X = x_k)$$

where $\Pr(X = x_i)$ is the probability that X has the value x_i according to the **pmf**.

- The expectation of a random variable is also called the **mean, average** or **expected value** and is frequently denoted with the letter μ .

Standard Deviation of DRV

- Another important statistic is the **standard deviation**, defined as,

$$\sigma \stackrel{\text{def}}{=} \sqrt{E[(X - \mu)^2]} . \quad \text{Variance} = E(X^2) - [E(X)]^2$$

- **Variance**, denoted as σ^2 or $\text{var}(X)$, is defined as,
 $\sigma^2 = E[(X - \mu)^2]$

- For a **discrete random variable**, the standard deviation is given by

$$\sigma = \sqrt{\Pr(X = x_1)(x_1 - \mu)^2 + \dots + \Pr(X = x_k)(x_k - \mu)^2}$$

where $\mu = E(X)$.

Continuous Random Variable

- A continuous random variable (CRV) takes an infinite number of possible values in some interval.

- Examples include height, weight, and time.

- The number of values of a continuous random variable X is infinite, **the probability $\Pr(X = c)$ for any c is 0.**

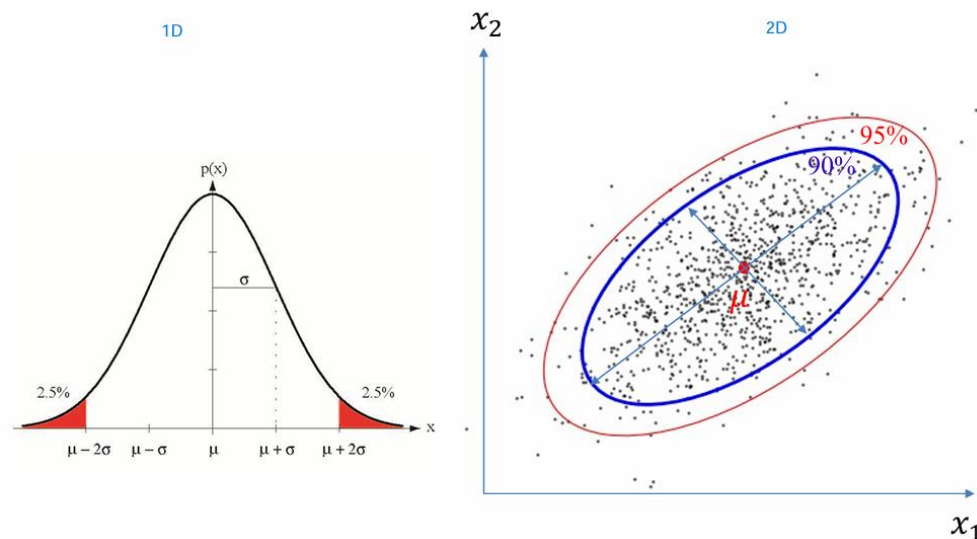
- Therefore, instead of the list of probabilities, the probability distribution of a CRV is described by a probability density function (pdf).

- The pdf is a function whose range is nonnegative and the **area under the curve is equal to 1.**

Expectation & Variance of CRV

- The **expectation** of a **continuous random variable** X is given by $E[x] \stackrel{\text{def}}{=} \int_R x f_X(x) dx$
where f_X is the **pdf** of the variable X and $\int_R$ is the integral of function $x f_X$.
- The **variance** of a **continuous random variable** X is given by $\sigma^2 \stackrel{\text{def}}{=} \int_R (X - \mu)^2 f_X(x) dx$
- **Integral** is an equivalent of the **summation** over all values of the function when the function has a continuous domain.
- It equals the **area under the curve** of the function.
- The property of the pdf that the **area under its curve is 1** mathematically means that $\int_R f_X(x) dx = 1$

Mean and Standard Deviation of a Gaussian Distribution



Example 1

- **Independent random variables**
- Consider tossing a fair coin twice, what is the probability of having (H,H)? Assuming a coin has two sides, H=head and T=Tail
 - $\Pr(x=H, y=H) = \Pr(x=H)\Pr(y=H) = (1/2)(1/2) = 1/4$

Example 2

- **Dependent random variables**
- Given 2 balls with different colors (**Red** and Black), what is the probability of first drawing B and then **R**? Assuming we are drawing the balls **without replacement**.
- The space of outcomes of taking two balls sequentially without replacement:
 - B-R, R-B
 - Thus having B-R is 1/2 .
- Mathematically:
 - $\Pr(x=B, y=R) = \Pr(y=R | x=B) \Pr(x=B) = 1 \times (1/2) = 1/2$

Conditional Probability

Example 3

- **Dependent random variables**

- Given 3 balls with different colors (R,G,B), and we draw 2 balls. What is the probability of first having B and then G, if we draw **without replacement**?
- The space of outcomes of taking two balls sequentially without replacement:

R-G | G-B | B-R

R-B | G-R | B-G Thus, $\Pr(y=G, x=B) = 1/6$

- Mathematically:

$$\begin{aligned}\Pr(y=G, x=B) &= \Pr(y=G | x=B) \Pr(x=B) \\ &= (1/2) \times (1/3) \\ &= 1/6\end{aligned}$$

Two Basic Rules

- Sum Rule and Product Rule (Conditional Probability)

- Sum Rule

Suppose we have two dice and want to find the probability of rolling a sum of 7. The event "sum = 7" can happen in different ways: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1). Use sum rule and add up the probabilities of all these individual events to get probability of rolling a sum of 7.

$$\Pr(X = x) = \sum_Y \Pr(X = x, Y = y_i)$$

- Product Rule

Also known as Conditional Probability Rule.

$$\Pr(X = x, Y = y) = \Pr(Y = y | X = x) \Pr(X = x)$$

$$P(A \cap B) = P(A | B) * P(B)$$

Bayes' Rule

- The conditional probability $\Pr(Y = y | X = x)$ is the probability of the random variable Y to have a specific value y , given that another random variable X has a specific value of x .
- The **Bayes' Rule** (also known as the **Bayes' Theorem**):

$$\Pr(Y = y | X = x) = \frac{\Pr(X = x | Y = y) \Pr(Y = y)}{\Pr(X = x)}$$

posterior
likelihood
prior
evidence

Commonly used when qn gives $P(A | B)$ and asked to find $P(B | A)$

Terms Involved: Posterior, Likelihood, Prior, Evidence.

Example 1

- Drawing a sample of fruit from a box
 - First pick a box, and then draw a sample of fruit from it
 - B: variable for Box, can be r (red) or b (blue)
 - F: variable for Fruit, can be o (orange) or a (apple)

- $\Pr(B=r)=0.4$

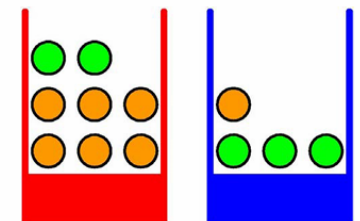
prior

- $\Pr(F=o | B=r) = 0.75$

likelihood

- $\Pr(F=o) = 0.45$

evidence



- $\Pr(B=r | F=o) = \Pr(F=o | B=r) * \Pr(B=r) / \Pr(F=o)$

$$= 0.75 * 0.4 / 0.45 = 2/3$$

posterior

$$\Pr(B=o | F=o) = 1 - 2/3 = 1/3$$

Example 2

Suppose the random variable X has the following probability mass function (pmf) listed in the table below. k is unknown.

X	1	2	3	4	5
Pr[X]	0.1	0.05	0.05	0.6	k 0.2

What is the probability that X takes a value of odd numbers?

$$0.1 + 0.05 + 0.2 = 0.35$$

Operations on Vectors and Matrices

Addition and Subtraction

$$\mathbf{x} + \mathbf{y} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \end{bmatrix}$$

$$\mathbf{x} - \mathbf{y} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} x_1 - y_1 \\ x_2 - y_2 \end{bmatrix}$$

Scalar Multiplication

$$a \mathbf{x} = a \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} ax_1 \\ ax_2 \end{bmatrix}$$

$$\frac{1}{a} \mathbf{x} = \frac{1}{a} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \frac{1}{a} x_1 \\ \frac{1}{a} x_2 \end{bmatrix}$$

Matrix or Vector Transpose

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{x}^T = [x_1 \quad x_2]$$

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix}, \quad \mathbf{X}^T = \begin{bmatrix} x_{1,1} & x_{2,1} & x_{3,1} \\ x_{1,2} & x_{2,2} & x_{3,2} \\ x_{1,3} & x_{2,3} & x_{3,3} \end{bmatrix}$$

Dot Product or Inner Product of Vectors

Dot Product for VECTORS!! Not matrices. As long as both vectors have same number of elements and are 1D arrays

Dot Product or Inner Product of Vectors:

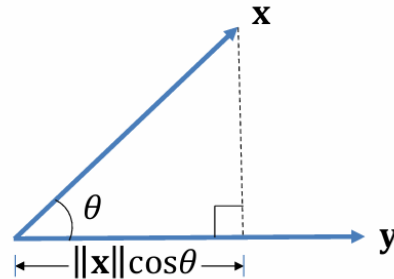
$$\begin{aligned}\mathbf{x} \cdot \mathbf{y} &= \mathbf{x}^T \mathbf{y} \\ &= [x_1 \ x_2] \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \\ &= x_1 y_1 + x_2 y_2\end{aligned}$$

Geometric definition:

$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos\theta$$

where θ is the angle between $\mathbf{x}$ and $\mathbf{y}$,
and $\|\mathbf{x}\| = \sqrt{\mathbf{x} \cdot \mathbf{x}}$ is the Euclidean length of vector $\mathbf{x}$

E.g. $\mathbf{a} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \mathbf{c} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow \mathbf{a} \cdot \mathbf{c} = 2*1 + 3*0 = 2$



Matrix-Vector Product

$$\begin{aligned}\mathbf{W}\mathbf{x} &= \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \\ &= \begin{bmatrix} w_{1,1}x_1 + w_{1,2}x_2 + w_{1,3}x_3 \\ w_{2,1}x_1 + w_{2,2}x_2 + w_{2,3}x_3 \end{bmatrix}\end{aligned}$$

Vector-Matrix Product

$$\begin{aligned}\mathbf{x}^T \mathbf{W} &= [x_1 \ x_2] \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \\ &= [(x_1 w_{1,1} + x_2 w_{2,1}) \quad (x_1 w_{1,2} + x_2 w_{2,2}) \quad (x_1 w_{1,3} + x_2 w_{2,3})]\end{aligned}$$

Matrix-Matrix Product

$$\begin{aligned}\mathbf{X}\mathbf{W} &= \begin{bmatrix} x_{1,1} & \dots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{m,1} & \dots & x_{m,d} \end{bmatrix} \begin{bmatrix} w_{1,1} & \dots & w_{1,h} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \dots & w_{d,h} \end{bmatrix} \\ &= \begin{bmatrix} (x_{1,1}w_{1,1} + \dots + x_{1,d}w_{d,1}) & \dots & (x_{1,1}w_{1,h} + \dots + x_{1,d}w_{d,h}) \\ \vdots & \ddots & \vdots \\ (x_{m,1}w_{1,1} + \dots + x_{m,d}w_{d,1}) & \dots & (x_{m,1}w_{1,h} + \dots + x_{m,d}w_{d,h}) \end{bmatrix} \\ &= \begin{bmatrix} \sum_{i=1}^d x_{1,i}w_{i,1} & \dots & \sum_{i=1}^d x_{1,i}w_{i,h} \\ \vdots & \ddots & \vdots \\ \sum_{i=1}^d x_{m,i}w_{i,1} & \dots & \sum_{i=1}^d x_{m,i}w_{i,h} \end{bmatrix}\end{aligned}$$

If $\mathbf{X}$ is $m \times d$ and $\mathbf{W}$ is $d \times h$, then the outcome is a $m \times h$ matrix

Matrix Inverse

Definition:

Every invertible matrix is non-singular (because invertibility requires a nonzero determinant).
Every non-singular matrix is invertible (because a nonzero determinant ensures the inverse exists).

A d -by- d square matrix **A** is **invertible** (also **nonsingular**)

if there exists a d -by- d square matrix **B** such that

$$\begin{aligned} \mathbf{A} * \mathbf{B} &= \mathbf{I} \\ \mathbf{A} &= \mathbf{C} * \mathbf{B}^{-1} \end{aligned}$$

$\mathbf{AB} = \mathbf{BA} = \mathbf{I}$ (identity matrix)

Only square matrix can be invertible. A non-square matrix has no inverse.

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & 0 & & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & & 1 & 0 \\ 0 & 0 & \dots & 0 & 1 \end{bmatrix} \quad d\text{-by-}d \text{ dimension}$$

Matrix Inverse Computation

Matrix inverse computation

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \text{adj}(\mathbf{A})$$

- $\det(\mathbf{A})$ is the **determinant** of **A**
- $\text{adj}(\mathbf{A})$ is the **adjugate** or **adjoint** of **A**

Determinant Computation

Example: 2x2 matrix

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\det(\mathbf{A}) = |\mathbf{A}| = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

Determinant computation $\det(\mathbf{A}) = \sum_{j=1}^k (-1)^{i+j} a_{ij} M_{ij}$

Example: 3x3 matrix, use the first row ($i = 1$)

$$\begin{aligned} |\mathbf{A}| &= \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \begin{vmatrix} \square & \square & \square \\ \square & e & f \\ \square & h & i \end{vmatrix} - b \begin{vmatrix} \square & \square & \square \\ d & \square & f \\ g & \square & i \end{vmatrix} + c \begin{vmatrix} \square & \square & \square \\ d & e & \square \\ g & h & \square \end{vmatrix} \\ &= a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix} \\ &= a(ei - fh) - b(di - fg) + c(dh - eg) \end{aligned}$$

Adjugate & Minor

- $\text{adj}(\mathbf{A})$ is the **adjugate or adjoint of A**
- $\text{adj}(\mathbf{A})$ is the transpose of the **cofactor matrix C** of $\mathbf{A} \rightarrow \text{adj}(\mathbf{A}) = \mathbf{C}^T$
- **Minor** of an element in a matrix $\mathbf{A}$ is defined as the **determinant** obtained by deleting the row and column in which that element lies

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \text{Minor of } a_{12} \text{ is } M_{12} = \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix}$$

- The (i, j) entry of the **cofactor matrix C** is the minor of (i, j) element times a **sign** factor

$$\text{Cofactor } C_{ij} = (-1)^{i+j} M_{ij}$$

- The **determinant** of $\mathbf{A}$ can also be defined by minors as

$$\det(\mathbf{A}) = \sum_{j=1}^k a_{ij} C_{ij} = (-1)^{i+j} a_{ij} M_{ij}$$

Computing Inverse Using Determinant & Adjugate/Adjoint

Consider a 3x3 matrix

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}. \quad \text{The minor of } a_{22} = \begin{vmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{vmatrix}$$

Its cofactor matrix is

$$\mathbf{C} = \begin{pmatrix} + \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} & - \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} & + \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix} \\ - \begin{vmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{vmatrix} & + \begin{vmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{vmatrix} & - \begin{vmatrix} a_{11} & a_{12} \\ a_{31} & a_{32} \end{vmatrix} \\ + \begin{vmatrix} a_{12} & a_{13} \\ a_{22} & a_{23} \end{vmatrix} & - \begin{vmatrix} a_{11} & a_{13} \\ a_{21} & a_{23} \end{vmatrix} & + \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} \end{pmatrix} \quad \text{adj}(\mathbf{A}) = \mathbf{C}^T$$

$$\det(\mathbf{A}) = \sum_{j=1}^k a_{ij} C_{ij} = (-1)^{i+j} a_{ij} M_{ij}$$

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \text{adj}(\mathbf{A})$$

$$\text{Minor of } a_{12} \text{ is } M_{12} = \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} \quad \text{adj}(\mathbf{A}) = \mathbf{C}^T$$

$$\text{Cofactor } C_{ij} = (-1)^{i+j} M_{ij} \quad \det(\mathbf{A}) = \sum_{j=1}^k (-1)^{i+j} a_{ij} M_{ij}$$

ONLY square matrix has cofactor matrix.

$$\text{E.g. } \mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad \mathbf{C} = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

To find the cofactor matrix, go through each element of the ENTIRE MATRIX and remove the whole row and column in which the element resides. Then, for the remaining rows and columns (minor), calculate the determinant for it and multiply it with $(-1)^{(\text{row} + \text{column of the element})}$.

$$\text{adj}(\mathbf{A}) = \mathbf{C}^T = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad \det(\mathbf{A}) = |\mathbf{A}| = ad - bc$$

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \text{adj}(\mathbf{A}) = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Example

$$\text{Find the cofactor matrix of } \mathbf{A} \text{ given that } \mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix}.$$

Solution:

$$\begin{aligned} a_{11} &\Rightarrow \begin{vmatrix} 4 & 5 \\ 0 & 6 \end{vmatrix} = 24, & a_{12} &\Rightarrow - \begin{vmatrix} 0 & 5 \\ 1 & 6 \end{vmatrix} = 5, & a_{13} &\Rightarrow \begin{vmatrix} 0 & 4 \\ 1 & 0 \end{vmatrix} = -4, \\ a_{21} &\Rightarrow - \begin{vmatrix} 2 & 3 \\ 0 & 6 \end{vmatrix} = -12, & a_{22} &\Rightarrow \begin{vmatrix} 1 & 3 \\ 1 & 6 \end{vmatrix} = 3, & a_{23} &\Rightarrow - \begin{vmatrix} 1 & 2 \\ 1 & 0 \end{vmatrix} = 2, \\ a_{31} &\Rightarrow \begin{vmatrix} 2 & 3 \\ 4 & 5 \end{vmatrix} = -2, & a_{32} &\Rightarrow - \begin{vmatrix} 1 & 3 \\ 0 & 5 \end{vmatrix} = -5, & a_{33} &\Rightarrow \begin{vmatrix} 1 & 2 \\ 0 & 4 \end{vmatrix} = 4, \end{aligned}$$

$$\text{The cofactor matrix C is thus } \begin{bmatrix} 24 & 5 & -4 \\ -12 & 3 & 2 \\ -2 & -5 & 4 \end{bmatrix}.$$

Systems of Linear Equations (In-Depth)

- Consider a system of m linear equations with d variables or unknowns $w_1, \dots, w_d$:
No. of samples = No. of equations (m)
No. of features = No. of variables (d)

$$\begin{aligned}x_{1,1}w_1 + x_{1,2}w_2 + \dots + x_{1,d}w_d &= y_1 \\x_{2,1}w_1 + x_{2,2}w_2 + \dots + x_{2,d}w_d &= y_2 \\&\vdots \\x_{m,1}w_1 + x_{m,2}w_2 + \dots + x_{m,d}w_d &= y_m.\end{aligned}$$

These equations can be written compactly in matrix-vector notation:

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Where

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \dots & x_{m,d} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

Note:

- The data matrix $\mathbf{X} \in \mathcal{R}^{m \times d}$ and the target vector $\mathbf{y} \in \mathcal{R}^m$ are given
- The unknown vector of parameters $\mathbf{w} \in \mathcal{R}^d$ is to be learnt

How many solutions?

A set of linear equations can have no solution, one solution, or multiple solutions:

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Where

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \dots & x_{m,d} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

X is Square	Even-determined	$m = d$	Equal number of equations and unknowns
X is Tall	Over-determined	$m > d$	More number of equations than unknowns
X is Wide	Under-determined	$m < d$	Fewer number of equations than unknowns

Square or Even-Determined System (rows = columns of X)

$$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m \times d}, \quad \mathbf{w} \in \mathcal{R}^{d \times 1}, \quad \mathbf{y} \in \mathcal{R}^{m \times 1}$$

1. Square or even-determined system: $m = d$

- Equal number of equations and unknowns, i.e., $\mathbf{X} \in \mathcal{R}^{d \times d}$
- One unique solution** if $\mathbf{X}$ is invertible or all rows/columns of $\mathbf{X}$ are linearly independent Else, infinite or no solutions.
- If all rows or columns of $\mathbf{X}$ are linearly independent, then $\mathbf{X}$ is invertible.

Solution:

If $\mathbf{X}$ is invertible (or $\mathbf{X}^{-1}\mathbf{X} = \mathbf{I}$), then pre-multiply both sides by $\mathbf{X}^{-1}$

$$\begin{aligned}\mathbf{X}^{-1}\mathbf{X}\mathbf{w} &= \mathbf{X}^{-1}\mathbf{y} \\ \Rightarrow \hat{\mathbf{w}} &= \mathbf{X}^{-1}\mathbf{y}\end{aligned}$$

(Note: we use a *hat* on top of $\mathbf{w}$ to indicate that it is a specific point in the space of $\mathbf{w}$)

Example 1 $w_1 + w_2 = 4$ (1) Two unknowns
 $w_1 - 2w_2 = 1$ (2) Two equations

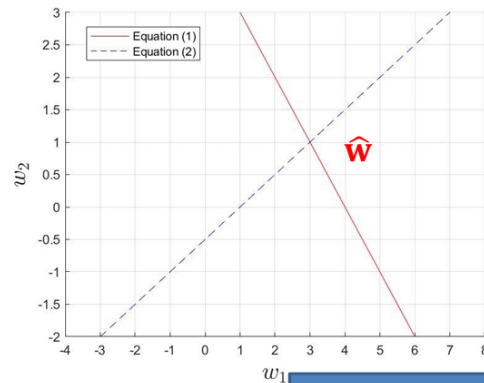
$$\begin{bmatrix} 1 & 1 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

$$\hat{\mathbf{w}} = \mathbf{X}^{-1} \mathbf{y}$$

$$= \begin{bmatrix} 1 & 1 \\ 1 & -2 \end{bmatrix}^{-1} \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

$$= \frac{-1}{3} \begin{bmatrix} -2 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \text{adj}(\mathbf{A}) \quad \text{adj}(\mathbf{A}) = \mathbf{C}^T = \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad \det(\mathbf{A}) = ad - bc$$



Python demo 3

Over-Determined System (rows > columns of X)

$$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m \times d}, \quad \mathbf{w} \in \mathcal{R}^{d \times 1}, \quad \mathbf{y} \in \mathcal{R}^{m \times 1}$$

2. Over-determined system: $m > d$

- More equations than unknowns
- $\mathbf{X}$ is non-square (tall) and hence not invertible
- **Has no exact solution in general ***
- An **approximated solution** is available using the left inverse

If the **left-inverse** of $\mathbf{X}$ exists such that $\mathbf{X}^\dagger \mathbf{X} = \mathbf{I}$, then pre-multiply both sides by $\mathbf{X}^\dagger$ results in

$$\mathbf{X}^\dagger \mathbf{X} \mathbf{w} = \mathbf{X}^\dagger \mathbf{y} \Rightarrow \hat{\mathbf{w}} = \mathbf{X}^\dagger \mathbf{y}$$

For $\mathbf{X}$ to have a left inverse, it must have full column rank.
Rank($\mathbf{X}$) = No. of columns

Definition:

A matrix $\mathbf{B}$ that satisfies $\mathbf{B}_{d \times m} \mathbf{A}_{m \times d} = \mathbf{I}$ is called a **left-inverse** of $\mathbf{A}$.

The **left-inverse** of $\mathbf{X}$: $\mathbf{X}^\dagger = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ given $\mathbf{X}^T \mathbf{X}$ is invertible.

[$\mathbf{X}, \mathbf{y}$] = Appending the target vector $\mathbf{y}$ as an additional column to $\mathbf{X}$.

Note: * exception: when $\text{rank}(\mathbf{X}) = \text{rank}([\mathbf{X}, \mathbf{y}])$, there is a solution.

Example 2 $w_1 + w_2 = 1$ (1) Two unknowns
 $w_1 - w_2 = 0$ (2) Three equations
 $w_1 = 2$ (3)

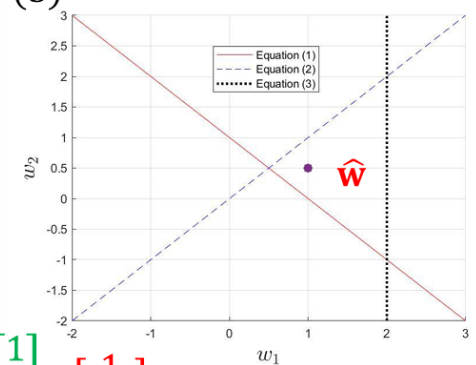
$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$$

No exact solution
 Approximated solution

$$\hat{\mathbf{w}} = \mathbf{X}^\dagger \mathbf{y} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

$$= \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$$

$\mathbf{X}^T \mathbf{X}$ is invertible



Python demo 4

Under-Determined System (rows < columns of X)

$$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m \times d}, \quad \mathbf{w} \in \mathcal{R}^{d \times 1}, \quad \mathbf{y} \in \mathcal{R}^{m \times 1}$$

3. Under-determined system: $m < d$

- More unknowns than equations
- Infinite number of solutions in general *

For $\mathbf{X}$ to have a right inverse, it must have full row rank.
Rank($\mathbf{X}$) = No. of rows

If the **right-inverse** of $\mathbf{X}$ exists such that $\mathbf{X}\mathbf{X}^\dagger = \mathbf{I}$, then the d -vector $\mathbf{w} = \mathbf{X}^\dagger \mathbf{y}$ (one of the infinite cases) satisfies the equation $\mathbf{X}\mathbf{w} = \mathbf{y}$, i.e.,

$$\begin{aligned} \text{^ formula for under-determined system } \mathbf{X}\mathbf{w} = \mathbf{y} &\Rightarrow \mathbf{X}\mathbf{X}^\dagger \mathbf{y} = \mathbf{y} \\ &\Rightarrow \mathbf{I}\mathbf{y} = \mathbf{y} \end{aligned}$$

Definition:

A matrix $\mathbf{B}$ that satisfies $\mathbf{A}_{m \times d} \mathbf{B}_{d \times m} = \mathbf{I}$ is called a **right-inverse** of $\mathbf{A}$.

The **right-inverse** of $\mathbf{X}$: $\mathbf{X}^\dagger = \mathbf{X}^T (\mathbf{X}\mathbf{X}^T)^{-1}$ given $\mathbf{X}\mathbf{X}^T$ is invertible.

If $\mathbf{X}$ is right-invertible, we can **find a unique constrained solution**.

Note: * exception: no solution if the system is inconsistent $\text{rank}(\mathbf{X}) < \text{rank}([\mathbf{X}, \mathbf{y}])$

Derivation of Right-Inverse

Derivation:

$$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m \times d}, \quad \mathbf{w} \in \mathcal{R}^{d \times 1}, \quad \mathbf{y} \in \mathcal{R}^{m \times 1}$$

A unique solution is yet possible by constraining the search using $\mathbf{w} = \mathbf{X}^T \mathbf{a}$

If $\mathbf{X}\mathbf{X}^T$ is invertible, let $\mathbf{w} = \mathbf{X}^T \mathbf{a}$, then

$$\begin{aligned} \mathbf{X}\mathbf{X}^T \mathbf{a} &= \mathbf{y} \\ \Rightarrow \hat{\mathbf{a}} &= (\mathbf{X}\mathbf{X}^T)^{-1} \mathbf{y} \\ \Rightarrow \hat{\mathbf{w}} &= \mathbf{X}^T \hat{\mathbf{a}} = \underbrace{\mathbf{X}^T (\mathbf{X}\mathbf{X}^T)^{-1}}_{\mathbf{X}^\dagger} \mathbf{y} \end{aligned}$$

right-inverse

Example 3 $w_1 + 2w_2 + 3w_3 = 2$ (1) Three unknowns
 $w_1 - 2w_2 + 3w_3 = 1$ (2) Two equations

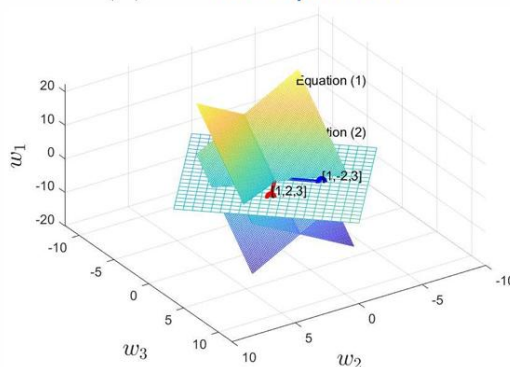
$$\begin{bmatrix} 1 & 2 & 3 \\ 1 & -2 & 3 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Infinitely many solutions along the intersection line

Here $\mathbf{X}\mathbf{X}^T$ is invertible

$$\begin{aligned} \hat{\mathbf{w}} &= \mathbf{X}^T (\mathbf{X}\mathbf{X}^T)^{-1} \mathbf{y} \\ &= \begin{bmatrix} 1 & 1 \\ 2 & -2 \\ 3 & 3 \end{bmatrix} \begin{bmatrix} 14 & 6 \\ 6 & 14 \end{bmatrix}^{-1} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.25 \\ 0.45 \end{bmatrix} \end{aligned}$$

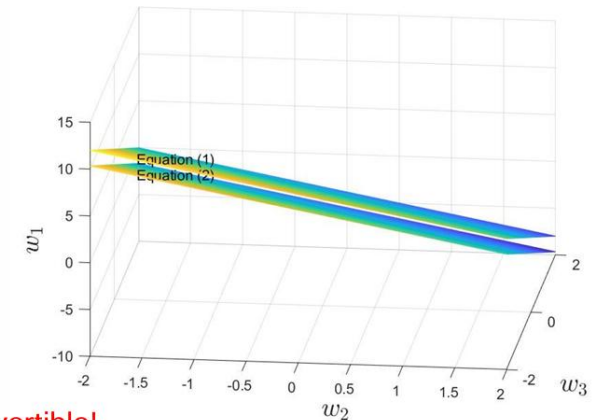
Constrained solution



No Solution

Example 4 $w_1 + 2w_2 + 3w_3 = 2$ (1) Three unknowns
 $3w_1 + 6w_2 + 9w_3 = 1$ (2) Two equations

$$\begin{bmatrix} 1 & 2 & 3 \\ 3 & 6 & 9 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$



Both $\mathbf{X}\mathbf{X}^T$ and $\mathbf{X}^T\mathbf{X}$ are not invertible!

There is no solution for the system

Set

- A set is an unordered collection of unique elements
 - Denoted as a calligraphic capital character e.g., $\mathcal{S}, \mathcal{R}, \mathcal{N}$ etc
 - When an element x belongs to a set $\mathcal{S}$, we write $x \in \mathcal{S}$
- A set of numbers can be finite - include a fixed amount of values - Denoted using **accolades**, e.g. $\{1, 3, 18, 23, 235\}$ or $\{x_1, x_2, x_3, x_4, \dots, x_d\}$
- A set can be **infinite** and include all values in some interval
 - If a set of real numbers includes all values between a and b, including a and b, it is denoted using square brackets as $[a, b]$
 - If the set does not include the values a and b, it is denoted using parentheses as (a, b)

- Examples:

- The special set denoted by $\mathcal{R}$ includes all real numbers from minus infinity to plus infinity.

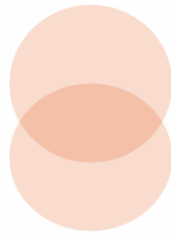
- The set $[0, 1]$ includes values like 0, 0.0001, 0.25, 0.9995, and 1.0

Set Operations

- **Intersection** of two sets:

$$\mathcal{S}_3 \leftarrow \mathcal{S}_1 \cap \mathcal{S}_2$$

$$\text{Example: } \{1, 3, 5, 8\} \cap \{1, 8, 4\} = \{1, 8\}$$



- **Union** of two sets:

$$\mathcal{S}_3 \leftarrow \mathcal{S}_1 \cup \mathcal{S}_2$$

$$\text{Example: } \{1, 3, 5, 8\} \cup \{1, 8, 4\} = \{1, 3, 4, 5, 8\}$$



Functions

- A function is a relation that associates each element x of a set $\mathcal{X}$, the **domain** of the function, to a single element y of another set $\mathcal{Y}$, the **codomain** of the function

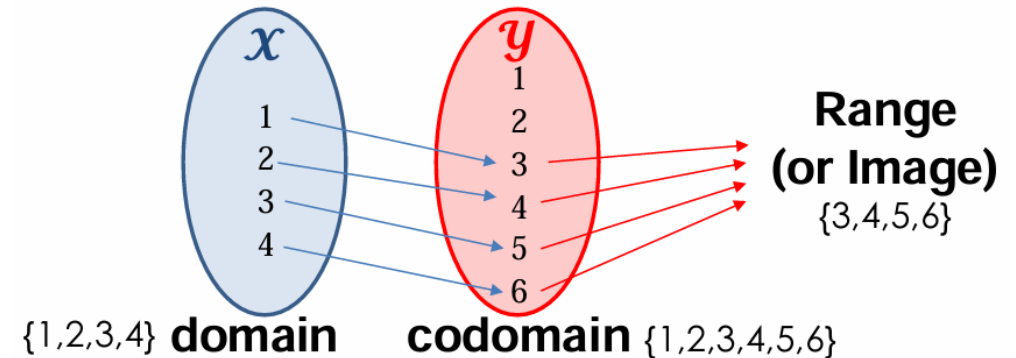
- If the function is called f , this relation is denoted $y = f(x)$

- The element x is the argument or input of the function

- y is the value of the function or the output

- The symbol used for representing the input is the variable of the function

- $f(x)$ f is a function of the variable x ; $f(x, w)$ f is a function of the variable x and w .



Scalar & Vector Functions

- A scalar function is a function that maps one or more inputs (which may be scalars, vectors, or matrices) to a single scalar output.

- A vector function is a function that maps inputs (which may be scalars, vectors, or matrices) to a vector output.

- A **scalar function** can have vector argument

- E.g. $y = f(\mathbf{x}) = x_1 + x_2 + 2x_3$

- A **vector function**, denoted as $\mathbf{y} = \mathbf{f}(\mathbf{x})$ is a function that returns a vector $\mathbf{y}$

- Input argument can be a **vector** $\mathbf{y} = \mathbf{f}(\mathbf{x})$ or a **scalar** $y = f(x)$

- E.g. $\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -x_1 \\ x_2 \end{bmatrix}$

- E.g. $\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -2x_1 \\ 3x_1 \end{bmatrix}$

Maps inputs in the domain to outputs in the codomain.

- The notation $f: \mathcal{R}^d \rightarrow \mathcal{R}$ means that f is a function that maps real d -vectors to real numbers
 - i.e., f is a scalar-valued function of d -vectors
- If $\mathbf{x}$ is a d -vector argument, then $f(\mathbf{x})$ denotes the value of the function f at $\mathbf{x}$
 - i.e., $f(\mathbf{x}) = f(x_1, x_2, \dots, x_d)$, $\mathbf{x} \in \mathcal{R}^d$, $f(\mathbf{x}) \in \mathcal{R}$

Input is 4-d even though x_3 is missing.

- Example: we can define a function $f: \mathcal{R}^4 \rightarrow \mathcal{R}$ by
$$f(\mathbf{x}) = x_1 + x_2 - x_4^2$$

Inner Product Function

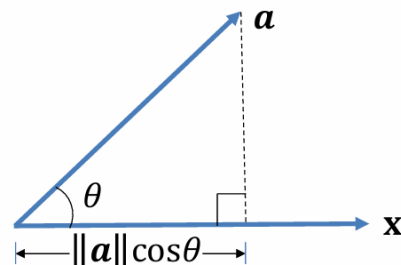
- Suppose $\mathbf{a}$ is a d -vector. We can define a scalar valued function f of d -vectors, given by

$$f(\mathbf{x}) = \mathbf{a}^T \mathbf{x} = a_1 x_1 + a_2 x_2 + \dots + a_d x_d \quad (1)$$

for any d -vector $\mathbf{x}$

Basically dot 2 vectors

- The inner product of its d -vector argument $\mathbf{x}$ with some (fixed) d -vector $\mathbf{a}$
- We can also think of f as forming a **weighted sum** of the elements of $\mathbf{x}$; the elements of $\mathbf{a}$ give the weights



Linear Functions

A function $f: \mathcal{R}^d \rightarrow \mathcal{R}$ is **linear** if it satisfies the following two properties:

- **Homogeneity**
 - For any d -vector $\mathbf{x}$ and any scalar α , $f(\alpha \mathbf{x}) = \alpha f(\mathbf{x})$
 - **Scaling** the (vector) argument is the same as scaling the function value
- **Additivity**
 - For any d -vectors $\mathbf{x}$ and $\mathbf{y}$, $f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$
 - **Adding** (vector) arguments is the same as adding the function values

Superposition & Linearity

- The inner product function $f(\mathbf{x}) = \mathbf{a}^T \mathbf{x}$ defined in equation (1) (slide 42) satisfies the property

$$\begin{aligned} f(\alpha \mathbf{x} + \beta \mathbf{y}) &= \mathbf{a}^T (\alpha \mathbf{x} + \beta \mathbf{y}) \\ &= \mathbf{a}^T (\alpha \mathbf{x}) + \mathbf{a}^T (\beta \mathbf{y}) \\ &= \alpha (\mathbf{a}^T \mathbf{x}) + \beta (\mathbf{a}^T \mathbf{y}) \\ &= \alpha f(\mathbf{x}) + \beta f(\mathbf{y}) \end{aligned}$$

for all d -vectors $\mathbf{x}$, $\mathbf{y}$, and all scalars α , β .

- This property is called **superposition**, which consists of **homogeneity** and **additivity**
- A **function** that satisfies the superposition property is called **linear**
- If a function f is **linear**, superposition extends to linear combinations of any number of vectors:

$$f(\alpha_1 \mathbf{x}_1 + \dots + \alpha_k \mathbf{x}_k) = \alpha_1 f(\mathbf{x}_1) + \dots + \alpha_k f(\mathbf{x}_k)$$

for any d vectors $\mathbf{x}_1 + \dots + \mathbf{x}_k$, and any scalars $\alpha_1 + \dots + \alpha_k$.

Equation (1) refers to the equation in Inner Product Function.

Affine Functions

A linear function plus a constant is called an affine function

A linear function $f: \mathcal{R}^d \rightarrow \mathcal{R}$ is **affine** if and only if it can be expressed as $f(\mathbf{x}) = \mathbf{a}^T \mathbf{x} + b$ for some d -vector $\mathbf{a}$ and scalar b , which is called the **offset (or bias)**

Linear function usually passes through the origin. Affine functions generally move up and down, giving the offset.

Example:

$$f(\mathbf{x}) = 2.3 - 2x_1 + 1.3x_2 - x_3$$

This function is affine, with $b = 2.3$, $\mathbf{a}^T = [-2, 1.3, -1]$.

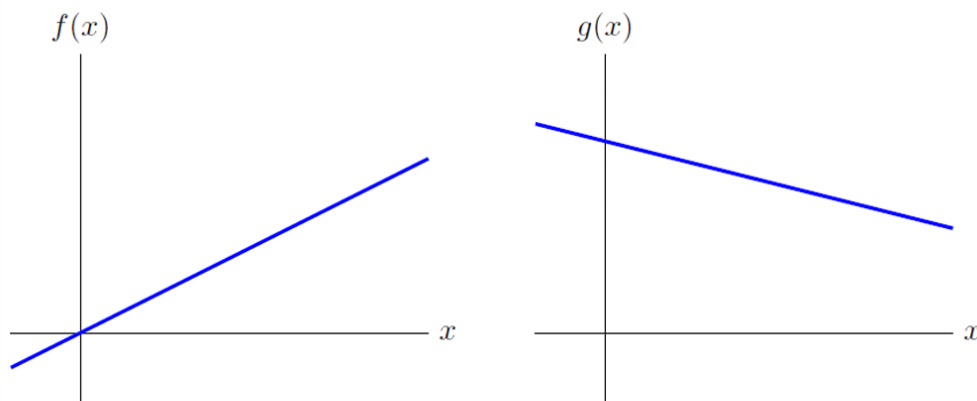


Figure 2.1 Left. The function f is linear. Right. The function g is affine, but not linear.

Quick Summary

- Operations on Vectors and Matrices
 - Dot-product, matrix inverse
- Systems of Linear Equations $\mathbf{X}\mathbf{w} = \mathbf{y}$
 - Matrix-vector notation, linear dependency, invertible
 - Even-, over-, under-determined linear systems
- Set and Functions

Assignment 1 (week 6 Fri)
Tutorial 4

$\mathbf{X}$ is Square	Even-determined	$m = d$	One unique solution in general	$\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$
$\mathbf{X}$ is Tall	Over-determined	$m > d$	No exact solution in general; An approximated solution	$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ Left-inverse
$\mathbf{X}$ is Wide	Under-determined	$m < d$	Infinite number of solutions in general; Unique constrained solution	$\hat{\mathbf{w}} = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T)^{-1} \mathbf{y}$ Right-inverse

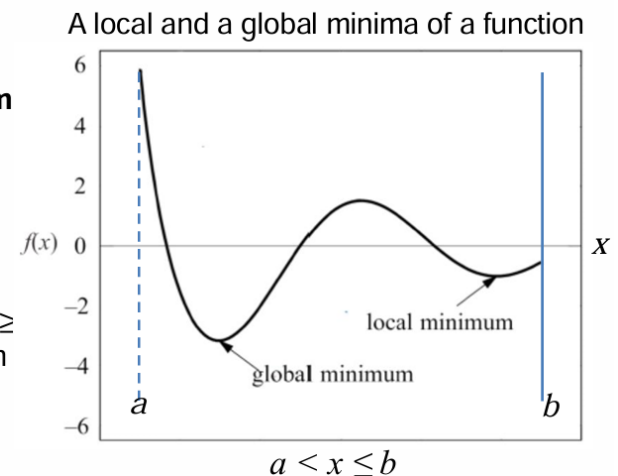
- Scalar and vector functions
- Inner product function
- Linear and affine functions

python package *numpy*
Inverse: *numpy.linalg.inv(X)*
Transpose: *X.T*

Maximum & Minimum

- $f(x)$ has a **local minimum** at $x = c$ if $f(x) \geq f(c)$ for every x in some open interval around $x = c$
- $f(x)$ has a **global minimum** at $x = c$ if $f(x) \geq f(c)$ for all x in the domain of f

The problem of finding the minimum or maximum of a certain function is called optimization problem.



Note: An **interval** is a set of real numbers with the property that any number that lies between two numbers in the set is also included in the set.
An **open interval** does not include its endpoints and is denoted using parentheses. E.g. $(0, 1)$ means "all numbers greater than 0 and less than 1".

Max/Min & Arg Max/Min

Max and Arg Max

- Given a set of values $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$,
- The operator $\max_{a \in \mathcal{A}} f(a)$ returns the highest value $f(a)$ for all elements in the set $\mathcal{A}$ This returns the maximum "y" value
- The operator $\arg \max_{a \in \mathcal{A}} f(a)$ returns the element of the set $\mathcal{A}$ that maximizes $f(a)$ This returns the "x" value that results in the maximum "y" value.
- When the set is **implicit** or **infinite**, we can write

$$\max_a f(a) \quad \text{or} \quad \arg \max_a f(a)$$

E.g. $f(a) = 3a, a \in [0,1] \rightarrow \max_a f(a) = 3$ and $\arg \max_a f(a) = 1$

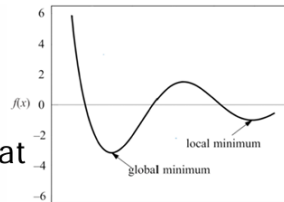
Min and **Arg Min** operate in a similar manner

Note: **arg max** returns a value from the **domain** of the function and **max** returns from the **range (codomain)** of the function.

Derivative

Derivative and Gradient

- The **derivative** f' of a function f is a function that describes how fast f grows (or decreases)
 - If the derivative is a **constant** value, e.g. 5 or -3
 - The function f grows (or decreases) **constantly at any point x of its domain**
 - When the derivative f' is a function
 - If f' is **positive** at some x , then the function f **grows** at this point
 - If f' is **negative** at some x , then the function f **decreases** at this point
 - The derivative of **zero** at x means that the function's **slope** at x is **horizontal** (e.g. maximum or minimum points)



- The process of finding a derivative is called **differentiation**.

Gradient

- Gradient is the generalization of derivative for functions that take several inputs (or one input in the form of a vector or some other complex structure).

Differentiation of a **scalar** function w.r.t. a **vector**

Differentiation of a scalar function w.r.t. a vector

If $f(\mathbf{x})$ is a **scalar function** of d variables, $\mathbf{x}$ is a $d \times 1$ vector. Then differentiation of $f(\mathbf{x})$ w.r.t. $\mathbf{x}$ results in a $d \times 1$ vector

$$\frac{df(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix}$$

Perform differentiation of $f(\mathbf{x})$ with respect to each d variables. Similar to your partial differentiation, when differentiating w.r.t. one variable, the other variables are considered as constants.

$$\mathbf{T}: \mathbb{R}^3 \rightarrow \mathbb{R}^1$$

(x, y, z)
 3×1

This is referred to as the **gradient** of $f(\mathbf{x})$ and often written as $\nabla_{\mathbf{x}} f$.

$$\text{E.g. } f(\mathbf{x}) = ax_1 + bx_2 \quad \nabla_{\mathbf{x}} f = \begin{bmatrix} a \\ b \end{bmatrix}$$

Ref: Duda, Hart, and Stork, "Pattern Classification", 2001

- The gradient of a function is a vector of partial derivatives
- Scalar function produces a single scalar output. Vector function produces a vector output.

Differentiation of a **vector** function w.r.t. a **vector**

Differentiation of a **vector** function w.r.t. a **vector**

If $\mathbf{f}(\mathbf{x})$ is a **vector** function of size $h \times 1$ and $\mathbf{x}$ is a $d \times 1$ vector. Then differentiation of $\mathbf{f}(\mathbf{x})$ results in a $h \times d$ matrix

$$\frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_h}{\partial x_1} & \dots & \frac{\partial f_h}{\partial x_d} \end{bmatrix}$$

E.g. Differentiating a vector with a vector.
Look at below $d\mathbf{A}\mathbf{x}/d\mathbf{x} = \mathbf{A}$.

- The matrix is referred to as the **Jacobian** of $\mathbf{f}(\mathbf{x})$.

Some Vector-Matrix Differentiation Formula

Some Vector-Matrix Differentiation Formulae

$\mathbf{A}\mathbf{x}$ here forms a matrix. 2×1

$$\frac{d(\mathbf{A}\mathbf{x})}{d\mathbf{x}} = \mathbf{A}$$

Basically, you take each row and differentiate it w.r.t each element in the vector $\mathbf{x}$.

$\mathbf{b}^T\mathbf{x}$ here forms a linear equation. Or similar to scalar function. 3×1

$$\frac{d(\mathbf{b}^T\mathbf{x})}{d\mathbf{x}} = \mathbf{b}$$

Same thing as the left-hand side, just that you sub $\mathbf{b}^T$ as $(\mathbf{y}^T\mathbf{A})$. To get final value, just transpose $(\mathbf{y}^T\mathbf{A})$ to get $(\mathbf{A}^T\mathbf{y})$. It's probably differentiating a scalar function too.

$$\frac{d(\mathbf{y}^T\mathbf{A}\mathbf{x})}{d\mathbf{x}} = \mathbf{A}^T\mathbf{y}$$

$$\frac{d(\mathbf{x}^T\mathbf{A}\mathbf{x})}{d\mathbf{x}} = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$$

When you partial differentiate this with $[x_1, x_2, \dots, x_d]$, you get back $\mathbf{A}$, not row vector because actually you are differentiating a scalar function and no longer a vector function.

$$f(\mathbf{x}) = \mathbf{a}^T\mathbf{x} = a_1x_1 + a_2x_2 + \dots + a_dx_d$$

Derivations: <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf>
Inner product of 2 vectors produces a scalar output.

Ref: Duda, Hart, and Stork, "Pattern Classification", 2001 (Appendix)

- Take $\mathbf{x}$ as a 3×1 vector and $\mathbf{A}$ as 2×3 matrix.

NOTE: $(\mathbf{y}^T\mathbf{A})^T = \mathbf{A}^T\mathbf{y}$ (Flip their positions and transpose them)

Linear Regression

• Linear regression is a popular regression learning algorithm that learns a model which is a linear combination of features of the input example.

$$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m \times d}, \quad \mathbf{w} \in \mathcal{R}^{d \times 1}, \quad \mathbf{y} \in \mathcal{R}^{m \times 1}$$

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \dots & x_{m,d} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

Use of Linear Regression

- Test set should NOT be used to train the ML model.

Problem Statement: To predict the unknown y for a given $\mathbf{x}$ (**testing**)

- We have a collection of labeled examples (**training**) $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$
 - m is the size of the collection
 - $\mathbf{x}_i$ is the d -dimensional feature vector of example $i = 1, \dots, m$ (input)
 - y_i is a **real-valued target** (1-D)
 - Note:
 - If y can take any real value = Regression
 - If y is categorical (e.g., 0 or 1) = Classification
 - when y_i is **continuous** valued, it is a **regression problem**
 - when y_i is **discrete** valued, it is a **classification problem**
- We want to **build a model** $f_{\mathbf{w},b}(\mathbf{x})$ as a linear combination of features of example $\mathbf{x}$: $f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{x}^T\mathbf{w} + b$
 - $\mathbf{x}$: This is the feature vector (input data). It contains multiple variables (features).
 - $\mathbf{w}$: This is the weight vector (parameters the model learns). Each feature in $\mathbf{x}$ has a corresponding weight in $\mathbf{w}$, which determines how important that feature is.
 - b : This is the bias term (also called the intercept). It shifts the model up or down to fit the data better. Without b , the model would always pass through the origin (zero point).
- The notation $f_{\mathbf{w},b}$ means that the model f is parametrized by two values: $\mathbf{w}$ and b
 - The whole equation means multiply each feature by its corresponding weight, sum them all together, add the bias term b .

Learning Objective Function

- Squared errors of a **single data point** is called the loss function.
- We want to **minimise** the mean squared errors by finding **optimal values of w and b**.

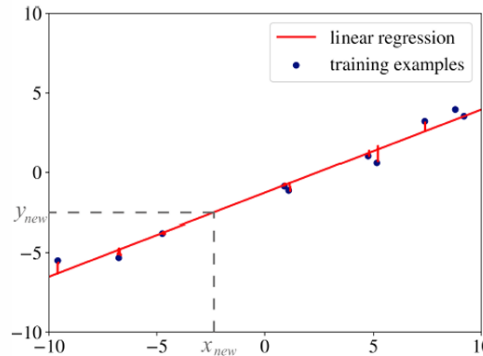
Learning objective function

- To find the optimal values for $\mathbf{w}^*$ and b^* which **minimizes** the following expression:

Known as mean squared errors

$$\frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(\mathbf{x}_i) - y_i)^2$$

- In mathematics, the expression we minimize or maximize is called an **objective function**, or, simply, an **objective**. Sum up the squared errors.



$(f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2$ is called the **loss function**: a measure of the difference between $f_{\mathbf{w}}(\mathbf{x}_i)$ and y_i or a penalty for misclassification of example i .

Combining bias into feature & weight vector

- To find the optimal values for $\mathbf{w}^*$ which **minimizes** the following expression:

$$\sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2$$

with $f_{\mathbf{w}}(\mathbf{x}_i) = \mathbf{x}_i^T \mathbf{w}$,

where we define $\mathbf{w} = [b, w_1, \dots, w_d]^T = [w_0, w_1, \dots, w_d]^T$,

and $\mathbf{x}_i = [1, x_{i,1}, \dots, x_{i,d}]^T = [x_{i,0}, x_{i,1}, \dots, x_{i,d}]^T, i = 1, \dots, m$

The first entry is always 1, so that it automatically accounts for the bias.

- This particular choice of the loss function is called **squared error loss**

Note: The normalization factor $\frac{1}{m}$ can be omitted as it does not affect the optimization.

- Pad a column of 1s to the left of feature vector.
- An extra variable w_0 is padded to the top of the weight vector to represent bias/offset, b .
- Originally (page 14), the equation is $f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{x}^T \mathbf{W} + b = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b$
- Instead of handling the bias separately, we rearrange the equation to include it within the weight vector.
- Since bias is now included as a column in feature matrix, the number of rows of the weight vector also increase by 1.

Cost Function

- All model-based learning algorithms have a loss function
- What we do to find the best model is to minimize the objective known as the cost function
- Cost function is a **sum of loss functions** over training set plus possibly some model complexity penalty (regularization)
- Sometimes, the model overfits, meaning it memorizes the training data instead of learning general patterns.
- To prevent this, we add a penalty term to the cost function. New cost function will add $\lambda * R(w)$. λ is the regularization strength (controls how much we penalize complexity) and $R(w)$ is a function that penalizes large weights.
- Common types of regularization: L1 Regularization (Ridge Regression), L2 Regularization (Lasso Regression).
- In linear regression, the **cost function** is given by the average loss, also called the **empirical risk** because we do not have all the data (e.g. testing data)
- Empirical Risk is just cost function/# of loss functions. Or simply just the **Mean Squared Errors** instead of just Sum of Squared Errors.
- The average of all penalties is obtained by applying the model to the training data.

Learning (Training)

- Consider the set of feature vector $\mathbf{x}_i$ and target output y_i indexed by $i = 1, \dots, m$, a linear model $f_w(\mathbf{x}) = \mathbf{x}^T \mathbf{w}$ can be stacked as

$$\begin{aligned} f_w(\mathbf{X}) = \mathbf{X}\mathbf{w} & \longleftrightarrow \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} \\ & = \begin{bmatrix} \mathbf{x}_1^T \mathbf{w} \\ \vdots \\ \mathbf{x}_m^T \mathbf{w} \end{bmatrix} \\ \text{where } \mathbf{x}_i^T \mathbf{w} & = [1, x_{i,1}, \dots, x_{i,d}] \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_d \end{bmatrix} \end{aligned}$$

Learning Model Learning target vector

Note: The bias/offset term is responsible for translating the line/plane/hyperplane away from the origin.

Least Square Regression

In vector-matrix notation, the minimization of the objective function can be written compactly using $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$:

$$J(\mathbf{w}) = \mathbf{e}^T \mathbf{e}$$

^ This represents the difference between predicted values ($\mathbf{X}\mathbf{w}$) and true values ($\mathbf{y}$).

To find the optimal $\mathbf{w}$, we need to differentiate $J(\mathbf{w})$ and set the derivative to zero. Find the gradient of $J(\mathbf{w})$ w.r.t $\mathbf{w}$ since it is scalar function differentiation.

$$\begin{aligned} J(\mathbf{w}) &= (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= (\mathbf{w}^T \mathbf{X}^T - \mathbf{y}^T) (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{y} - \mathbf{y}^T \mathbf{X} \mathbf{w} + \mathbf{y}^T \mathbf{y} \\ \text{All terms are scalar! } &> = \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{y}^T \mathbf{X} \mathbf{w} + \mathbf{y}^T \mathbf{y}. \end{aligned}$$

Note: when $f_w(\mathbf{X}) = \mathbf{X}\mathbf{w}$, then

$$\sum_{i=1}^m (f_w(\mathbf{x}_i) - y_i)^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}).$$

- The goal of linear regression is to find the best weight vector w that minimizes the error between the predicted and actual values.

- For a regression line $y = ax + b$, the estimated a^* and b^* minimize the sum of squared errors between the predicted y' and the target output y for all training samples.

Differentiating $J(w)$ with respect to w and setting the result to 0 :

$$\begin{aligned} \frac{\partial}{\partial w} J(w) &= 0 \\ \frac{\partial}{\partial w} (w^T X^T X w - 2y^T X w + y^T y) &= 0 \\ \Rightarrow 2X^T X w - 2X^T y &= 0 \\ \Rightarrow X^T X w &= X^T y \end{aligned}$$

$\Rightarrow$ Any minimizer $\hat{w}$ of $J(w)$ must satisfy $X^T(Xw - y) = 0$.
If $X^T X$ is invertible, then

Learning/training: $\hat{w} = (X^T X)^{-1} X^T y$

Prediction/testing: $\hat{f}_w(X_{new}) = X_{new} \hat{w}$

You input your X_{new} into the model and it predicts your target output.

- If $X^T X$ is not invertible, we need regularization (Ridge Regression) or Gradient Descent.

Example 1

Example 1 Training set $\{(x_i, y_i)\}_{i=1}^m$

$\{x = -9\} \rightarrow \{y = -6\}$
$\{x = -7\} \rightarrow \{y = -6\}$
$\{x = -5\} \rightarrow \{y = -4\}$
$\{x = 1\} \rightarrow \{y = -1\}$
$\{x = 5\} \rightarrow \{y = 1\}$
$\{x = 9\} \rightarrow \{y = 4\}$

$$X = \begin{bmatrix} 1 & -9 \\ 1 & -7 \\ 1 & -5 \\ 1 & 1 \\ 1 & 5 \\ 1 & 9 \end{bmatrix} \quad w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} -6 \\ -6 \\ -4 \\ -1 \\ 1 \\ 4 \end{bmatrix}$$

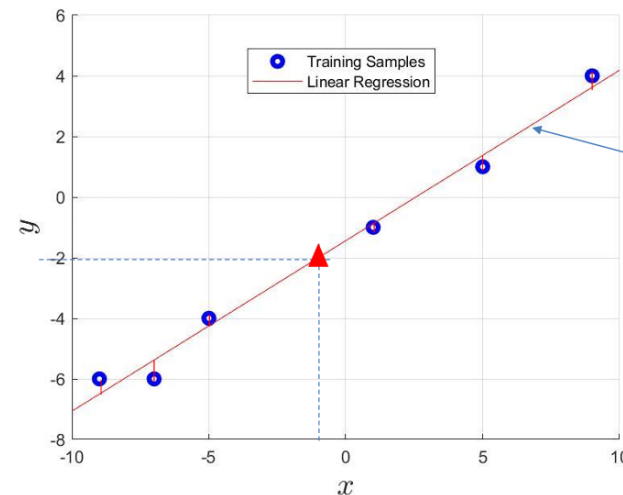
NOTE: First column of X is all 1 to account for bias/offset! Notice how w has 2 rows even though there's only one feature. w_0 represents the bias/offset!

This set of linear equations has **no exact solution**

However, $X^T X$ is invertible

Least square approximation

$$\begin{aligned} \hat{w} &= X^\dagger y = (X^T X)^{-1} X^T y \\ &= \begin{bmatrix} 6 & -6 \\ -6 & 262 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ -9 & -7 & -5 & 1 & 5 & 9 \end{bmatrix} \begin{bmatrix} -6 \\ -6 \\ -4 \\ -1 \\ 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix} \end{aligned}$$



$$\begin{aligned} \hat{y} &= X \hat{w} \\ &= \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix} \\ &= -1.4375 + 0.5625x \end{aligned}$$

Prediction:
Test set

$$\{x = -1\} \rightarrow \{y = ?\}$$

The first column = 1 to account for the bias/offset trained in the model.

$$\begin{aligned} \hat{y} &= \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix} \\ &= -2 \end{aligned}$$

Linear Regression on one-dimensional samples

Python demo 1

Example 2

Example 2

$$\{(x_i, y_i)\}_{i=1}^m$$

$$\{x_1 = 1, x_2 = 1, x_3 = 1\} \rightarrow \{y = 1\}$$

$$\{x_1 = 1, x_2 = -1, x_3 = 1\} \rightarrow \{y = 0\}$$

$$\{x_1 = 1, x_2 = 1, x_3 = 3\} \rightarrow \{y = 2\}$$

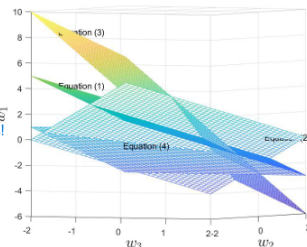
$$\{x_1 = 1, x_2 = 1, x_3 = 0\} \rightarrow \{y = -1\}$$

Training set

$$\begin{matrix} \mathbf{X} & \mathbf{w} & \mathbf{y} \\ \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix} & \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} & \begin{bmatrix} 1 \\ 0 \\ 2 \\ -1 \end{bmatrix} \end{matrix}$$

No need to add offset to w as well. That's why no w_0 .

REMEMBER: If first column of feature matrix is not constant, you NEED to add a column of 1s to account for bias!



This set of linear equations has no exact solution

However, $\mathbf{X}^T \mathbf{X}$ is invertible

$$\hat{\mathbf{w}} = \mathbf{X}^+ \mathbf{y} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Least square approximation

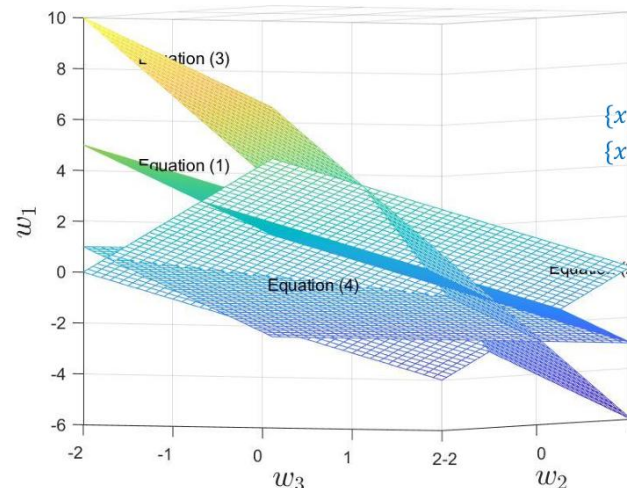
$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \\ -1 \end{bmatrix} = \begin{bmatrix} -0.7500 \\ 0.1786 \\ 0.9286 \end{bmatrix}$$

For planes like this, we are trying to find the intersection of planes.

- In this case, the first column/feature is all 1 (usually not the case, will be more random), you DO NOT NEED to add another column of 1 to account for bias/offset!

- Any constant will work, no need to be 1, as the weight vector will adjust accordingly.

The four linear equations



Prediction:

Test set

$$\{x_1 = 1, x_2 = 6, x_3 = 8\} \rightarrow \{y = ?\}$$

$$\{x_1 = 1, x_2 = 0, x_3 = -1\} \rightarrow \{y = ?\}$$

$$\hat{\mathbf{y}} = \hat{\mathbf{f}}_{\mathbf{w}}(\mathbf{X}_{\text{new}}) = \mathbf{X}_{\text{new}} \hat{\mathbf{w}}$$

$$\hat{\mathbf{y}} = \begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} -0.7500 \\ 0.1786 \\ 0.9286 \end{bmatrix} = \begin{bmatrix} 7.7500 \\ -1.6786 \end{bmatrix}$$

Prediction here outputs a single scalar output for each sample. The next slide introduces a model that outputs a vector (multiple outputs) for one sample.

Learning of Vectored Function (Multiple Outputs)

For one sample: a linear model $\mathbf{f}_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^T \mathbf{W}$

Vector function

For m samples: $\mathbf{F}_{\mathbf{w}}(\mathbf{X}) = \mathbf{XW} = \mathbf{Y}$ $\mathbf{W}$ and $\mathbf{Y}$ no longer vector, capital bold for matrix

$$\begin{matrix} \text{Sample 1} \rightarrow \\ \vdots \\ \text{Sample } m \rightarrow \end{matrix} \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_m^T \end{bmatrix} \mathbf{W} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \dots & x_{m,d} \end{bmatrix} \begin{bmatrix} w_{0,1} & \dots & w_{0,h} \\ w_{1,1} & \dots & w_{1,h} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \dots & w_{d,h} \end{bmatrix}$$

Output could be like:
Age, Net Worth, # of kids

$$\begin{matrix} \text{Sample 1's output} \rightarrow \\ \vdots \\ \text{Sample } m \text{'s output} \rightarrow \end{matrix} \begin{bmatrix} y_{1,1} & \dots & y_{1,h} \\ \vdots & \vdots & \vdots \\ y_{m,1} & \dots & y_{m,h} \end{bmatrix}$$

With 3 outputs, matrix $\mathbf{w}$ should have 3 columns.

$$\mathbf{X} \in \mathcal{R}^{m \times (d+1)}, \mathbf{W} \in \mathcal{R}^{(d+1) \times h}, \mathbf{Y} \in \mathcal{R}^{m \times h}$$

Capital bold E now since no longer scalar.

$$\text{Objective: } \sum_{i=1}^m (\mathbf{f}_w(\mathbf{x}_i) - \mathbf{y}_i)^2 = \mathbf{E}^T \mathbf{E}$$

Least Squares Regression of Multiple Outputs

In matrix notation, the sum of squared errors cost function can be written compactly using $\mathbf{E} = \mathbf{X}\mathbf{W} - \mathbf{Y}$:

$J(\mathbf{W})$ here is no longer scalar but a matrix.

$$J(\mathbf{W}) = \text{trace}(\mathbf{E}^T \mathbf{E}) \\ = \text{trace}[(\mathbf{X}\mathbf{W} - \mathbf{Y})^T (\mathbf{X}\mathbf{W} - \mathbf{Y})]$$

If $\mathbf{X}^T \mathbf{X}$ is invertible, then

The learning for the multiple outputs is very similar to that of the single outputs, we're just replacing vector, $\mathbf{y}$ and $\mathbf{w}$, with the matrix $\mathbf{Y}$ and $\mathbf{W}$ and everything else is the same.

Because $\mathbf{W}$ is a matrix, there are different weight for different output thus the outputs are independent of each other.

Learning/training: $\hat{\mathbf{W}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$

Prediction/testing: $\hat{\mathbf{F}}_w(\mathbf{X}_{new}) = \mathbf{X}_{new} \hat{\mathbf{W}}$

Least Squares Regression of Multiple Outputs

$$J(\mathbf{W}) = \text{trace}(\mathbf{E}^T \mathbf{E})$$

$$= \text{trace} \left(\begin{bmatrix} \mathbf{e}_1^T \\ \vdots \\ \mathbf{e}_h^T \end{bmatrix} [\mathbf{e}_1 \quad \mathbf{e}_2 \quad \dots \quad \mathbf{e}_h] \right)$$

$$= \text{trace} \left(\begin{bmatrix} \mathbf{e}_1^T \mathbf{e}_1 & \mathbf{e}_1^T \mathbf{e}_2 & \dots & \mathbf{e}_1^T \mathbf{e}_h \\ \mathbf{e}_2^T \mathbf{e}_1 & \mathbf{e}_2^T \mathbf{e}_2 & \dots & \mathbf{e}_2^T \mathbf{e}_h \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{e}_h^T \mathbf{e}_1 & \mathbf{e}_h^T \mathbf{e}_2 & \dots & \mathbf{e}_h^T \mathbf{e}_h \end{bmatrix} \right) = \sum_{k=1}^h \mathbf{e}_k^T \mathbf{e}_k$$

Summation of the diagonal elements.

Example 3

Training set $\{x_1=1, x_2=1, x_3=1\} \rightarrow \{y_1=1, y_2=0\}$

$\{x_i, y_i\}_{i=1}^m$ $\{x_1=1, x_2=-1, x_3=1\} \rightarrow \{y_1=0, y_2=1\}$

$\{x_1=1, x_2=1, x_3=3\} \rightarrow \{y_1=2, y_2=-1\}$

$\{x_1=1, x_2=1, x_3=0\} \rightarrow \{y_1=-1, y_2=3\}$

$$\text{Bias} \rightarrow \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -1 \\ -1 & 3 \end{bmatrix}$$

of features = # of rows in $\mathbf{W}$
of outputs = # of columns in $\mathbf{W}$

This set of linear equations has NO exact solution

$$\hat{\mathbf{W}} = \mathbf{X}^+ \mathbf{Y} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

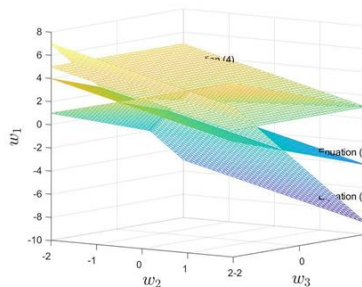
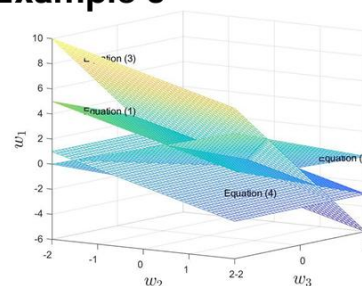
$\mathbf{X}^T \mathbf{X}$ is invertible

Least square approximation

$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -1 \\ -1 & 3 \end{bmatrix} = \begin{bmatrix} -0.75 & 2.25 \\ 0.1786 & 0.0357 \\ 0.9286 & -1.2143 \end{bmatrix}$$

y_1 y_2

Example 3



Prediction:

Test set: two new samples

$\{x_1=1, x_2=6, x_3=8\} \rightarrow \{y_1=?, y_2=?\}$

$\{x_1=1, x_2=0, x_3=-1\} \rightarrow \{y_1=?, y_2=?\}$

$$\hat{\mathbf{Y}} = \mathbf{X}_{new} \hat{\mathbf{W}}$$

$$\text{Bias} \rightarrow \begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} -0.75 & 2.25 \\ 0.1786 & 0.0357 \\ 0.9286 & -1.2143 \end{bmatrix} = \begin{bmatrix} 7.75 & -7.25 \\ -1.6786 & 3.4643 \end{bmatrix}$$

Python demo 2

Example 4

The values of feature x and their corresponding values of multiple outputs target y are shown in the table below.

Based on the least square regression, what are the values of $\mathbf{w}$?
Based on the current mapping, when $x = 2$, what is the value of y ?

x	[3]	[4]	[10]	[6]	[7]
y	[0, 5]	[1.5, 4]	[-3, 8]	[-4, 10]	[1, 6]

$$\hat{\mathbf{W}} = \mathbf{X}^+ \mathbf{Y} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y} = \begin{bmatrix} 1.9 & 3.6 \\ -0.4667 & 0.5 \end{bmatrix}$$

Python demo 3

$$\hat{\mathbf{Y}}_{new} = \mathbf{X}_{new} \hat{\mathbf{W}} = \begin{bmatrix} 1 & 2 \end{bmatrix} \hat{\mathbf{W}} = \begin{bmatrix} 0.9667 & 4.6 \end{bmatrix} \quad \text{Prediction}$$

Quick Summary

Summary

- Notations, Vectors, Matrices
- Operations on Vectors and Matrices
 - Dot-product, matrix inverse
- Systems of Linear Equations $f_w(\mathbf{X}) = \mathbf{X}\mathbf{w} = \mathbf{y}$
 - Matrix-vector notation, linear dependency, invertible
 - Even-, over-, under-determined linear systems
- Functions, Derivative and Gradient
 - Inner product, linear/affine functions
 - Maximum and minimum, partial derivatives, gradient
- Least Squares, Linear Regression
 - Objective function, loss function
 - Least square solution, training/learning and testing/prediction
 - Linear regression with multiple outputs

Learning/training $\hat{\mathbf{W}} = (\mathbf{X}_{train}^T \mathbf{X}_{train})^{-1} \mathbf{X}_{train}^T \mathbf{y}_{train}$

Prediction/testing $\mathbf{y}_{test} = \mathbf{X}_{test} \hat{\mathbf{W}}$

- Classification
 - Ridge Regression
 - Polynomial Regression
- Python packages: numpy, pandas, matplotlib.pyplot, numpy.linalg, and sklearn.metrics (for mean_squared_error), numpy.linalg.pinv



Linear Methods for Classification

- We have a collection of labelled examples
- m is the size of the collection
- $\mathbf{x}_i$ is the d -dimensional feature vector of example $i = 1, \dots, m$
- y_i is discrete target label (e.g., $y_i \in \{-1, +1\}$ or $\{0, 1\}$ for binary classification problems)

Note:

- When y_i is continuous valued $\rightarrow$ a regression problem
- When y_i is discrete valued $\rightarrow$ a classification problem
- Linear model: $f_{w,b}(\mathbf{x}) = \mathbf{x}^T \mathbf{w} + b$ or in compact form $f_w(\mathbf{x}) = \mathbf{x}^T \mathbf{w}$ (having the offset term absorbed into the inner product)

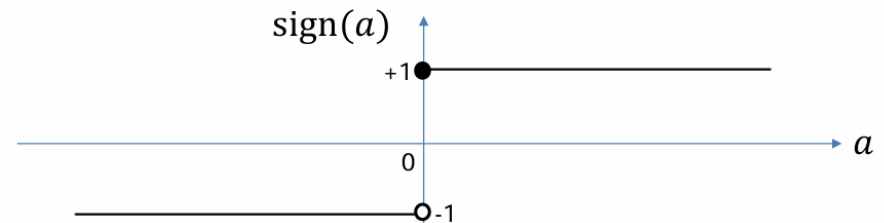
Binary Classification

If $\mathbf{X}^T \mathbf{X}$ is invertible, then

Learning: $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$, $y_i \in \{-1, +1\}, i = 1, \dots, m$

Prediction: $\hat{f}_w^c(\mathbf{x}_{new}) = \text{sign}(\mathbf{x}_{new}^T \hat{\mathbf{w}})$ for each row $\mathbf{x}_{new}^T$ of $\mathbf{X}_{new}$

$\text{sign}(a) = +1$ for $a \geq 0$ and -1 for $a < 0$



Example 1

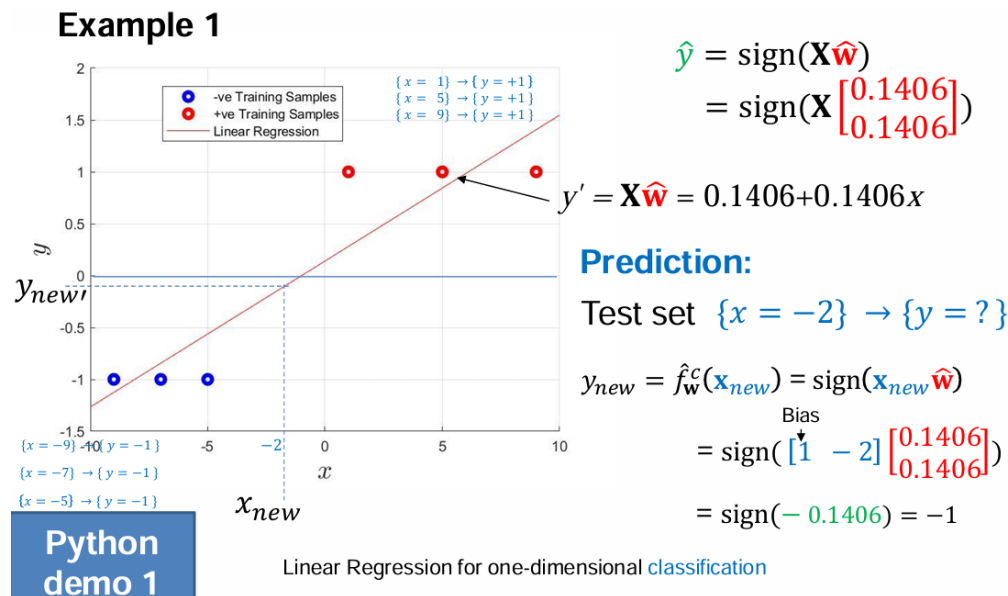
Example 1 Training set $\{x_i, y_i\}_{i=1}^m$

	$\mathbf{X}$	$\mathbf{w}$	$\mathbf{y}$	
Bias	$\begin{bmatrix} 1 & -9 \\ 1 & -7 \\ 1 & -5 \\ 1 & 1 \\ 1 & 5 \\ 1 & 9 \end{bmatrix}$	$\begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$	$\begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$	$\{x = -9\} \rightarrow \{y = -1\}$ $\{x = -7\} \rightarrow \{y = -1\}$ $\{x = -5\} \rightarrow \{y = -1\}$ $\{x = 1\} \rightarrow \{y = +1\}$ $\{x = 5\} \rightarrow \{y = +1\}$ $\{x = 9\} \rightarrow \{y = +1\}$

This set of linear equations has NO exact solution

$$\hat{\mathbf{w}} = \mathbf{X}^+ \mathbf{y} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad \mathbf{X}^T \mathbf{X} \text{ is invertible}$$

$$= \begin{bmatrix} 6 & -6 \\ -6 & 262 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ -9 & -7 & -5 & 1 & 5 & 9 \end{bmatrix} = \begin{bmatrix} 0.1406 \\ 0.1406 \end{bmatrix} \quad \text{Least square approximation}$$



Multi-Category Classification

If $\mathbf{X}^T \mathbf{X}$ is invertible, then

Learning: $\hat{\mathbf{W}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}, \quad \mathbf{Y} \in \mathbb{R}^{m \times C}$

Prediction: $\hat{f}_{\mathbf{w}}^c(\mathbf{x}_{\text{new}}) = \arg \max_{k=1, \dots, C} (\mathbf{x}_{\text{new}}^T \hat{\mathbf{W}}(:, k))$ for each $\mathbf{x}_{\text{new}}^T$ of $\mathbf{X}_{\text{new}}$

arg max returns the index of the max element.

Each row (of $i = 1, \dots, m$) in $\mathbf{Y}$ has an **one-hot** encoding/assignment:

e.g., target for class-1 is labelled as $\mathbf{y}_i^T = [1, 0, 0, \dots, 0]$ for the i th sample,
 target for class-2 is labelled as $\mathbf{y}_j^T = [0, 1, 0, \dots, 0]$ for the j th sample,
 target for class-C is labelled as $\mathbf{y}_m^T = [0, 0, \dots, 0, 1]$ for the m th sample.

Example 2 Three Class Classification

Example 2 Three class classification

Training set $\{x_i, y_i\}_{i=1}^m$

$\{x_1 = 1, x_2 = 1\} \rightarrow \{y_1 = 1, y_2 = 0, y_3 = 0\}$	Class 1
$\{x_1 = -1, x_2 = 1\} \rightarrow \{y_1 = 0, y_2 = 1, y_3 = 0\}$	Class 2
$\{x_1 = 1, x_2 = 3\} \rightarrow \{y_1 = 1, y_2 = 0, y_3 = 0\}$	Class 1
$\{x_1 = 1, x_2 = 0\} \rightarrow \{y_1 = 0, y_2 = 0, y_3 = 1\}$	Class 3

	$\mathbf{X}$	$\mathbf{W}$	$\mathbf{Y}$
Bias	$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

This set of linear equations has NO exact solution.

Least square approximation

$$\hat{\mathbf{W}} = \mathbf{X}^+ \mathbf{Y} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y} \quad \mathbf{X}^T \mathbf{X} \text{ is invertible}$$

$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0.2857 & -0.5 & 0.2143 \\ 0.2857 & 0 & -0.2857 \end{bmatrix}$$

REMEMBER, one-hot encoding can only have a single 1 in each row!

Example 2 Prediction

Test set $\mathbf{X}_{new}$ $\{x_1=6, x_2=8\} \rightarrow \{\text{class 1, 2, or 3?}\}$
 $\{x_1=0, x_2=-1\} \rightarrow \{\text{class 1, 2, or 3?}\}$

$$\hat{\mathbf{Y}} = \mathbf{X}_{new} \hat{\mathbf{W}} = \begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0.2857 & -0.5 & 0.2143 \\ 0.2857 & 0 & -0.2857 \end{bmatrix}$$

Category prediction:

$$\begin{aligned} \hat{f}_{\mathbf{w}}^c(\mathbf{X}_{new}) &= \arg \max_{k=1,\dots,C} (\hat{\mathbf{Y}}(:, k)) \\ &= \arg \max_{k=1,\dots,C} \left(\begin{bmatrix} 4 & -2.50 & -0.50 \\ -0.2587 & 0.50 & 0.7857 \end{bmatrix} \right) \\ &= \begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{matrix} \rightarrow \text{Class 1} \\ \rightarrow \text{Class 3} \end{matrix} \end{aligned}$$

Python
demo 2

For each row of $\mathbf{Y}$, the **column position** of the **largest** number (across all columns for that row) determines the **class label**.

E.g. in the first row, the maximum number is 4 which is in column 1. Therefore, the resulting predicted class is 1.

Ridge Regression

- We cannot guarantee $\mathbf{X}^T \mathbf{X}$ is invertible. Do this when it is not.
- $\mathbf{X}^T \mathbf{X}$ is non-invertible (singular) when features are collinear (perfect multicollinearity) and the number of features d exceeds the number of samples m .
- In such cases, the ordinary least squares (OLS) solution does not exist. Ridge Regression fixes this as $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}$ is **always invertible**.
- Previously, Linear Regression is minimising the sum of squared errors. Now, Ridge Regression minimises the sum of squared errors + λ * sum of weight squared.
- Ridge regression: Shrinks the regression coefficients $\mathbf{w}$ by imposing a penalty on their size.

- It is a regularization technique that prevents overfitting by discouraging overly large coefficients by shrinking their values toward zero.

- The larger the value of λ , the greater the amount of shrinkage, the smaller $\mathbf{w}^T \mathbf{w}$ is, the more the penalty on parameters.

Recall Linear regression

Objective: $\hat{\mathbf{w}} = \arg \min \sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$

The learning computation: $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

We cannot guarantee that the matrix $\mathbf{X}^T \mathbf{X}$ is invertible

$\mathbf{X}^T \mathbf{X}$ is non-invertible (singular) when features are collinear (perfect multicollinearity) and the number of features d exceeds the number of samples m . In such cases, the ordinary least squares (OLS) solution does not exist. Ridge Regression fixes this as $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}$ is always invertible.

Ridge regression: shrinks the regression coefficients $\mathbf{w}$ by imposing a penalty on their size A regularization technique that prevents overfitting by discouraging overly large coefficients by shrinking their values toward zero.

Objective: $\hat{\mathbf{w}} = \arg \min \sum_{i=1}^m (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2 + \lambda \sum_{j=1}^d w_j^2$
 $= \arg \min (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$

Here $\lambda \geq 0$ is a complexity parameter that controls the amount of shrinkage: the larger the value of λ , the greater the amount of shrinkage.

We basically cut down by penalizing the numbers of parameters. We want the models to be simple.

The simpler the model is.

Note: m samples & d parameters

Further on the next page.

We want to minimize both errors.

Using a linear model:

We want to minimize both errors

$$\min_{\mathbf{w}} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$$

Solution:

$$\frac{\partial}{\partial \mathbf{w}} ((\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}) = \mathbf{0}$$

$$\Rightarrow 2\mathbf{X}^T \mathbf{X}\mathbf{w} - 2\mathbf{X}^T \mathbf{y} + 2\lambda \mathbf{w} = \mathbf{0}$$

$$\Rightarrow \mathbf{X}^T \mathbf{X}\mathbf{w} + \lambda \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

$$\Rightarrow (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

where $\mathbf{I}$ is the $d \times d$ identity matrix

Here on, we shall focus on single column of output $\mathbf{y}$ in derivations in the sequel

Learning: $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$

Ridge Regression in Primal Form (rows > columns)

- Samples more than Features (Over-Determined)

$(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})$ is invertible for $\lambda > 0$,

Learning: $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{X}_{new}) = \mathbf{X}_{new} \hat{\mathbf{w}}$

Ridge Regression in Dual Form (rows < columns)

- In real world, Ridge is always better than Linear like how adding a bias is always better than not accounting for it.
- Samples less than Features (Under-Determined)

Computation is always done on the smaller dimensions (m or d)

$(\mathbf{X}\mathbf{X}^T + \lambda \mathbf{I})$ is invertible for $\lambda > 0$,



m x m identity matrix

Learning: $\hat{\mathbf{w}} = \mathbf{X}^T (\mathbf{X}\mathbf{X}^T + \lambda \mathbf{I})^{-1} \mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{X}_{new}) = \mathbf{X}_{new} \hat{\mathbf{w}}$

IMPORTANT: Computation is always done on the smaller dimensions, m or d!

- A validation data set is used to determine the optimized value of lambda. (Outside of EE2211, lambda will be given in questions).

Polynomial Regression

Motivation: nonlinear decision surface

- Based on the sum of products of the variables
- E.g. when the input dimension is $d=2$, a polynomial function of degree = 2 is:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_{12} x_1 x_2 + w_{11} x_1^2 + w_{22} x_2^2.$$

Black terms = Linear Regression = Polynomial Regression of Degree 1.

Polynomial Expansion

- The linear model $f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^T \mathbf{w}$ can be written as

$$\begin{aligned} f_{\mathbf{w}}(\mathbf{x}) &= \mathbf{x}^T \mathbf{w} \\ &= \sum_{i=0}^d x_i w_i, \quad x_0 = 1 \\ &= w_0 + \sum_{i=1}^d x_i w_i. \end{aligned}$$

n is the number of features, r is the degree

- By including additional terms involving the products of pairs of components of $\mathbf{x}$, we obtain a quadratic model:

$$f_{\mathbf{w}}(\mathbf{x}) = \underbrace{w_0}_{\text{Bias/Offset}} + \sum_{i=1}^d \underbrace{w_i x_i}_{\text{Linear Terms}} + \sum_{i=1}^d \sum_{j=1}^d \underbrace{w_{ij} x_i x_j}_{\text{Quadratic Terms}}.$$

$$2^{\text{nd}} \text{ order: } f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_{12} x_1 x_2 + w_{11} x_1^2 + w_{22} x_2^2$$

$$3^{\text{rd}} \text{ order: } f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_{12} x_1 x_2 + w_{11} x_1^2 + w_{22} x_2^2 + \sum_{i=1}^d \sum_{j=1}^d \sum_{k=1}^d w_{ijk} x_i x_j x_k, \quad d = 2$$

Ref: Duda, Hart, and Stork, "Pattern Classification", 2001 (Chp.5)

$$C(n, r) = \frac{(n+r-1)!}{r! (n-1)!}$$

Formula for number of terms in a degree:

$$\frac{(n + r - 1)!}{r! (n - 1)!}$$

where n is the number of features, r is the degree.

- So if the degree is 2, feature is 2, JUST for degree 2 terms, using the formula, there are 3 degree 2 terms. NEED to add degree 1 and 0.
- Degree 0 is bias/offset/intercept! If count degree 0, don't add another 1 to number of parameters to learn!

Generalized Linear Discriminant Function

- In general:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=1}^d w_{ij} x_i x_j + \sum_{i=1}^d \sum_{j=1}^d \sum_{k=1}^d w_{ijk} x_i x_j x_k + \dots$$

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{P} \mathbf{w} \quad (\text{Note: } \mathbf{P} \triangleq \mathbf{P}(\mathbf{X}) \text{ for symbol simplicity})$$

$$= \begin{bmatrix} \mathbf{p}_1^T \mathbf{w} \\ \vdots \\ \mathbf{p}_m^T \mathbf{w} \end{bmatrix} \quad \text{where } \mathbf{p}_l^T \mathbf{w} = [1, x_{l,1}, \dots, x_{l,d}, \dots, x_{l,i} x_{l,j}, \dots, x_{l,i} x_{l,j} x_{l,k}, \dots]$$

$$\begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \\ \vdots \\ w_{ij} \\ \vdots \\ w_{ijk} \\ \vdots \end{bmatrix}$$

$l = 1, \dots, m$; d denotes the dimension of input features; m denotes the number of samples

Weierstrass Approximation Theorem

- Every continuous function defined on a closed interval $[a, b]$ can be uniformly approximated as closely as desired by a polynomial function.

Notes:

- For high dimensional input features (large d value) and high polynomial order, the number of polynomial terms becomes explosive! (i.e., grows exponentially)
- For high dimensional problems, polynomials of order larger than 3 is seldom used.

Example 3



Example 3

Training set $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^m$

2nd order polynomial model

$\{x_1=0, x_2=0\} \rightarrow \{y=-1\}$
 $\{x_1=1, x_2=1\} \rightarrow \{y=-1\}$
 $\{x_1=1, x_2=0\} \rightarrow \{y=+1\}$
 $\{x_1=0, x_2=1\} \rightarrow \{y=+1\}$

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_{12} x_1 x_2 + w_{11} x_1^2 + w_{22} x_2^2$$

$$= \begin{bmatrix} 1 & x_1 & x_2 & x_1 x_2 & x_1^2 & x_2^2 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_{12} \\ w_{11} \\ w_{22} \end{bmatrix}$$

Python
demo 3

Stack the 4 training samples as a matrix

Python function automatically adds offset for you!
Order of the terms is slightly different too.

$$\mathbf{P} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & x_{1,1}x_{1,2} & x_{1,1}^2 & x_{1,2}^2 \\ 1 & x_{2,1} & x_{2,2} & x_{2,1}x_{2,2} & x_{2,1}^2 & x_{2,2}^2 \\ 1 & x_{3,1} & x_{3,2} & x_{3,1}x_{3,2} & x_{3,1}^2 & x_{3,2}^2 \\ 1 & x_{4,1} & x_{4,2} & x_{4,1}x_{4,2} & x_{4,1}^2 & x_{4,2}^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

• Python Poly function **automatically adds** offset for you! Order of the terms is slightly different too.

• Example 3 is continued below.

Quick Polynomial Regression Summary

- After you found P from previous slide, replace X with P in Ridge Regression and you get the below formula.
- X_{new} has to be converted to P_{new} too. m and d is now referring to P not X!

Ridge Regression in Primal Form (m > d)

For $\lambda > 0$,

Learning: $\hat{\mathbf{w}} = (\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^T \mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{P}(\mathbf{X}_{new})) = \mathbf{P}_{new} \hat{\mathbf{w}}$

Ridge Regression in Dual Form (m < d)

For $\lambda > 0$,

Learning: $\hat{\mathbf{w}} = \mathbf{P}^T (\mathbf{P} \mathbf{P}^T + \lambda \mathbf{I})^{-1} \mathbf{y}$

Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{P}(\mathbf{X}_{new})) = \mathbf{P}_{new} \hat{\mathbf{w}}$

For Regression Applications

- Learn **continuous** valued y using either primal form or dual form
- Prediction: $\hat{f}_{\mathbf{w}}(\mathbf{P}(\mathbf{X}_{new})) = \mathbf{P}_{new} \hat{\mathbf{w}}$

For Classification Applications

- Learn **discrete** valued y ($y \in \{-1, +1\}$) or **Y** (one-hot) using either primal form or dual form
- Binary Prediction: $\hat{f}_{\mathbf{w}}^c(\mathbf{P}(\mathbf{X}_{new})) = \text{sign}(\mathbf{P}_{new} \hat{\mathbf{w}})$
- Multi-Category Prediction: $\hat{f}_{\mathbf{w}}^c(\mathbf{P}(\mathbf{X}_{new})) = \arg \max_{k=1, \dots, C} (\mathbf{P}_{new} \hat{\mathbf{w}}(:, k))$
- It's the same thing just that you have to find P first to replace X.

Example 3 (cont'd)

Example 3 (cont'd)

Training set

2nd order polynomial model

$$P = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & x_{1,1}x_{1,2} & x_{1,1}^2 & x_{1,2}^2 \\ 1 & x_{2,1} & x_{2,2} & x_{2,1}x_{2,2} & x_{2,1}^2 & x_{2,2}^2 \\ 1 & x_{3,1} & x_{3,2} & x_{3,1}x_{3,2} & x_{3,1}^2 & x_{3,2}^2 \\ 1 & x_{4,1} & x_{4,2} & x_{4,1}x_{4,2} & x_{4,1}^2 & x_{4,2}^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$\hat{w} = P^T (PP^T)^{-1} y$$

$$= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 6 & 3 & 3 \\ 1 & 3 & 3 & 1 \\ 1 & 3 & 1 & 3 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -4 \\ 1 \\ 1 \end{bmatrix}$$

Python demo 3

Example 3 (cont'd)

Prediction

Test set

Test point 1: $\{x_1 = 0.1, x_2 = 0.1\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

Test point 2: $\{x_1 = 0.9, x_2 = 0.9\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

Test point 3: $\{x_1 = 0.1, x_2 = 0.9\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

Test point 4: $\{x_1 = 0.9, x_2 = 0.1\} \rightarrow \{y = \text{class } -1 \text{ or } +1?\}$

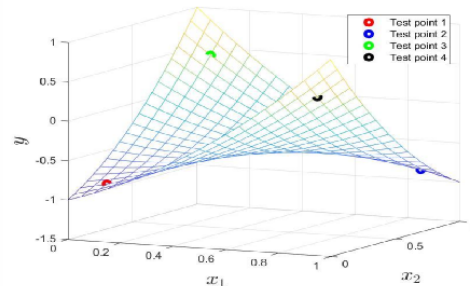
$$\hat{y} = P_{\text{new}} \hat{w}$$

$$= \begin{bmatrix} 1 & 0.1 & 0.1 & 0.01 & 0.01 & 0.01 \\ 1 & 0.9 & 0.9 & 0.81 & 0.81 & 0.81 \\ 1 & 0.1 & 0.9 & 0.09 & 0.01 & 0.81 \\ 1 & 0.9 & 0.1 & 0.09 & 0.81 & 0.01 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 1 \\ -4 \\ 1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & x_1 & x_2 & x_1x_2 & x_1^2 & x_2^2 \end{bmatrix}$$

$$\hat{f}_w^c(P(X_{\text{new}})) = \text{sign}(\hat{y}) = \text{sign} \begin{bmatrix} -0.82 \\ -0.82 \\ 0.46 \\ 0.46 \end{bmatrix}$$

$$= \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix} \begin{array}{l} \text{---> Class } -1 \\ \text{---> Class } -1 \\ \text{---> Class } +1 \\ \text{---> Class } +1 \end{array}$$

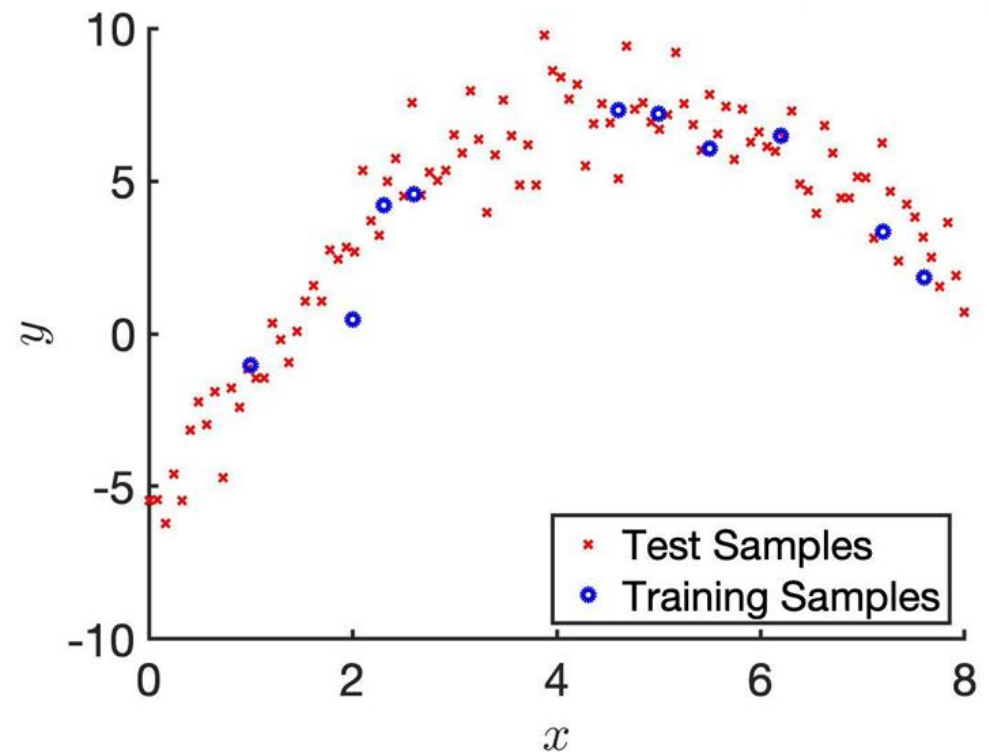


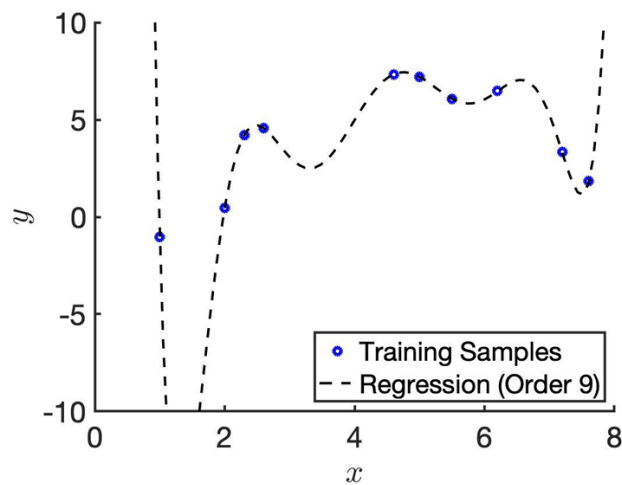
Python Packages

1. sklearn.preprocessing (PolynomialFeatures)
2. np.sign
3. sklearn.model_selection (train_test_split)
4. sklearn.preprocessing (OneHotEncoder)

Overfitting

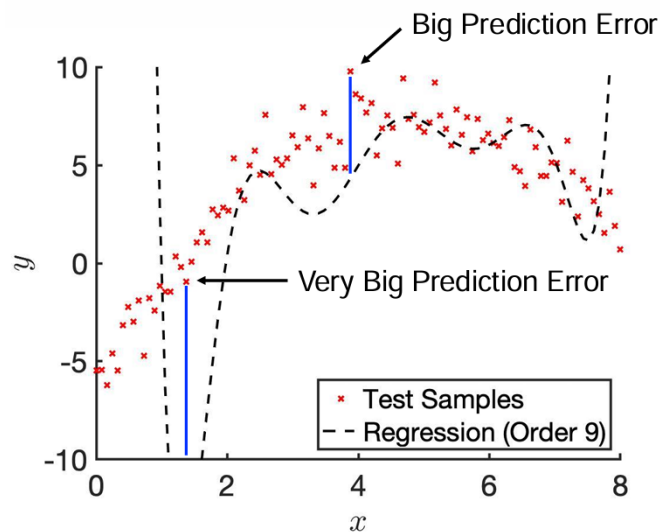
In practice we usually want the training set to be larger than the test set.





	Training Set Fit	
Order 9	Good	

^ This is order 9 polynomial regression. Fits training data really well.



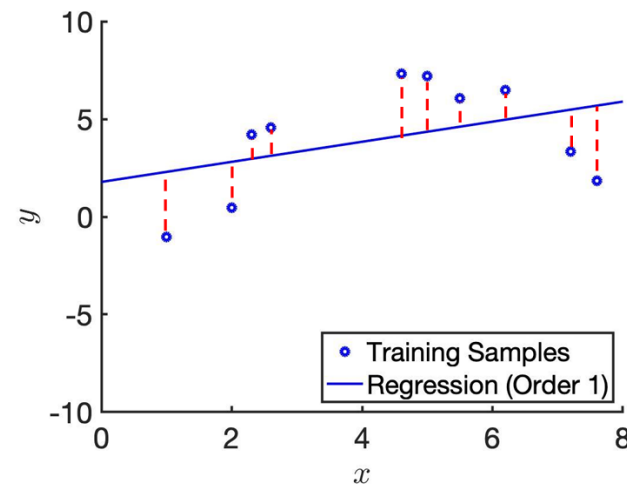
	Training Set Fit	Test Set Fit
Order 9	Good	Bad

But it only fits well to training set and not new unseen test data.

Overfitting: Good training performance, bad test performance.

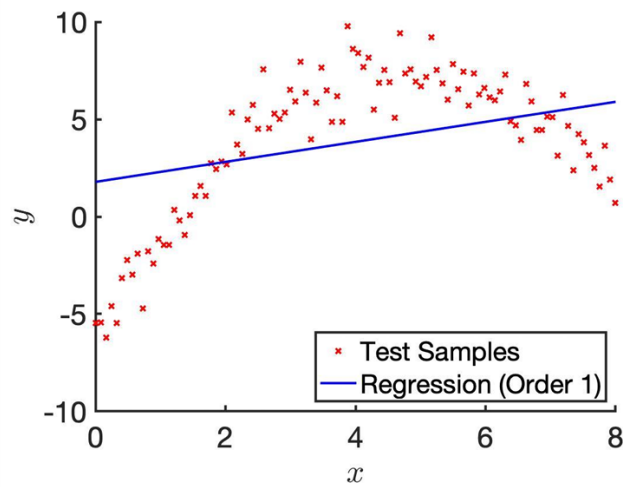
- If we take one of the blue lines and compute the square of its length, this is called “squared error” for that particular data point.
- If we average squared errors across all the red crosses, it’s called mean squared error (MSE) in the test set.

Underfitting



	Training Set Fit	Test Set Fit
Order 9	Good	Bad
Order 1	Bad	

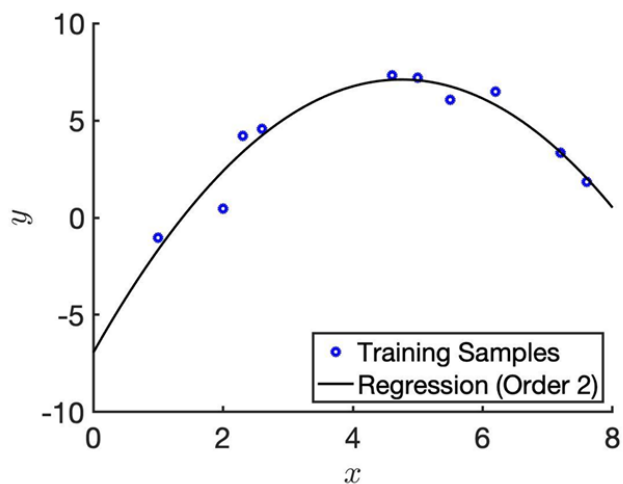
^ Order 1 Polynomial Regression (Also called Linear Regression).



	Training Set Fit	Test Set Fit
Order 9	Good	Bad
Order 1	Bad	Bad

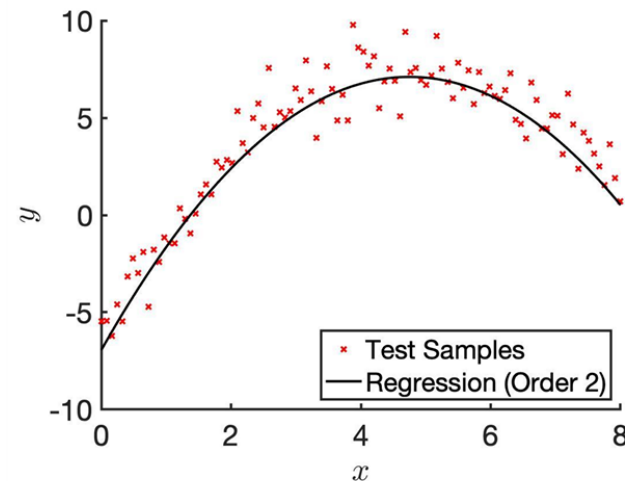
Underfitting: Bad training performance, bad test performance.

Just Nice Fitting



	Training Set Fit	Test Set Fit
Order 9	Good	Bad
Order 1	Bad	Bad
Order 2	Good	

^ 2nd order Polynomial Regression Model



	Training Set Fit	Test Set Fit
Order 9	Good	Bad
Order 1	Bad	Bad
Order 2	Good	Good

Just Nice Fitting: Good performance on BOTH training and test data.

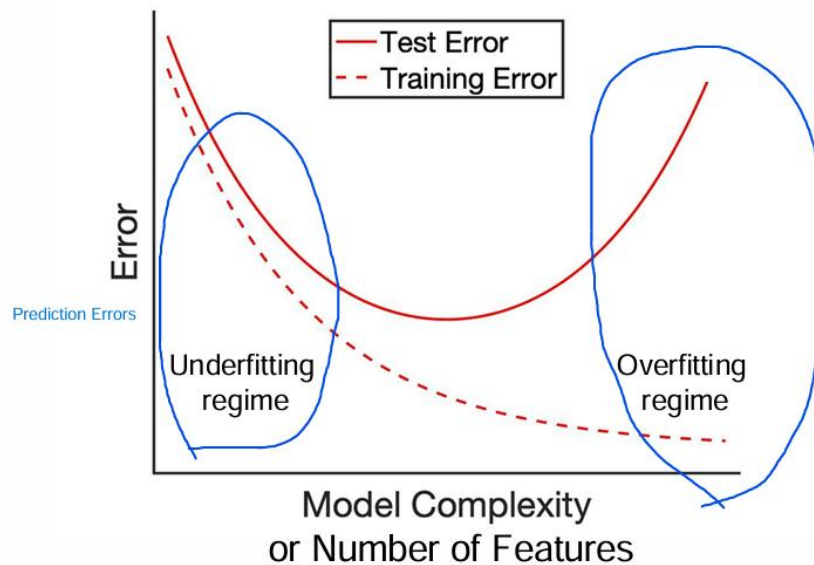
Reasons for Overfitting & Underfitting

- Overfitting occurs when model predicts the training data well, but predicts new data (e.g., from test set) poorly
- Reason 1: Model is too complex for the data. Previous example: Fit order 9 polynomial to 10 data points.
- Reason 2: Too many features but number of training samples too small. Even linear model can overfit, e.g., linear model with 9 input features (i.e., w is 10-D) and 10 data points in training set => data might not be enough to estimate 10 unknowns well (**Under-Determined**)
- Solutions: Use **simpler** models (e.g., lower order polynomial), or use **regularization** (see next part of lecture).

^ This is why test set is important in ML, to detect overfitting issues.

- Underfitting is the inability of trained model to predict the targets in the training set
- Reason 1: Model is too simple for the data. Previous example: Fit order 1 polynomial to 10 data points that came from an order 2 polynomial. **Solution:** Try more complex model.
- Reason 2: Features are not informative enough. Like the features are **not related to** what you are predicting. Imagine if you are trying to predict how smart someone is, but the features you choose is their hairstyle, food they eat, etc. **Solution:** Try to develop more informative features.

Overfitting/ Underfitting Schematic



Underfitting: Both Test & Training Errors are high.

Overfitting: Test Errors are high, but Training Errors are low.

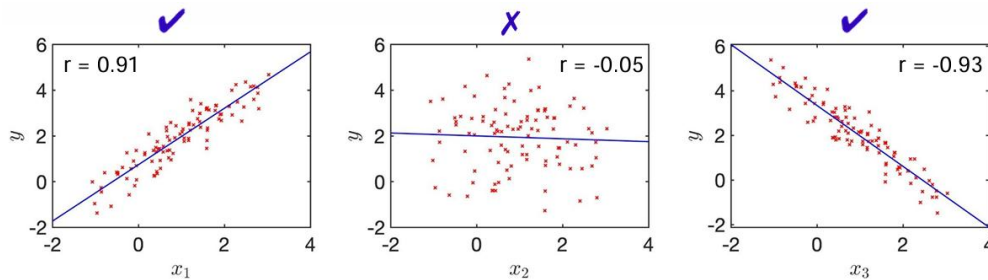
To reduce overfitting, **reduce** number of features using **feature selection**.

Feature Selection

- Less features might reduce overfitting → Want to discard useless features & keep good features, so perform feature selection.
- Feature selection procedure:
 - Step 1: feature selection in training set.
 - Step 2: fit model using selected features in training set.
 - Step 3: evaluate trained model using test set.
- Very common mistake:
 - Feature selection with test set (or full dataset) leads to inflated performance
 - Do not perform feature selection with test data

Selecting Features with Pearson Correlation Coefficient, r

- Given features x , we want to predict target y
- Assume x & y both continuous
- Compute Pearson's correlation coefficient between **each feature** & target y in the training set
- x_1, x_2, x_3 below are the "each feature" mentioned above.



Pearson's correlation r measures linear relationship between two variables

There are 2 options after you perform the Pearson's Correlation.

Two options:

- Option 1: Pick K features with largest **absolute** correlations
- Option 2: Pick all features with absolute correlations $> C$
- K & C are "magic" numbers set by the ML practitioner

- Other metrics besides Pearson's correlation are possible (Pearson is just the most common one)

Regularization

- Regularization is an umbrella term that includes methods forcing learning algorithm to build less-complex models.
- Motivation 1: Solve an ill-posed problem. For example, estimate 10th order polynomial with just 5 datapoints.
- Motivation 2: Reduce overfitting.
- For example, in previous lecture, we added $\lambda \mathbf{w}^T \mathbf{w}$:

$$\underset{\mathbf{w}}{\operatorname{argmin}} (\mathbf{P}\mathbf{w} - \mathbf{y})^T (\mathbf{P}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$$

- Minimizing with respect to $\mathbf{w}$, primal solution is

$$\hat{\mathbf{w}} = (\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^T \mathbf{y}$$

- For $\lambda > 0$, matrix **IS ALWAYS** invertible (Motivation 1)
- Regularization makes it ALWAYS invertible. This is true as long as λ is more than 0.

$\hat{\mathbf{w}}$ (the weight matrix) might also perform better in test set, i.e., reduces overfitting (Motivation 2) - will show example later

- Consider minimization from previous slide

$$\underset{\mathbf{w}}{\operatorname{argmin}} (\mathbf{P}\mathbf{w} - \mathbf{y})^T (\mathbf{P}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$$

Sum of Errors of the Data

Cost function quantifying data fitting error in training set

Regularization

- $\mathbf{w}^T \mathbf{w} = w_0^2 + w_1^2 + \dots + w_d^2$ — L2 - Regularization

^ L2 - Regularization

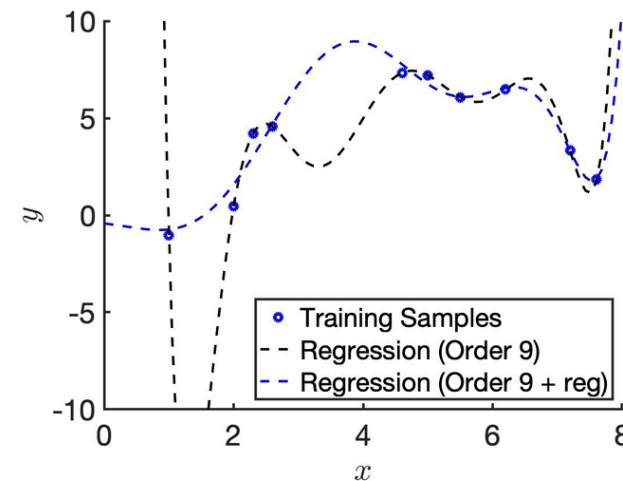
- Encourage $w_0, \dots, w_d$ to be small (also called shrinkage or weight-decay) => constrain model complexity
- More generally, most machine learning algorithms can be formulated as the following optimization problem:

$$\underset{\mathbf{w}}{\operatorname{argmin}} \text{Data-Loss}(\mathbf{w}) + \lambda \text{Regularization}(\mathbf{w})$$

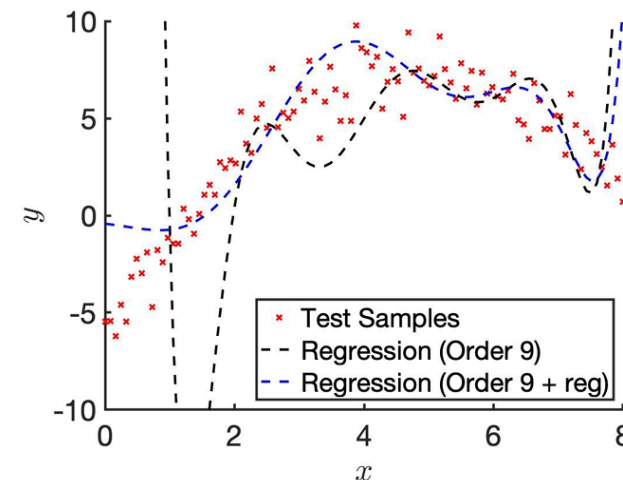
- Data-Loss(w)** quantifies fitting error to training set given parameters $\mathbf{w}$: smaller error => better fit to training data
- Regularization(w)** penalizes more complex models

When $w_0, w_1, w_2, w_3, \dots$ are very small due to large λ , these terms like w_2, w_3 , etc refers to the quadratic and cubic terms. By having lesser of them, the model is less quadratic and cubic thus less complexity.

Regularization Example



	Training Set Fit	Test Set Fit
Order 9	Good	Bad
Order 9, $\lambda = 1$	Good	

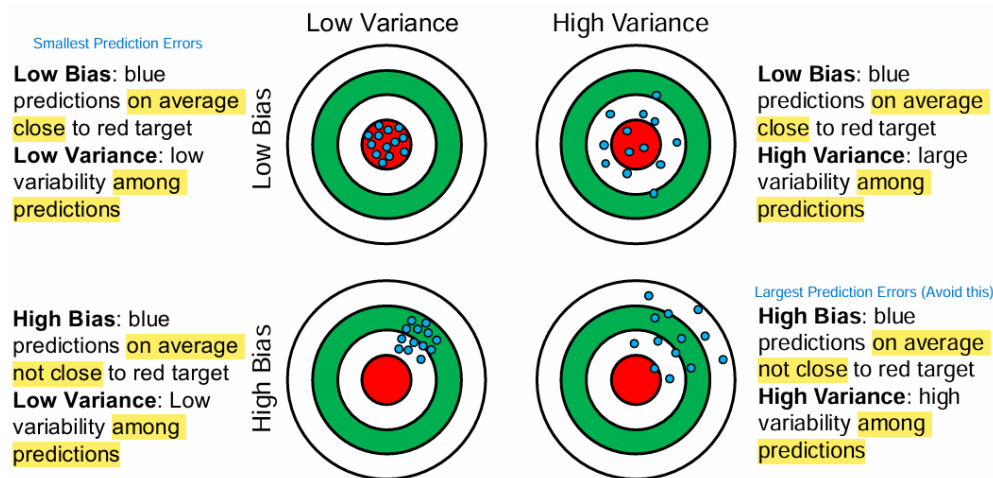


	Training Set Fit	Test Set Fit
Order 9	Good	Bad
Order 9, $\lambda = 1$	Good	Good

Python demo

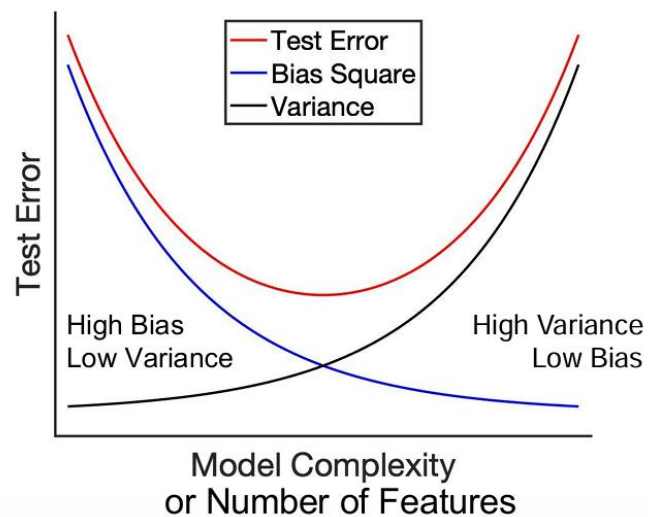
Bias versus Variance

- Suppose we are trying to predict red target below:



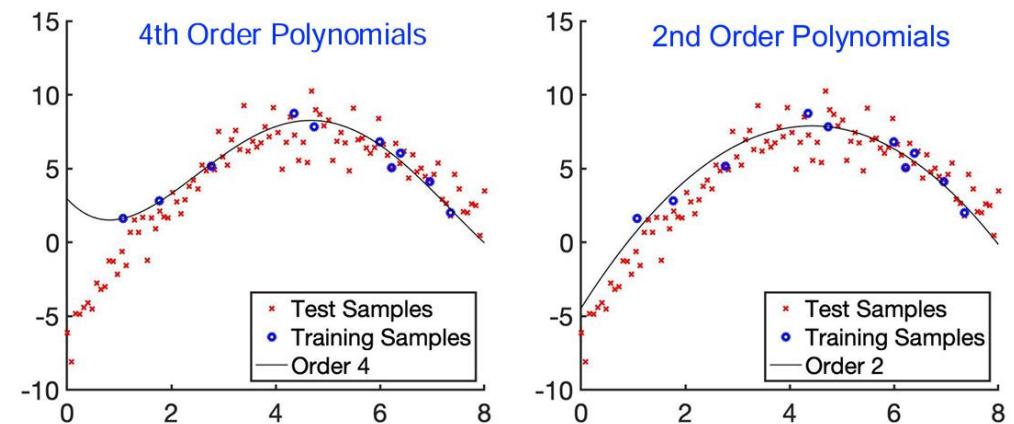
Bias + Variance Trade off

- Test error = Bias Squared + Variance + Irreducible Noise

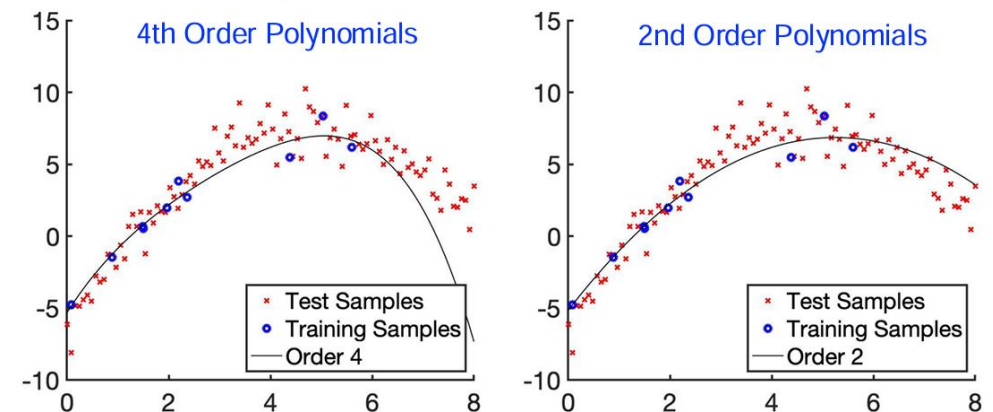


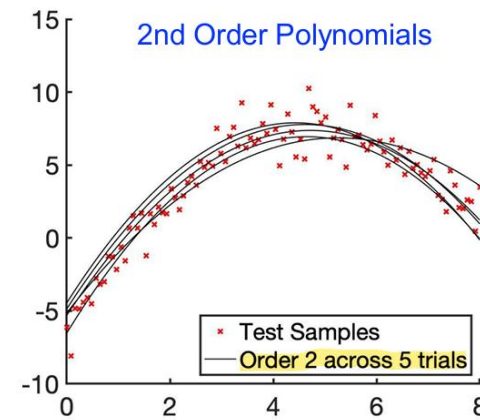
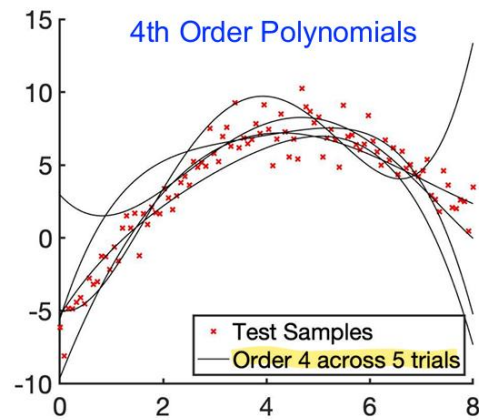
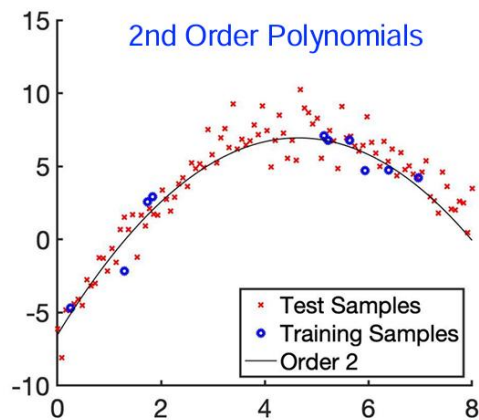
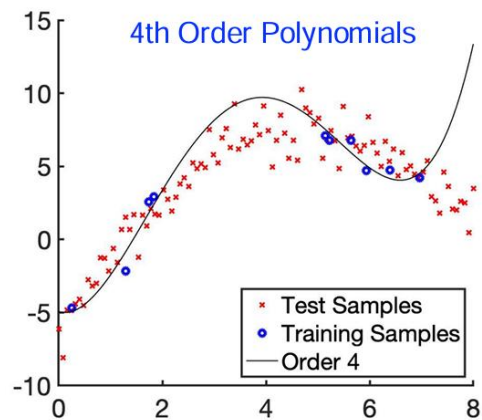
Bias + Variance Example

- Simulate data from order 2 polynomial (+ noise)
- Randomly sample 10 training samples each time
- Fit with order 2 polynomial
- Fit with order 4 polynomial

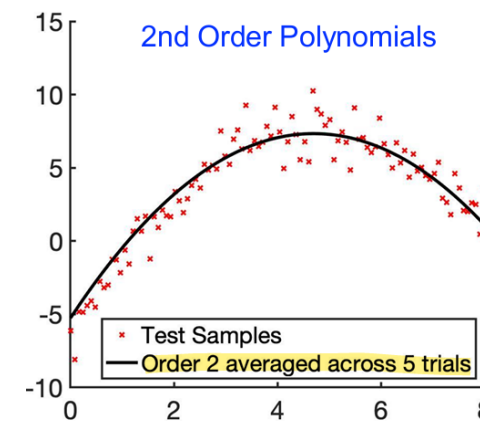
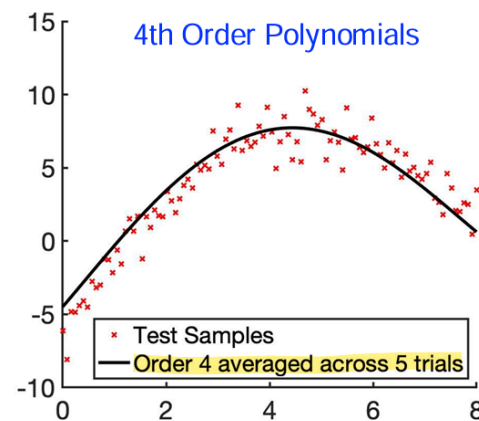
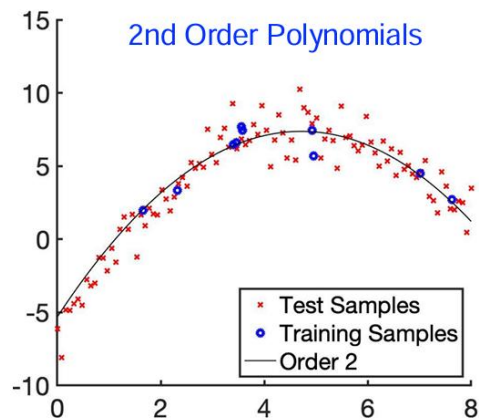
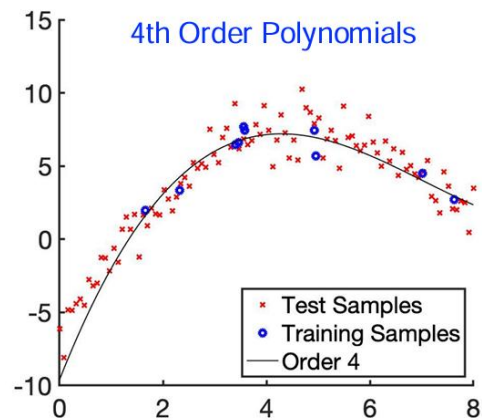


Randomly sample 10 training samples each time but Test Set stays the same. Likewise for all the slides below.





^ We can see that Order 2 polynomial is low variance and order 4 polynomial is high variance.



^ Both order polynomials are low bias! Bias is calculated using average, that's why u see only one curve and the curve fits well for both.

To conclude, Order 2 achieve lower test error due to lower variance.

Bias-Variance Decomposition Theorem

- Test error = Bias Squared + Variance + Irreducible Noise
- Mathematical details in optional uploaded material (won't be tested)
- “Variance” refers to variability of prediction models across **different training sets**.
 - In previous example, every time the training set of 10 samples changes, the trained model changes.
- “Variance” quantifies variability **across trained models**.
- “Bias” refers to how well an **average prediction model** will perform
 - In previous example, every time the training set of 10 samples changes, the trained model changes.
- If we **average the trained models**, how well will this average trained model perform against real data?
- “Irreducible Noise” reflects the fact that even if we are perfect modelers, it might not be possible to predict target y with 100% accuracy from feature(s) x .

Loss Function & Learning Model

- For polynomial regression (previous lectures)

$$\begin{aligned}\operatorname{argmin}_{\mathbf{w}} C(\mathbf{w}) &= \operatorname{argmin}_{\mathbf{w}} (\mathbf{P}\mathbf{w} - \mathbf{y})^T (\mathbf{P}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w} \\ &= \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^m (\mathbf{p}_i^T \mathbf{w} - y_i)^2 + \lambda \mathbf{w}^T \mathbf{w}\end{aligned}$$

- Let $f(\mathbf{x}_i, \mathbf{w})$ be the prediction of target y_i from features $\mathbf{x}_i$ for i -th training sample. For example, suppose $f(\mathbf{x}_i, \mathbf{w}) = \mathbf{p}_i^T \mathbf{w}$, then above becomes

$$\operatorname{argmin}_{\mathbf{w}} C(\mathbf{w}) = \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^m (f(\mathbf{x}_i, \mathbf{w}) - y_i)^2 + \lambda \mathbf{w}^T \mathbf{w}$$

L is the loss function, and it refers to the squared error of a single data point. It takes in predicted and true value of y .

- Let $L(f(\mathbf{x}_i, \mathbf{w}), y_i)$ be the penalty for predicting $f(\mathbf{x}_i, \mathbf{w})$ when true value is y_i . For example, suppose $L(f(\mathbf{x}_i, \mathbf{w}), y_i) = (f(\mathbf{x}_i, \mathbf{w}) - y_i)^2$, then above becomes

$$\operatorname{argmin}_{\mathbf{w}} C(\mathbf{w}) = \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^m L(f(\mathbf{x}_i, \mathbf{w}), y_i) + \lambda \mathbf{w}^T \mathbf{w}$$

- From previous slide

$$\operatorname{argmin}_{\mathbf{w}} C(\mathbf{w}) = \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^m L(f(\mathbf{x}_i, \mathbf{w}), y_i) + \lambda \mathbf{w}^T \mathbf{w}$$

- To make it even more general, we can write

$$\operatorname{argmin}_{\mathbf{w}} C(\mathbf{w}) = \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^m L(f(\mathbf{x}_i, \mathbf{w}), y_i) + \lambda R(\mathbf{w})$$

Cost Function Loss Function Learning Model Regularization

Keywords: Cost Function, Loss Function, Learning Model, Regularization.

- **Learning model** f reflects our belief about the relationship between the features $\mathbf{x}_i$ & target y_i
- **Loss function** L is the penalty for predicting $f(\mathbf{x}_i, \mathbf{w})$ when the true value is y_i
- **Regularization** R encourages less complex models
- **Cost function** C is the final optimization criterion we want to minimize
- **Optimization** routine to find solution to cost function

Motivation for Gradient Descent

- Different learning function f , loss function L & regularization R give rise to different learning algorithms.
- In polynomial regression (previous lectures), optimal $\mathbf{w}$ can be written with the following “closed-form” formula (primal solution):

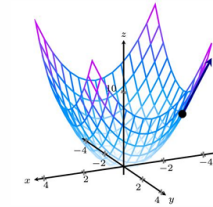
$$\hat{\mathbf{w}} = (\mathbf{P}_{train}^T \mathbf{P}_{train} + \lambda \mathbf{I})^{-1} \mathbf{P}_{train}^T \mathbf{y}_{train}$$

- For other learning function f , loss function L & regularization R , optimizing $C(\mathbf{w})$ might not be so easy.
- Usually have to estimate $\mathbf{w}$ iteratively with some algorithm.
- Optimization workhorse for modern machine learning is gradient descent.

Gradient Descent Algorithm

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \dots, w_d]^T$

- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w}) = \begin{pmatrix} \frac{\partial C}{\partial w_1} \\ \frac{\partial C}{\partial w_2} \\ \vdots \\ \frac{\partial C}{\partial w_d} \end{pmatrix}$



- $\nabla_{\mathbf{w}} C(\mathbf{w})$ is **vector & function** of $\mathbf{w}$
- $\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is increasing most rapidly, so $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is decreasing most rapidly

- Gradient Descent:

Initialize $\mathbf{w}_0$ and learning rate η ; Lowercase eta symbol

```

while true do
    Compute  $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$ 
    if converge then
        | return  $\mathbf{w}_{k+1}$ 
    end
end

```

According to multi-variable calculus, if **eta** is not too big, then $C(\mathbf{w}_{k+1}) < C(\mathbf{w}_k) \Rightarrow$ **we get better $\mathbf{w}$ after each iteration**

- Gradient descent **can only find local minimum**

- Because gradient = 0 at local minimum, so $\mathbf{w}$ won't change after that
- IMPORTANT: This algorithm **cannot guarantee** to find the global minimum. But sometimes local minimum = global minimum.
- Many variations of gradient descent, e.g., change how gradient is computed or learning rate η decreases with increasing k .
- Just like finding lambda for regularization, validation set is used to find the optimal learning rate to use.

Disadvantages of High Learning Rate:

- A high learning rate can cause the model to overshoot optimal parameters, leading to oscillations around the minimum or even divergence (exploding loss).
- The model may jump between local minima, failing to settle at a stable solution.
- Large updates to weights (look at how W_{k+1} is computed) can destabilize the training process, making it harder to find a reliable optimum.

Disadvantages of Slow Learning Rate:

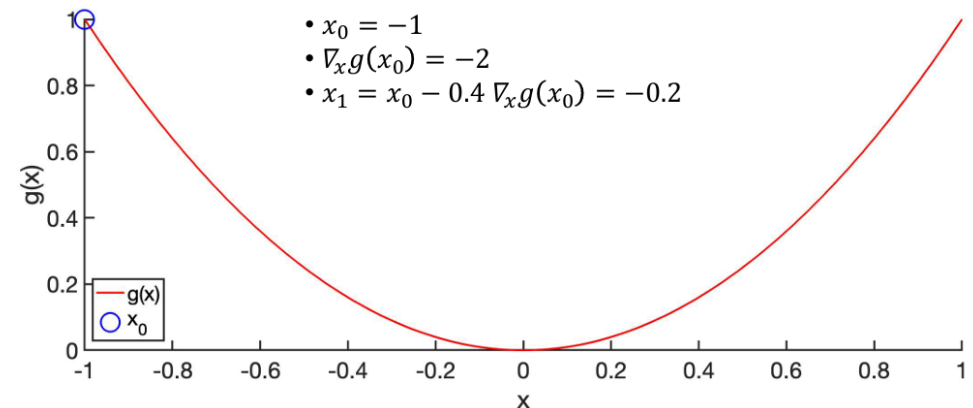
- A low learning rate requires many iterations to reach the minimum, increasing training time and computational cost.
- Training may stall if the learning rate is too small to make meaningful progress toward convergence.
- A fixed low learning rate might not adapt well to different phases of training (e.g., early exploration vs. fine-tuning).

Possible Convergence Criteria (Reasons to break the while loop)

- Set maximum iteration k .
 - Check percentage or absolute change in $\mathcal{C}$ below a threshold.
 - Check percentage or absolute change in $\mathbf{w}$ below a threshold.
- ^ If below some threshold or the change in w is so small, then stop.

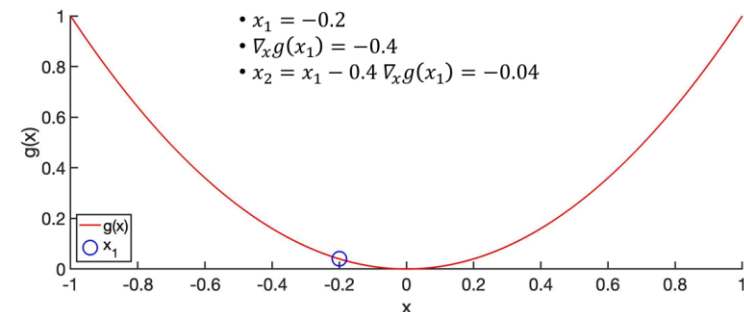
Find x to minimize $g(x) = x^2$

- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
 - Gradient $\nabla_x g(x) = 2x$
 - Initialize $x_0 = -1$, learning rate $\eta = 0.4$
 - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k)$



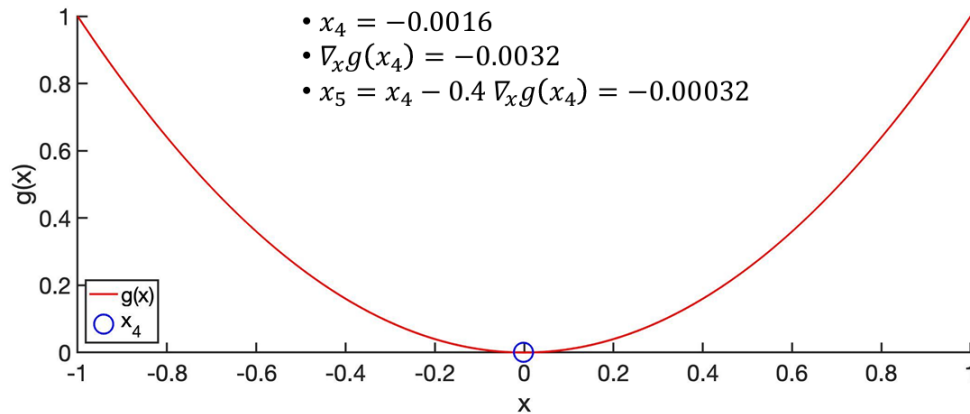
^ Basically derive the first derivative of the function, initialise learning rate as 0.4, start as $x = -1$. Use the formula and perform iterations.

- Obviously minimum corresponds to $x = 0$, but let's do gradient descent
 - Gradient $\nabla_x g(x) = 2x$
 - Initialize $x_0 = -1$, learning rate $\eta = 0.4$
 - At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k)$



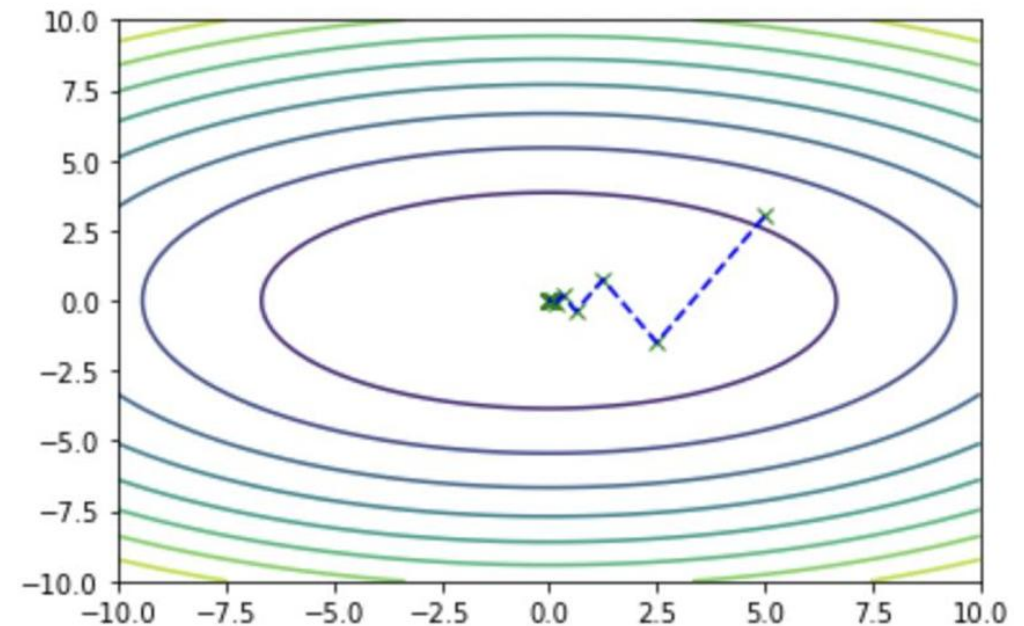
- Obviously minimum corresponds to $x = 0$, but let's do gradient descent

- Gradient $\nabla_x g(x) = 2x$
- Initialize $x_0 = -1$, learning rate $\eta = 0.4$
- At each iteration, $x_{k+1} = x_k - \eta \nabla_x g(x_k)$



- When learning rate is just nice, there will be rapid convergence.
- When learning rate is too small, convergence will be very slow.
- When learning rate is too big, calculations may overshoot local minimum which will result in slower convergence, or it may never converge at all.

Gradient Descent: Quadratic Function



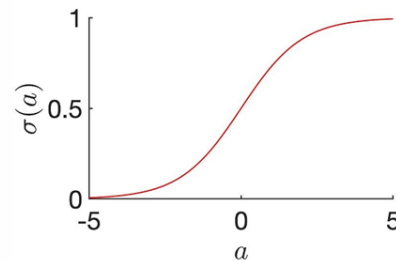
^ Convergence to the foot of the valley

Different Learning Models

- Different learning models $f(\mathbf{x}_i, \mathbf{w})$ reflect our beliefs about the relationship between the features $\mathbf{x}_i$ and target y_i
 - For example, $f(\mathbf{x}_i, \mathbf{w}) = \mathbf{p}_i^T \mathbf{w}$ assumes polynomial relationship between features and target
- Suppose we are performing classification (rather than regression), so y_i is class -1 or class 1
 - $\mathbf{p}_i^T \mathbf{w}$ is number between $-\infty$ to ∞ .
Can use to classify Class -1 and Class 1.
 - Can use **sigmoid function** to map $\mathbf{p}_i^T \mathbf{w}$ to between 0 and 1:

$$f(\mathbf{x}_i, \mathbf{w}) = \sigma(\mathbf{p}_i^T \mathbf{w})$$
$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

If $\mathbf{p}_i^T \mathbf{w}$ @ w (or a in the graph) is big, it is map to 1, if it is small, it is map to 0. If it is 0, it is map to 0.5.



^ Sigmoid function is an example of activation function.

- If $\mathbf{p}_i^T \mathbf{w}$ @ w (or a in the graph) is big, it is map to 1, if it is small, it is map to 0. If it is 0, it is map to 0.5.

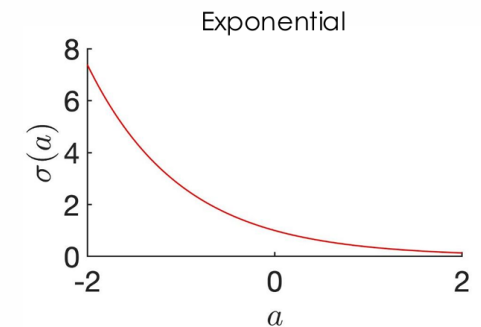
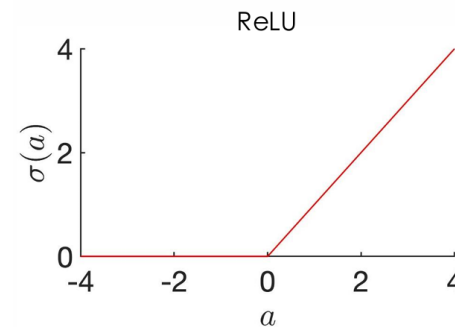
- Non-Linear: Allows neural networks to model complex, non-linear patterns.

- Output Range [0,1]: Makes it easy to interpret outputs as probabilities.

- More generally, in **one layer neural network**: $f(\mathbf{x}_i, \mathbf{w}) = \sigma(\mathbf{p}_i^T \mathbf{w})$, where **activation function** σ can be **sigmoid** or **some other functions** & **p** is linear

Other types of activation functions

- $f(\mathbf{x}_i, \mathbf{w}) = \sigma(\mathbf{p}_i^T \mathbf{w})$, where σ can be different functions:
- Rectified linear unit (ReLU): $\sigma(a) = \max(0, a)$ Popular
- Exponential: $\sigma(a) = \exp(-a)$ All can do anything, multi-classification, etc. It is just whether it is optimal.



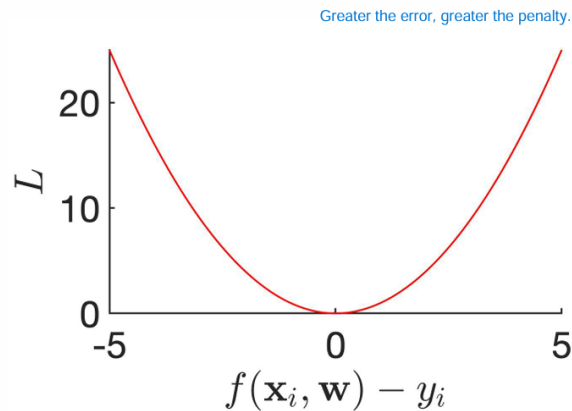
- All the activation functions can do anything, multi-classification, etc. It's only a matter of optimality.

- ReLU is very popular.

Different Loss Functions

- Different loss functions $L(f(\mathbf{x}_i, \mathbf{w}), y_i)$ encodes the penalty when we predict $f(\mathbf{x}_i, \mathbf{w})$ but the true value is y_i

– $L(f(\mathbf{x}_i, \mathbf{w}), y_i) = (f(\mathbf{x}_i, \mathbf{w}) - y_i)^2$ is called the **square error loss**



- The one we usually use is square error. Remember, loss function refers to a single data point.

Other types of loss function

- Say if you are performing classification rather than regression, square error loss makes lesser sense. Instead, you can use:

Binary Loss (0 - 1 Loss)

– Binary loss (or 0-1 loss): $L(f(\mathbf{x}_i, \mathbf{w}), y_i) = \begin{cases} 0 & \text{if } f(\mathbf{x}_i, \mathbf{w}) = y_i \\ 1 & \text{if } f(\mathbf{x}_i, \mathbf{w}) \neq y_i \end{cases}$

Here, instead of checking the actual value of y_{pred} and y_{actual} , convert y_{pred} to class first, then check if the class is same as $y_{\text{actual_class}}$. Here y_i is either class -1 or +1.

- Basically 0 penalty if our prediction function of x_i and w equals to y_i , 1 otherwise.

– In practice, **hard to constrain** $f(\mathbf{x}_i, \mathbf{w})$ to be exactly -1 or 1 , so we can declare “victory” if $f(\mathbf{x}_i, \mathbf{w})$ & y **have the same sign**:

$$L(f(\mathbf{x}_i, \mathbf{w}), y_i) = \begin{cases} 0 & \text{if } f(\mathbf{x}_i, \mathbf{w})y_i > 0 \\ 1 & \text{if } f(\mathbf{x}_i, \mathbf{w})y_i < 0 \end{cases}$$

^ Like binary loss function, only can be 0 and 1 depending on the sign of $f(x_i, w)$ and y_i . If they **both have same sign**, 0 penalty, 1 otherwise.

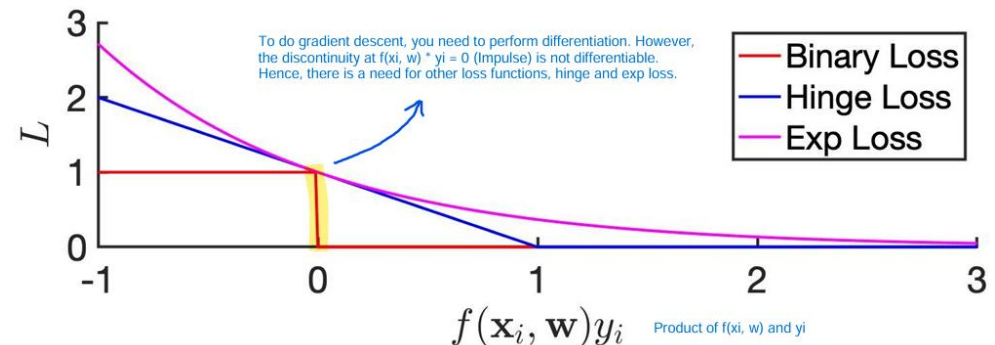
But binary loss is not differentiable, so there is 2 other possibilities:

Hinge Loss:

$$L(f(\mathbf{x}_i, \mathbf{w}), y_i) = \max(0, 1 - f(\mathbf{x}_i, \mathbf{w})y_i)$$

Exponential Loss:

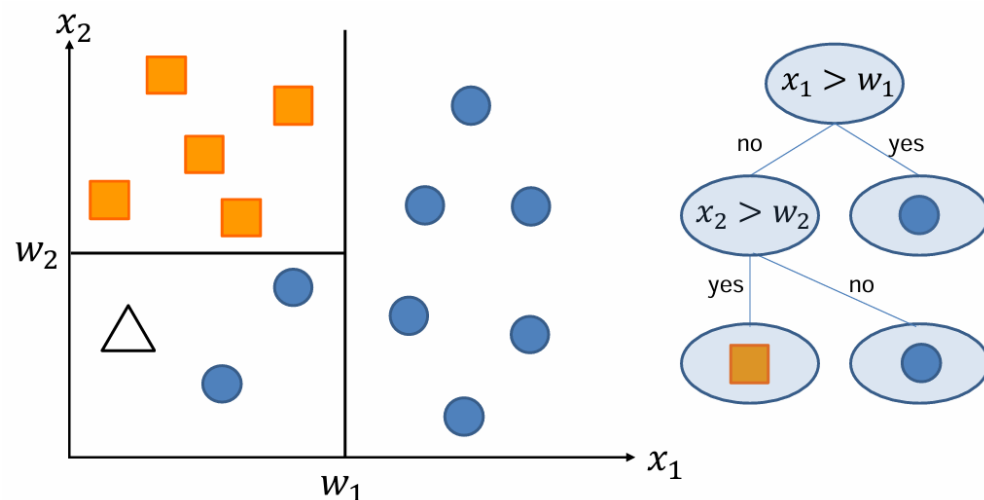
$$L(f(\mathbf{x}_i, \mathbf{w}), y_i) = \exp(-f(\mathbf{x}_i, \mathbf{w})y_i)$$



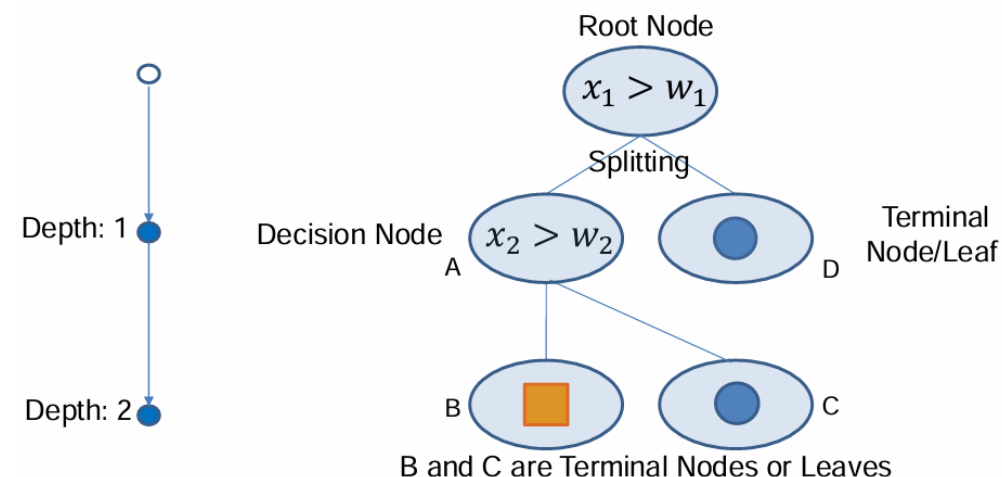
^ To do gradient descent, you need to perform differentiation. However, the discontinuity at $f(x_i, w) * y_i = 0$ (Impulse) is not differentiable. Hence, there is a need for other loss functions, hinge and exponential loss.

Decision Trees (Classification)

- Goal: predict class labels using two features x_1 & x_2
- In our test set, we observe datapoint Δ shown below. How would the decision tree classify this point?



Basic Terminologies



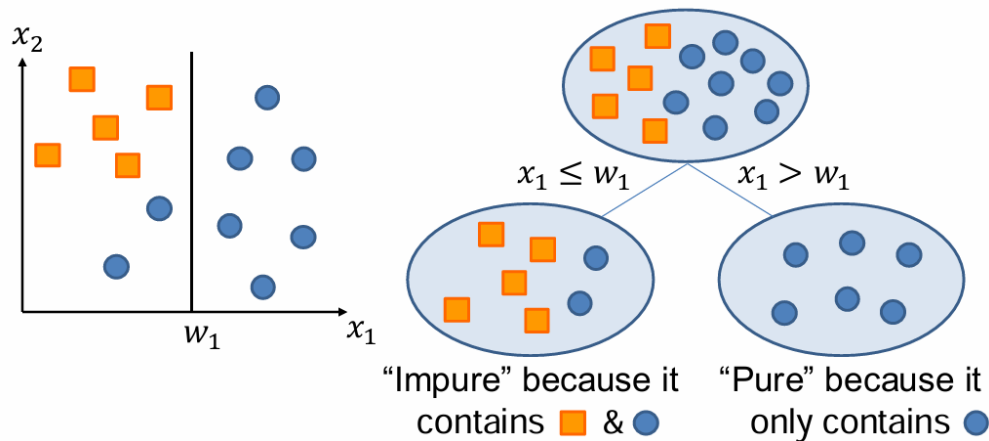
A-B-C forms a sub-tree or branch. A is parent node of B and C; B and C are children nodes of A. D is a leaf/terminal node.

Building a Classification Decision Tree

- Classification tree learning (or tree induction or tree growing) is the construction of a classification tree given training data.
- For a given training set, **there can be many trees with 0 training error**, so we prefer less “complex” trees. Like the previous example, instead of checking x_1 as the first decision, we check x_2 instead, giving us a new tree with 0 errors.
- **Complexity** can be defined as **number of nodes** in the tree
- Finding smallest tree is **computationally hard**, so we typically use some **greedy algorithm**, but it is **not guaranteed** to find the best tree.

- We need to first define the concept of node impurity

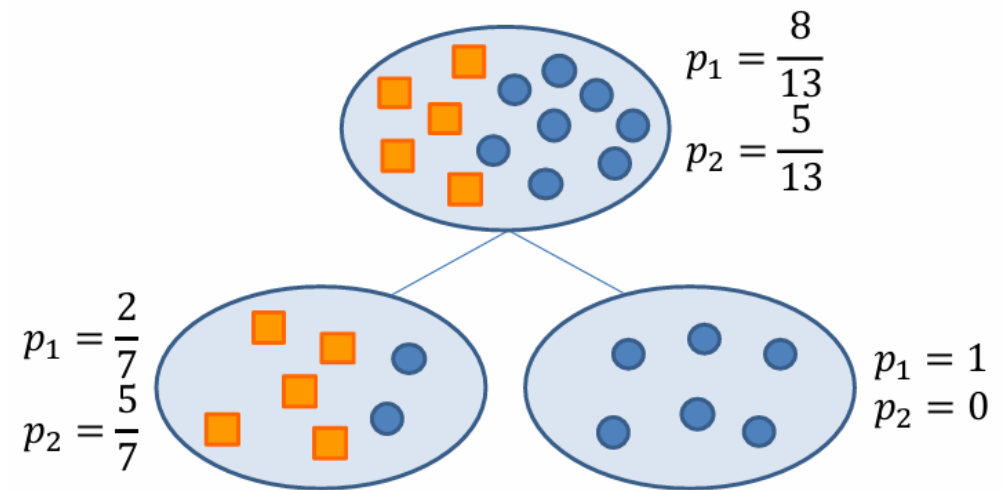
Node Impurity



- “Purity” is desirable because if a node contains only training data from one class, then prediction for training data in the node is perfect.

Node Impurity Measures

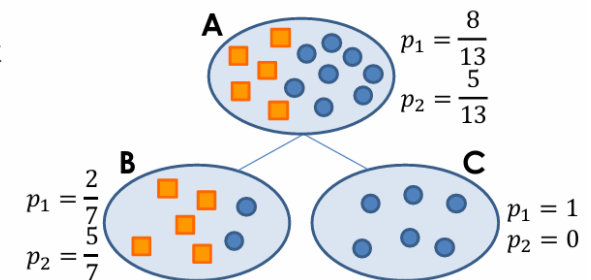
- Let blue circle be class 1 & orange square be class 2.
- For a particular node m , let p_i be the fraction (or probability) of data samples in node m belonging to class i .
- Let Q_m be impurity of node m
- 3 impurity measures: Gini, entropy, misclassification rate



Gini Impurity

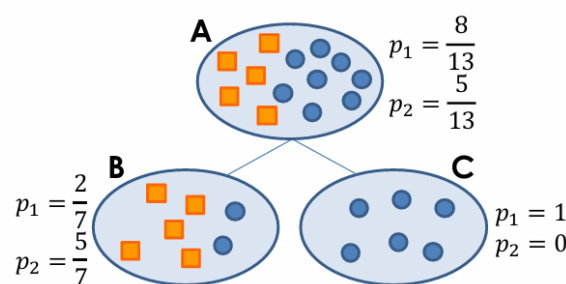
- Let $K = \#$ classes, define $Q_m = 1 - \sum_{i=1}^K p_i^2 = 1 - p_1^2 - p_2^2$
- Node A: $Q_A = 1 - (8/13)^2 - (5/13)^2 = 0.4734$
- Node B: $Q_B = 1 - (2/7)^2 - (5/7)^2 = 0.4082$
- Node C: $Q_C = 1 - 1^2 - 0^2 = 0$
- Overall Gini (depth 1) = fraction of data samples in node B $\times Q_B$ + fraction of data samples in node C $\times Q_C$ Formula for finding Gini Impurity for the whole level/depth.
 - $\left(\frac{7}{13}\right) \times 0.4082 + \left(\frac{6}{13}\right) \times 0 = 0.2198$
- Observe lower impurity at depth 1 compared with root
- Same Gini formula for more than 2 classes:

$$Q_m = 1 - \sum_{i=1}^K p_i^2$$



Entropy

- Let $K = \#$ classes, define $Q_m = -\sum_{i=1}^K p_i \log_2 p_i$ NOTE: log based 2
- Node A: $Q_A = -(8/13)\log_2(8/13) - (5/13)\log_2(5/13) = 0.9612$
- Node B: $Q_B = -(2/7)\log_2(2/7) - (5/7)\log_2(5/7) = 0.8631$
- Node C: $Q_C = -1\log_2 1 - 0\log_2 0 = 0$
- Overall entropy (depth 1) = fraction of data samples in node B x Q_B + fraction of data samples in node C x Q_C
 - $\left(\frac{7}{13}\right) \times 0.8631 + \left(\frac{6}{13}\right) \times 0 = 0.4648$

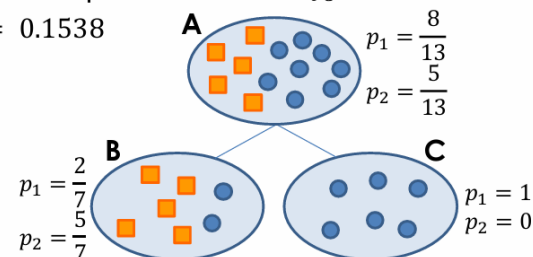


Misclassification Rate

- Let $K = \#$ classes, define $Q_m = 1 - \max_i p_i = 1 - \max(p_1, p_2)$
- Node A: $p_1 > p_2$, so best classification = class 1 $\Rightarrow Q_A = 1 - 8/13 = 5/13$
- Node B: $p_2 > p_1$, so best classification = class 2 $\Rightarrow Q_B = 1 - 5/7 = 2/7$
- Node C: $p_1 > p_2$, so best classification = class 1 $\Rightarrow Q_C = 1 - 1 = 0$
- Overall misclassification rate (depth 1) = fraction of data samples in node B x Q_B + fraction of data samples in node C x Q_C

$$\left(\frac{7}{13}\right) \times \left(\frac{2}{7}\right) + \left(\frac{6}{13}\right) \times 0 = 0.1538$$

- Observe lower impurity at depth 1 compared with root
- Same misclassification rate formula for more than 2 classes: $Q_m = 1 - \max_i p_i$



Classification Tree Learning

Algorithm: Classification Tree Learning

Input: Impurity measure Q , parameter max_depth & training set

Output: Tree

```

1 root ← all training samples
2 for d ← 1 to max_depth do Outer Loop: Loop through the levels/depth of the tree.
3   for each leaf node m at depth d - 1 do Inner Loop: Loop through each node in the previous level/depth.
4     Find best feature & best threshold, so splitting
5     node m into two reduces the most impurity
6     Use decision rule to distribute training samples
7     from node m across two new leaf nodes
6 return tree
  
```

Advantages / Disadvantages

Advantages

- Easy to visualize & understand tree
- Can work with a mix of **continuous and discrete** data
- Less data cleaning required
- Makes less assumptions about the relationship between features & target

Disadvantages

- Trees can become overly complex resulting in overfitting
- Trees can be unstable, e.g., small changes in training data can result in very different trees. Random Forest solve this (Below).

^ It becomes overfitting when the tree made is so complex that the decision node is so calibrated nicely to the training set that it does so well for the training set but fails when it comes to testing set.

How to reduce overfitting?

One or more of the following can help reduce overfitting:

- Set maximum depth for the tree, hence iterations.
- Set minimum number of samples for splitting a leaf node, e.g., if leaf node has less than 10 samples, then do not split node.
- Set minimum decrease in impurity, e.g., if selecting the best feature & threshold does not improve impurity by at least 1%, then do not split the leaf node.
- Instead of looking at all features when considering how to split a leaf node, we can randomly look at a subset (e.g., square root of the total number of features)

Decision Trees (Regression)

- Classification trees seek to predict discrete variables (i.e., classification)
- Regression trees seek to predict continuous variables (i.e., regression)
- Can use same approach as before, but instead of minimizing impurity, we can try to minimize mean square error (MSE)

- Suppose there are J_m training samples in a leaf node m of the regression tree with target values $y_1, y_2, \dots, y_{J_m}$
 - We can predict $\hat{y}_m = \frac{1}{J_m} \sum_{j=1}^{J_m} y_j$ Average
 - Then **MSE of node m** is given by $S_m = \frac{1}{J_m} \sum_{j=1}^{J_m} (y_j - \hat{y}_m)^2$
- Across all leaf nodes, **total MSE $S = \sum_m \frac{J_m}{N} S_m$** , where N is the total number of data samples

Regression Tree Learning

- Algorithm is basically the same as classification tree learning
- Various approaches to reduce overfitting for classification tree also apply here.

Algorithm: Regression Tree Learning

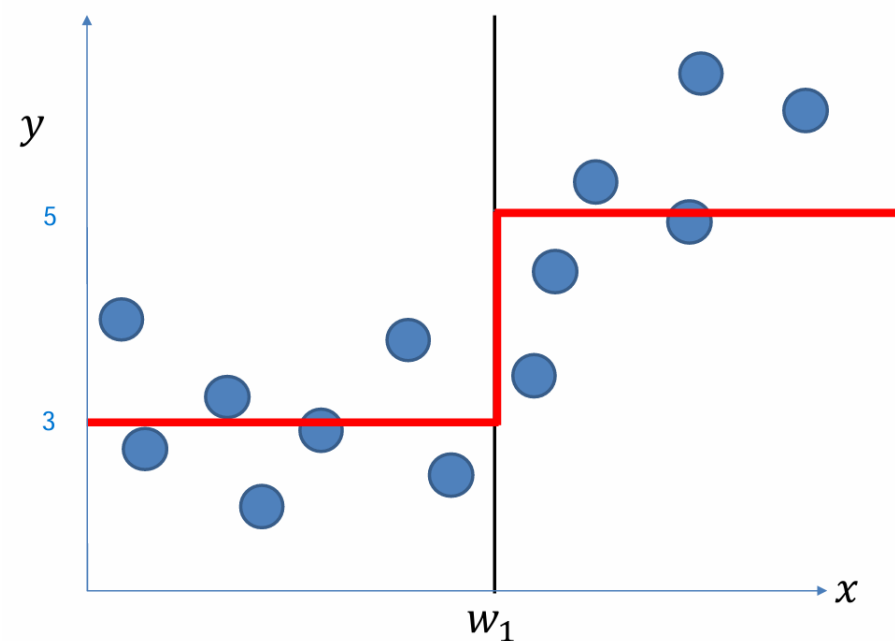
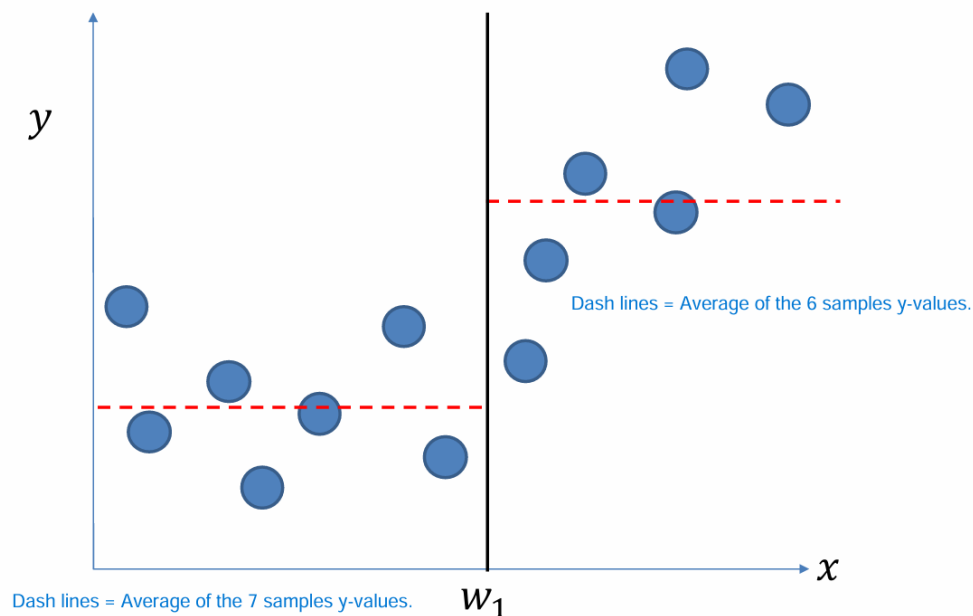
Input: parameter *max_depth* & training set

Output: Tree

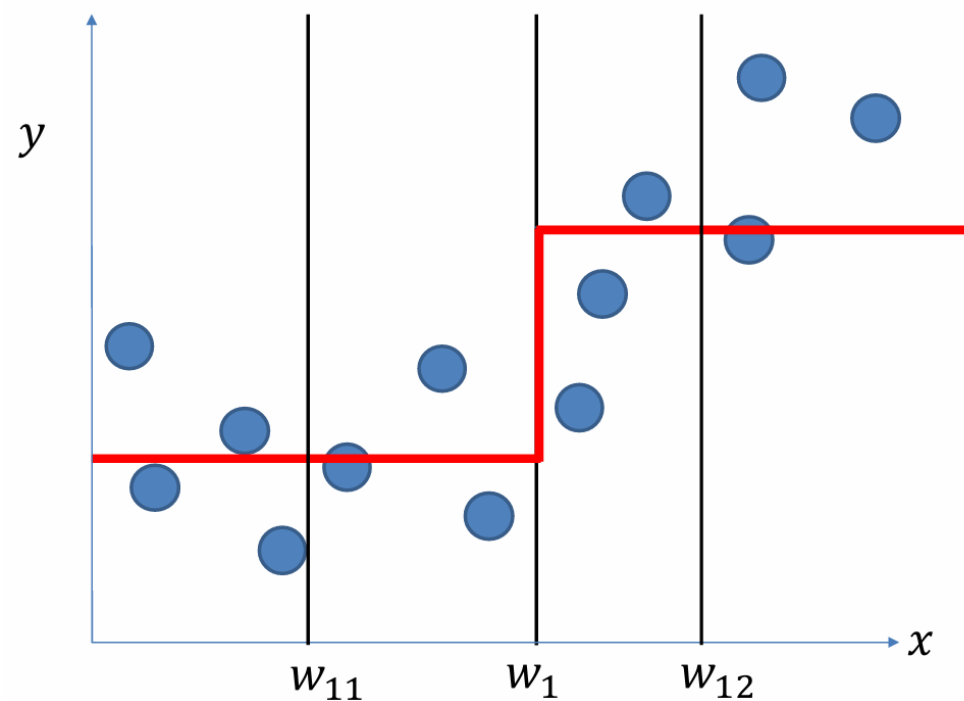
```
1 root ← all training samples
2 for  $d \leftarrow 1$  to  $max\_depth$  do
3   for each leaf node  $m$  at depth  $d - 1$  do
4     Find best feature & best threshold, so splitting
       node  $m$  into two reduces MSE the most
5     Use decision rule to distribute training samples
       from node  $m$  across two new leaf nodes
6 return tree
```

Same as classification tree learning, but instead of reducing impurity, we reduce MSE.

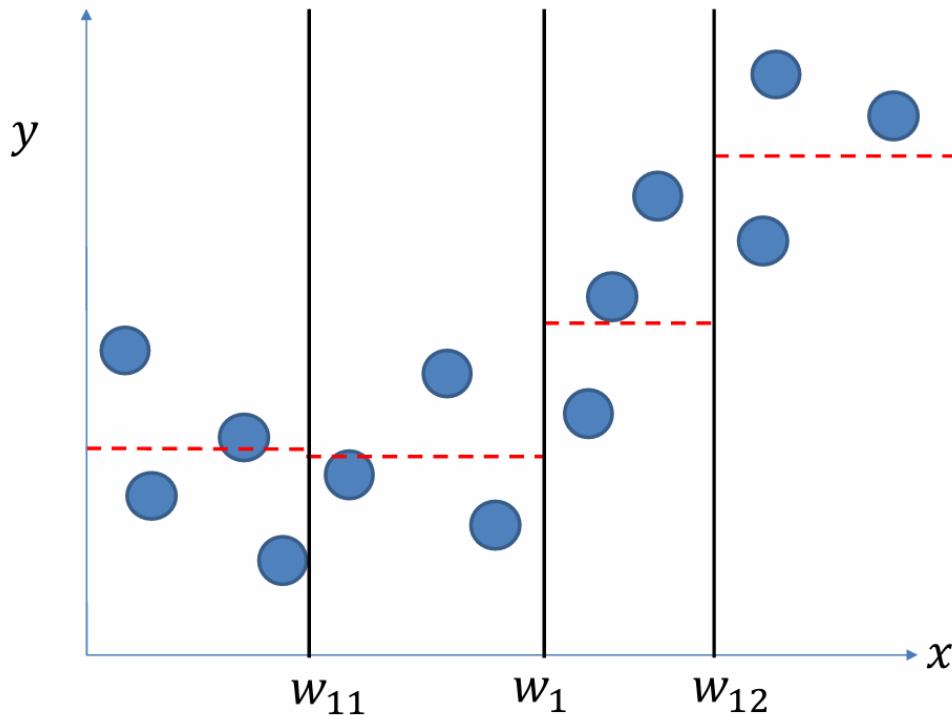
Regression Tree Visualisation



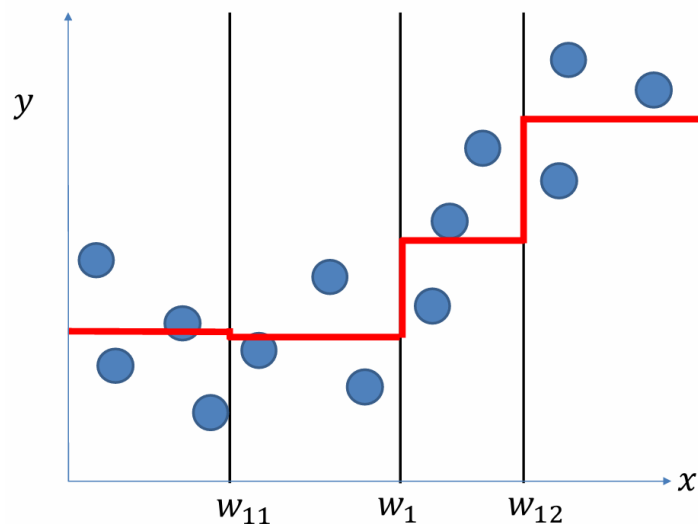
- The red line drawn is the prediction of the regression tree at a certain depth.
- For e.g. If the new data point feature x is less than w_1 , the tree will predict 3 as the output, else 5
- Looks like a step function now, but you can make it more accurate by breaking x into more sections. (Next slide)



- Imagine turning a discrete signal into analog by using quantization levels. The more you split, the more it looks like a curve.
- w_{11} , w_1 , w_{12} are the decision nodes.



- Calculate the averages for each section.



Regression Tree Example

Example of Regression Tree

- Consider house prices in Singapore.
- Target variable is Price P .
- Attributes are House Size S and Number of Rooms R .

	House Size ('000 sq ft)	Num of Rooms	Price ('000,000 SGD)
1	0.5	2	0.19
2	0.6	1	0.23
3	1.0	3	0.28
4	2.0	5	0.42
5	3.0	4	0.53
6	3.2	6	0.75
7	3.8	7	0.80

- Note that I have arranged the data points in increasing order of P , which so happens to be increasing order of S as well. However, this is not the same order as that of R .

Calculation of MSE for House Size Split

Split at $S = 0.75$

- Focus first on the House Size attribute S . If we set the threshold at $\tau = 0.75$, then the targets of the two classes are $\{0.19, 0.23\}$ and $\{0.28, 0.42, 0.53, 0.75, 0.80\}$. The individual conditional MSEs are

Use formula below, find average first.

$$\text{MSE}_{P|S < 0.75} = 4 \times 10^{-4} \quad \text{and} \quad \text{MSE}_{P|S \geq 0.75} = 0.0385.$$

- Thus, the averaged conditional MSE with a split of S at 0.75 is

$$\text{MSE}_{P|S(0.75)} = \frac{2}{7} \text{MSE}_{P|S < 0.75} + \frac{5}{7} \text{MSE}_{P|S \geq 0.75} = 0.0276.$$

- Sweep through all possible thresholds τ to determine the best threshold for attribute S .

In this case, 2.5 gives the best split by minimizing MSE.

$\text{MSE}_{P S(0.55)}$	$\text{MSE}_{P S(0.75)}$	$\text{MSE}_{P S(1.5)}$	$\text{MSE}_{P S(2.5)}$	$\text{MSE}_{P S(3.1)}$	$\text{MSE}_{P S(3.5)}$
0.0402	0.0276	0.0145	0.0102	0.0116	0.0325

$$S_m = \frac{1}{J_m} \sum_{j=1}^{J_m} (y_j - \hat{y}_m)^2 \quad \text{MSE } S = \sum_m \frac{J_m}{N} S_m$$

- In this example, the tree first split occurs at $S = 0.75$, resulting in 2 sets $\{0.19, 0.23\}$ (Left Child Node) & $\{0.28, 0.42, 0.53, 0.75, 0.80\}$ (Right Child Node).

- To find individual MSE of left child, first find the average so $0.19 + 0.23 / 2 = 0.21$.

- Next, find the mean squared errors of each data point:

$$[(0.19 - 0.21)^2 + (0.23 - 0.21)^2] / 2 = 0.0004$$

- Same for the right child node.

- To find the MSE of the whole depth/level: Take the ratio of samples in that node multiply by their individual MSE. Do this for all nodes in that level. For example, $2/7 * 0.0004 + 5/7 * 0.0385 = 0.0276$.

Calculation of MSE for # Rooms Split

- Rearrange the target variables in order of the house sizes. Doing so we get $(0.23, 0.19, 0.28, 0.53, 0.42, 0.75, 0.80)$. Now we sweep through all possible thresholds τ for R to get the following averaged conditional MSEs.
- We get the following table.

$MSE_{P R(1.5)}$	$MSE_{P R(2.5)}$	$MSE_{P R(3.5)}$	$MSE_{P R(4.5)}$	$MSE_{P R(5.5)}$	$MSE_{P R(6.5)}$
0.0435	0.0276	0.0145	0.0222	0.0116	0.0325



Choose the threshold with minimum MSE

^ MSE for # Rooms Split instead of House Size.

Where is the First Split?

- We should first split the dataset into two branches, the left branch indicating $S < 2.5$ and the right with $S \geq 2.5$.
- Split the dataset into two sub-datasets and we may decide to stop or split the R feature.
- If we decide to stop, then for any new/test house with a house size of < 2.5 , we will predict that its price is the average of the houses in our training set whose size is < 2.5 , i.e.,

$$(0.19 + 0.23 + 0.28 + 0.42) / 4 = 0.28.$$

- For a new/test house with a house size of ≥ 2.5 , we will predict that its price is the average of the houses in our training set whose size is ≥ 2.5 , i.e.,

$$(0.53 + 0.75 + 0.80) / 3 = 0.6933.$$

Python
demo

^ After finding the most optimal split, for prediction, you will take the average of that section as output.

How to reduce instability?

- Since the tree is very sensitive to even the smallest change in data, we shall include noises, disturbance to the tree for training (Perturbed Training) so that we force the model to learn how to handle such cases.
- From the one training set, make small changes and create M training sets with noise. (Perturbations)
- With M trees, output is the average prediction across M trees for regression and most frequent class for classification.

- For both classification & regression trees, **small perturbations to data can result in very different trees** => low bias, high variance
- To reduce variance, we can perturb training set to generate M perturbed training sets From the one training set, make small changes and create M training sets with noise.
 - Train one tree for each perturbed training set
 - Average predictions across the M trees?
- For example, if we have 100 **regression trees** (trained from 100 perturbed training sets)
 - Given features x from new test sample, the i -th tree predicts $f_i(x)$
 - Then final prediction is $\frac{1}{100} \sum_{i=1}^{100} f_i(x)$ Average Prediction
- For example, if we have 100 **classification trees** (trained from 100 perturbed training sets)
 - Given features x from new test sample, the i -th tree predicts $g_i(x)$
 - Then final prediction is the **most frequent class** among 100 predictions $g_1(x), g_2(x), \dots, g_{100}(x)$

Random Forest

- To perturb data, we can apply “bootstrapping” to the training set to create a new “bootstrapped” dataset.
- Bootstrapping is procedure in which we sample data **with replacement**.
- For example, given training set with 3 data samples $\{x_1, y_1\}, \{x_2, y_2\}, \{x_3, y_3\}$, a bootstrapped training set might comprise sample 1 $\{x_2, y_2\}$, sample 2 $\{x_2, y_2\}$ and sample 3 $\{x_1, y_1\}$.
- Bootstrapped dataset is the **same size** as original dataset.
- Bootstrapped dataset **might contain repeated** samples.
- Bootstrapped dataset **might not contain some samples** from original dataset.

Algorithm: Random Forest Learning

Input: parameter max_trees & N training samples

Output: Forest

```

1 for  $t \leftarrow 1$  to  $max\_trees$  do
2   | dataset  $\leftarrow$  bootstrap( $N$  training samples)
3   | trees[t]  $\leftarrow$  TreeLearning(dataset) Can be classification or regression tree learning function.
4 forest  $\leftarrow$  average(trees) Average for regression, Most Common Class for classification.
5 return forest

```

- To increase randomness, when training the trees, instead of looking at all features when considering how to split a node, we can randomly look at a subset (e.g., square root of the total number of features).

Performance Issues

- Dataset Partition:
 - Training/Validation/Testing
- Cross Validation
- Evaluation Metrics
 - Evaluating the quality of a trained machine learning system

Motivation

- We would like to train a Random Forest for face classification (i.e., to tell an image is a human face or not)

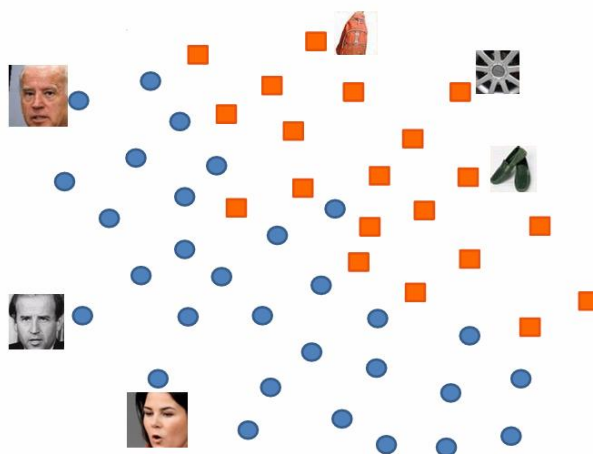


Faces



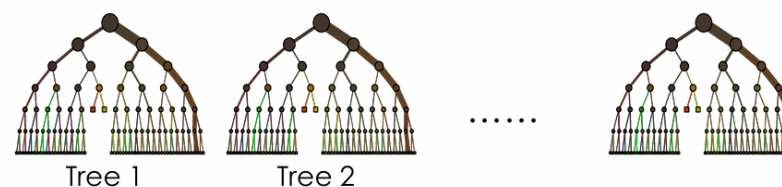
Non-faces

- We would like to train a Random Forest for face classification (i.e., to tell an image is a human face or not)
 - We will have **one dataset** to **train** the Random Forest



- We would like to train a Random Forest for face classification (i.e., to tell an image is a human face or not)
 - We will have **one dataset** to **train** the Random Forest
 - We will have tunable **(hyper)parameters** for the Random Forest. For example, **the number of trees** in the Random Forest
 - Shall we use 100 trees?
 - Shall we use 200 trees?
 - ...

We need to decide on the parameter



- Once we decide the number of trees, we will apply the Random Forest with the **selected parameter** on **unseen** test data.

Test Data



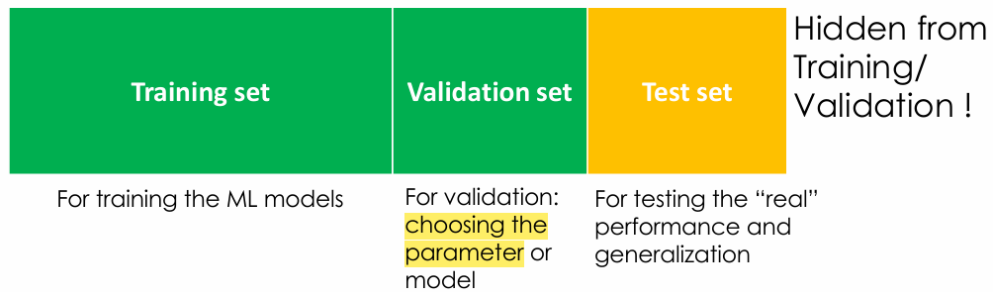
Yes!



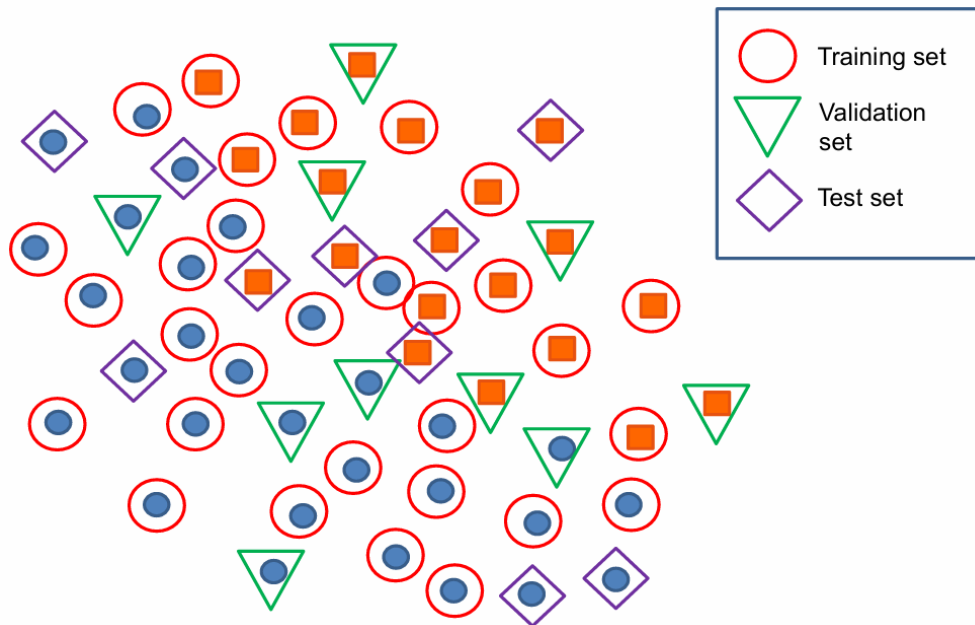
No!

Training, Validation, and Test

- In real-world application, we don't have test data, since they are unseen. Imagine developing a face detector app, you won't know whom you will test on
- In lab practice, we divide the dataset into three parts.



- Training Set and Validation Set is visible during training, while Test Set is hidden from the model during training.
- NEVER EVER touch test data during training/validation!!!



Example 1

- Assume I want to build a Random Forest. I have a parameter to decide: Shall I have 100 Trees or 200 Trees?

- What we do next is to use the training set to train two classifiers,

1. **C1**: Random Forest with 100 trees
2. **C2**: Random Forest with 200 trees

- They have the following accuracy:

1. **C1**: Random Forest with 100 trees: validation accuracy 90%
2. **C2**: Random Forest with 200 trees: validation accuracy 88%

- Which one to choose for real application, i.e., testing?

Answer: The one with higher validation accuracy, i.e., Random Forest with 100 trees!

Example 2

You need to decide what lambda to use for regularization, and you are deciding between 0.001 and 0.005. To know which to pick, you train 2 models using training set, one for $\lambda = 0.001$ and another for $\lambda = 0.005$. After that, you take the 2 models and test them on the validation set and see which is more accurate. After knowing which is more accurate, you can take it one step further by training a whole new model using both training and validation set combined along with the more accurate lambda (hyper-parameter). Then, test using test set.

K-fold Cross Validation

- In practice, we do the k-fold cross validation
- k is set by user, goal here is still to select hyper-parameter.
- In this example, k = 4, total dataset size is 800.

Step 1: Take out test set (200 samples) from the dataset

Step 2: We partition the remaining part (600 samples) of the dataset (after taking out the test set), into k equal parts (equal in terms of number of samples).

Step 3: We run k folds (i.e., k times) of experiments. Within each fold, we use one part as validation set, and the k-1 remaining parts as training set. We use different validation sets for different folds.

Fold 1	Train	Train	Train	Validation
Fold 2	Train	Train	Validation	Train
Fold 3	Train	Validation	Train	Train
Fold 4	Validation	Train	Train	Train

Order of the samples are kept the same across all folds.

- Each fold is the same 600 samples. Only the validation set within the fold is changed as we run k times of the experiment. Select different partitions as validation set. Each partition within a fold has 150 samples.

Step 3.1: Within each fold, if we have n parameter/model candidates, we will train n models, and we check their validation performance.

- Number of classifiers trained = $k * n = 4 * 2 = 8$

- In practice, we do the **k-fold cross validation**

Test	4-fold cross validation				Classifiers Trained	
	Fold 1	Train	Train	Train	Validation	C_1^1 C_2^1
	Fold 2	Train	Train	Validation	Train	C_1^2 C_2^2
	Fold 3	Train	Validation	Train	Train	C_1^3 C_2^3
	Fold 4	Validation	Train	Train	Train	C_1^4 C_2^4

1) C_1 : Random Forest with 100 trees

2) C_2 : Random Forest with 200 trees

Superscript represents the fold number.
Subscript represents the parameter number.

Step 4: We select the parameter/model with best average validation performance over k folds.

Example: which one to select for test?

	Fold 1 Accuracy on Validation Set 1	Fold 2 Accuracy on Validation Set 2	Fold 3 Accuracy on Validation Set 3	Fold 4 Accuracy on Validation Set 4	Average Accuracy on All Validation Sets
Classifier with Param1 (e.g. 100 trees)	88% C_1^1	89% C_1^2	93% C_1^3	92% C_1^4	90.5% ✓
Classifier with Param2 (e.g. 200 trees)	90% C_2^1	88% C_2^2	91% C_2^3	91% C_2^4	90%

^ Here, it shows Random Forest with 100 Trees is better.

Other Common Partitioning

- 10-Fold CV, 5-Fold CV, 3-Fold CV
- We may decide on the size of the test set, for example, 15%, 20%, 30% of the whole dataset, the rest for training/validation.

Test Performance

- The test set contains the examples that the learning algorithm has never seen before, so test performance shows **how well our model generalizes.**

Example: Xinchao uses k-fold cross validation to obtain an optimal parameter for his model (e.g., decision tree). This parameter, on the test set, achieves accuracy of 0.8. Yueming uses k-fold cross validation to obtain an optimal parameter for her model (e.g., random forest). This parameter, on the test set, achieves accuracy of 0.9. We can say, Yueming's model generalizes better than Xinchao's model.

Take home message:

- Validation performance -> Selecting parameters!
- Test performance -> The "real" performance of a model with selected parameter!
- Validation is however **NOT ALWAYS** used. Validation is used when you need to pick parameters or models. If you have no models or parameters to compare, you may consider partition the data into only training and test sets.

Evaluation Metrics

Regression:

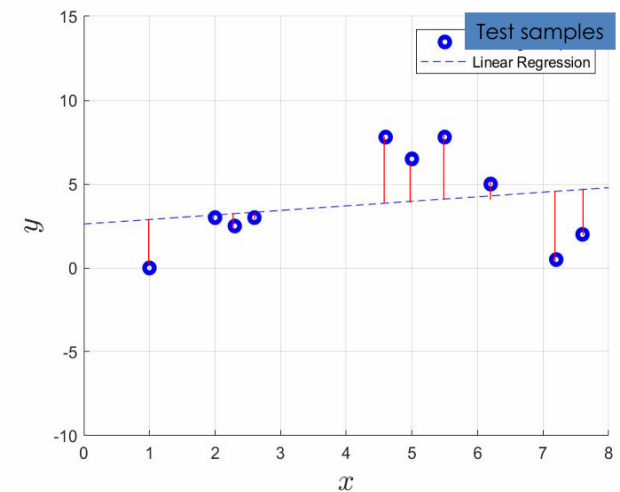
Mean Square Error

$$(\text{MSE} = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n})$$

Mean Absolute Error

$$(\text{MAE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n})$$

where y_i denotes the target output and $\hat{y}_i$ denotes the predicted output for sample i .



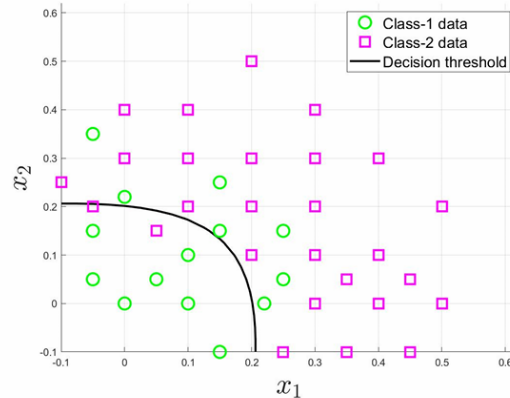
Classification:

- Decision threshold not the best as some of class-1 data is in class-2 data zone. Hence, the creation of Confusion Matrix to determine how well a classifier do. Changing the decision threshold will change the confusion matrix.
- Type I Error: In this context, the data is actually Class-2 but predicted as Class-1. (Since Class-1 is labelled Positive Class, it is False Positive)
- Type II Error: In this context, the data is actually Class-1 but it is predicted as Class-2. (Since Class-2 is labelled Negative Class, it is False Negative)

Classification

Class-1: Positive Class
Class-2: Negative Class

Confusion Matrix		
	Class-1 (predicted)	Class-2 (predicted)
Class-1 (actual)	7 (TP)	7 (FN)
Class-2 (actual)	2 (FP)	25 (TN)



TP: True Positive
FN: False Negative (i.e., **Type II Error**)
FP: False Positive (i.e., **Type I Error**)
TN: True Negative

- How many samples in the dataset have the real label of Class-2?

Ans: $2 + 25 = 27$. Total class 2 samples is the sum of the row, regardless of what class are predicted for them.

- How many samples are there in total?

Ans: $7 + 7 + 2 + 25 = 41$

- How many samples are correctly classified? How many are incorrectly classified?

Ans: Correct = $7 + 25 = 32$, Incorrect = $2 + 7 = 9$.

Binary Classification (Confusion Matrix)

Confusion Matrix for Binary Classification

	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)	
P (actual)	TP	FN	Recall $TP/(TP+FN)$
N (actual)	FP	TN	Accuracy $(TP+TN)/(TP+TN+FP+FN)$
	Precision $TP/(TP+FP)$		

- Binary here refers to 2 classes.

Binary Classification (Cost Matrix)

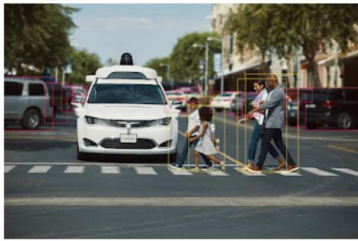
	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)	
P (actual)	$C_{p,p} * TP$	$C_{p,n} * FN$	Total cost: $C_{p,p} * TP +$ $C_{p,n} * FN +$ $C_{n,p} * FP +$ $C_{n,n} * TN$ First and last term usually 0 due to $C_{pp} = C_{nn} = 0$.
N (actual)	$C_{n,p} * FP$	$C_{n,n} * TN$	

- Main Idea: To assign different penalties for different entries. Higher penalties for more severe results. Smaller costs are preferred.
- Usually, $C_{p,p}$, and $C_{n,n}$, are set to 0 as they are correctly classified
- $C_{p,n}$, and $C_{n,p}$, may and may not equal.

Example

- Example of cost matrix

- Assume we would like to develop a self-driving car system
- We have an ML system that detects the pedestrians using camera, by conducting a binary classification
 - When it detects a person (positive class), the car should stop
 - When no person is detected (negative class), the car keeps going



Credit: automotiveworld.com

True Positive (cost $C_{p,p}$)

There is person, ML detects person and car stops

True Negative (cost $C_{n,n}$)

There is no person, car keeps going

False Positive (cost $C_{n,p}$)

There is no person, ML detects person and car stops

False Negative (cost $C_{p,n}$)

There is person, ML fails to detect person and car keeps going

$C_{n,p} \text{ ? } C_{p,n} (>, <, \text{ or } =)$ $C_{p,n}$ should be greater than $C_{n,p}$.

- In this example, $C_{p,n}$ should be greater than $C_{n,p}$. This is because the False Negative scenario is much more dangerous than False Positive.

- So, there is a need to be heavier/stronger punishment to prevent False Negative from happening.

- Usually what determines what is positive class is if "it happens" / "it is".

Unbalanced Data

	Class-1 (predicted)	Class-2 (predicted)
Class-1 (actual)	5 (TP)	5 (FN)
Class-2 (actual)	5 (FP)	985 (TN)

– Assume we have 1000 samples, of which 10 are positive and 990 are negative

– Accuracy = $990/1000 = 0.99$! Very high number!

– Yet, half of the Class-1 are Classified to Class-2!

The **goal is to highlight** the problems of the results!

In this case, we shall

1) Use cost matrix, assign different costs for each entry (Cost sensitive accuracy).

2) Use Precision and Recall! Precision = 0.5 and Recall = 0.5

- (True Positive Rate) $TPR = TP / (TP + FN)$ **Recall**
- (False Negative Rate) $FNR = FN / (TP + FN)$
- (True Negative Rate) $TNR = TN / (FP + TN)$
- (False Positive Rate) $FPR = FP / (FP + TN)$
- $TPR + FNR = 1$ (100% of positive-class data)
- $TNR + FPR = 1$ (100% of negative-class data)

	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)
P (actual)	TP	FN
N (actual)	FP	TN

Example

Classification

Split at $y = 0$, left side taken as negative label, right side as positive label.

Prediction function $y = f(x)$

Take the arrow as the decision threshold for prediction.

sample	N1	N2	P1	N3	P2	P3
input x	-4	-3	-2.5	-2	-1.5	-0.5
Prediction y	-1.1	-0.5	-0.1	0.2	0.6	0.9
Actual Label	-1	-1	1	-1	1	1

If threshold set to be $y=0$,
N3, P2, P3 will be taken as +1
P1, N2, N1 will be taken as -1

With this and actual label, we can make the confusion matrix.

	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)
P (actual)	TP = 2	FN = 1
N (actual)	FP = 1	TN = 2

Classification

Prediction function $y = f(x)$

We can change the threshold!

sample	N1	N2	P1	N3	P2	P3
input x	-4	-3	-2.5	-2	-1.5	-0.5
Prediction y	-1.1	-0.5	-0.1	0.2	0.6	0.9
Actual Label	-1	-1	1	-1	1	1

If threshold set to be $y=0.4$,
P2, P3 will be taken as +1
N3, P1, N2, N1 will be taken as -1

	$\hat{P}$ (predicted)	$\hat{N}$ (predicted)
P (actual)	TP = 2	FN = 1
N (actual)	FP = 0	TN = 3

- TP, FP, FN, TN will change with respect to thresholds!

Multi-Category Classification (Confusion Matrix)

Confusion Matrix for Multicategory Classification

	$P_{\hat{1}}$ (predicted)	$P_{\hat{2}}$ (predicted)		$P_{\hat{C}}$ (predicted)
P_1 (actual)	$P_{1,\hat{1}}$	$P_{1,\hat{2}}$	...	$P_{1,\hat{C}}$
P_2 (actual)	$P_{2,\hat{1}}$	$P_{2,\hat{2}}$	...	$P_{2,\hat{C}}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
P_C (actual)	$P_{C,\hat{1}}$	$P_{C,\hat{2}}$		$P_{C,\hat{C}}$

Other Issues

- Computational speed and memory consumptions are also important factors– Especially for mobile or edge devices
- Other factors– Parallelable, Modularity, Maintainability
- Not focus of this module

Practice Question

Suppose we have a dataset of 550 samples. We take out n samples as test set, and run k-fold cross validation on the remaining samples.



Within each fold, we know that, the number of training samples is **three times as large as** the number of validation samples, and **two times as large as** the number of test samples.

1. What is k ?

$K = 4$
 $n = 150$

2. What is n ?

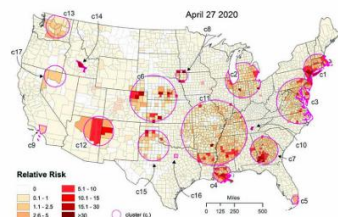
To find n ,
Represent n as parts, n has 1.5 parts. (Half of training)
So, 1 part = $550 / (1.5 + 3 + 1) = 100$.
 $n = 1.5 \text{ parts} = 100 * 1.5 \text{ parts} = 150$.

More Machine Learning Algorithms

- Introduction of unsupervised learning
- K-means Clustering = Hard Clustering
 - The most popular clustering technique
- Fuzzy Clustering = Soft Clustering

More Unsupervised Learning

Unsupervised Learning



Discovering Covid clusters



Business analysis

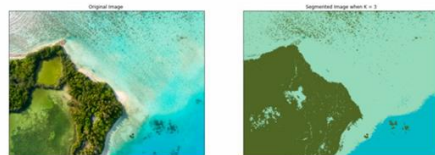
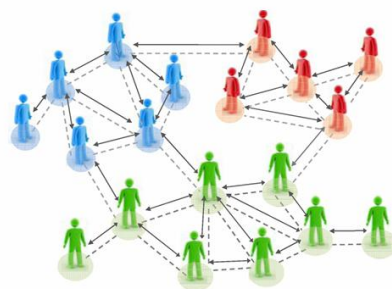


Image segmentation



Community detection in social networks

Motivation: we do not always have labelled data. In unsupervised learning, the dataset is a collection of unlabelled examples.

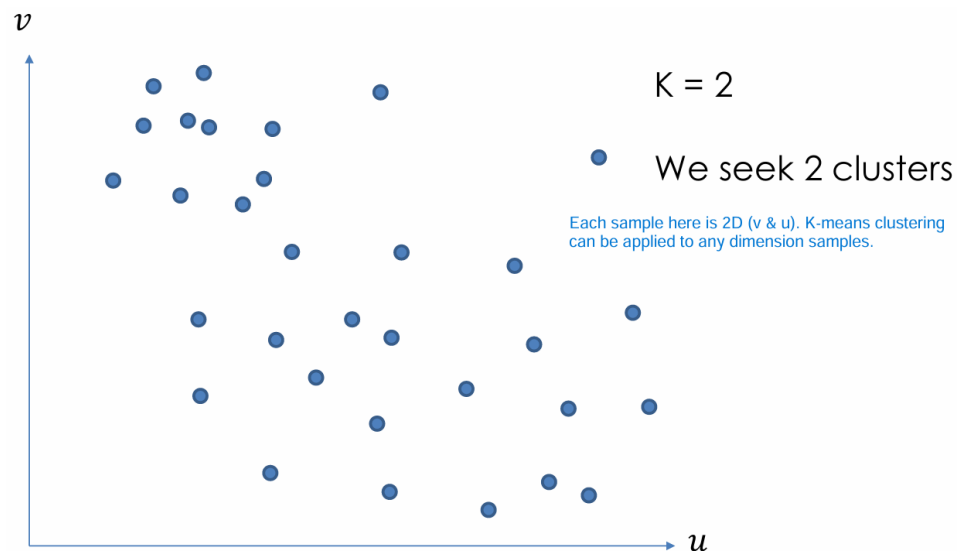
Evaluation of unsupervised learning is hard: The absence of labels representing the desired behaviour for your model means the absence of a solid reference point to judge the quality of your model. No ground truth to refer/compare.

Main Tasks/Approaches of Unsupervised Learning

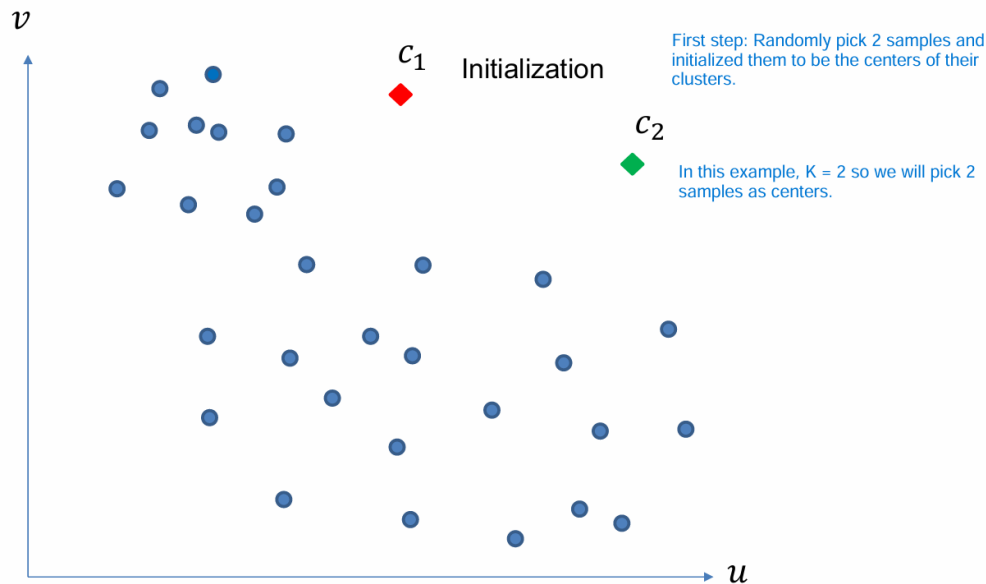
- **Clustering:** Groups a set of objects in such a way that objects in the same group (called a cluster) are more similar (in some sense) to each other than to those in other groups (clusters).
- **Density Estimation:** Models the probability density function (pdf) of the unknown probability distribution from which the dataset has been drawn.
- **Component Analysis:** Breaks down the data from the perspective of signal analysis.
- **Unsupervised Neural Networks:** Autoencoder

K-Means Clustering (2D)

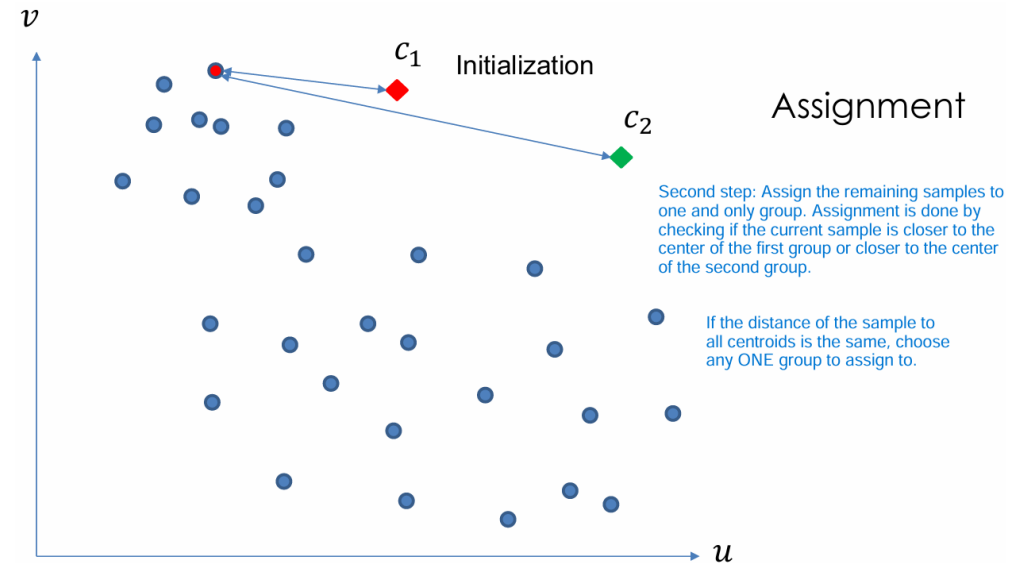
- K refers to the **number of groups/clusters**. K is specified by user.
- Samples without labels (Unsupervised)



- Each sample here is 2D (v & u). K-means clustering can be applied to ANY dimension samples.

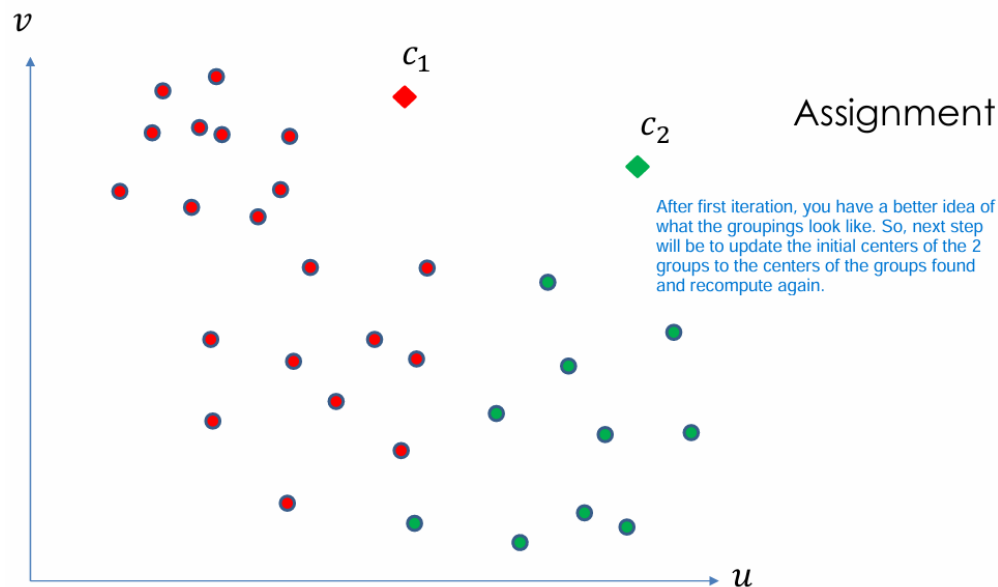


First step: Randomly pick 2 samples and initialized them to be the centres of their clusters. In this example, $K = 2$, so we will pick 2 samples as centres.



Second step: Assign the remaining samples to **one and only one** group. Assignment is done by checking if the current sample is closer to the centre of the first group or closer to the centre of the second group.

- If the distance of the sample to all centroids is the same, choose any ONE group to assign to.
- Taking which sample first for assignment do not matter as long as you loop through ALL samples.



- After first iteration where every sample belongs to a group, you have a better idea of what the groupings look like. So, next step will be to update the initial centres of the 2 groups to the current centres of the groups found and recompute again.

- To get centroid, compute average of the samples in the group.

- **NOTE:** The new centres may not be real samples in the dataset.

- Side Note: Centres are not updated every time we add a sample to a cluster because it is computationally cheaper to group everything first then find the new centre. (Number of updates is only once compared to multiple updates).

- Repeat the process until convergence. K-Means Clustering ALWAYS converges!!

- Centres stays the same = Groupings/Clustering stays the same.

Convergence: (1 & 2 imply each other)

1) Further iterations do not change groupings

2) Further iterations do not change centres.

Basic/Naive K-means Clustering

- Naive K-Means is sensitive to outliers as outliers will drag the clustering towards it.

Basic/Naïve K-means Clustering

Looping between Assignment and Centroid Update

1. First, we **choose** K — the number of clusters. Then we randomly select K feature vectors, called **centroids**, to the feature space.
2. Next, **compute the distance from each example x to each centroid c** using some metric, like the Euclidean distance. Then we **assign the closest centroid to each example** (like if we labeled each example with a centroid id as the label). *Assignment.*
3. For each centroid, we **calculate the average feature vector** of the examples labeled with it. These average feature vectors become the **new locations of the centroids**. *Centers update.*
4. We **recompute** the distance from each example to each centroid, modify the assignment and repeat the procedure until **the assignments don't change after the centroid locations are recomputed**.
5. Finally, we **conclude** the clustering with a list of assignments of centroids IDs to the examples.

- Example for this is the K-Means Clustering (2D) shown above.

Optimization Objective Function (within-cluster variance)

Minimize J

m : # of samples; i : index of samples
 K : # of clusters; k : index of clusters

$$J = \sum_{i=1}^m \sum_{k=1}^K w_{ik} \|\mathbf{x}_i - \mathbf{c}_k\|^2 \quad (1)$$

Minimize the distance between the sample and the centroid.

i -th sample

centroid for the k -th group

The term w_{ik} is equal to 1 for data point $\mathbf{x}_i$ if the data point belongs to cluster S_k , else $w_{ik} = 0$.

Note: The optimization objective function was called $C(\mathbf{w})$ in Lecture 8. Here, we use J (with parameters w_{ik} and $\mathbf{c}_k$) so that it is differentiated from the centroids $\mathbf{c}_k$.

$w_{ik} = 1$ if the i -th sample is assigned to k -th group, else 0.

Optimization Objective Function (within-cluster variance)

Minimize J

$w_{11} = ?$ 1 $\mathbf{x}_1$ ● $\mathbf{x}_3$
 $w_{12} = ?$ 0 $\mathbf{x}_2$ ● $\mathbf{c}_1$ ◆ $\mathbf{c}_2$
 $J = \sum_{i=1}^m \sum_{k=1}^K w_{ik} \|\mathbf{x}_i - \mathbf{c}_k\|^2 \quad (1)$ $\mathbf{x}_2$ ● $\mathbf{x}_4$
 $w_{41} = ?$ 0
 $w_{42} = ?$ 1

The term w_{ik} is equal to 1 for data point $\mathbf{x}_i$ if the data point belongs to cluster S_k , else $w_{ik} = 0$.

Note: The optimization objective function was called $C(\mathbf{w})$ in Lecture 8. Here, we use J (with parameters w_{ik} and $\mathbf{c}_k$) so that it is differentiated from the centroids $\mathbf{c}_k$.

Naïve K-Means Clustering

Naïve K-means Algorithm

1. Assignment Step (fix $\mathbf{c}$ and update $\mathbf{w}$):

Computing distances to all centroids

$$\mathbf{x}_i \in S_k \quad (w_{ik} = 1) \text{ if } \|\mathbf{x}_i - \mathbf{c}_k\|^2 < \|\mathbf{x}_i - \mathbf{c}_j\|^2 \quad (\text{else } w_{ik} = 0),$$

$i = 1, \dots, m; \quad j, k = 1, \dots, K.$

choosing the nearest centroid for the sample, and update w_{ik} to 1 when the closest group is found.

2. Update Step (fix $\mathbf{w}$ and update $\mathbf{c}$):

Compute the average for the group

$$\frac{\partial J}{\partial \mathbf{c}_k} = -2 \sum_{i=1}^m w_{ik} (\mathbf{x}_i - \mathbf{c}_k) = 0 \Rightarrow \mathbf{c}_k = \frac{\sum_{i=1}^m w_{ik} \mathbf{x}_i}{\sum_{i=1}^m w_{ik}}$$

Solving an optimization, i.e., setting derivative to 0

Note: $\|\mathbf{x} - \mathbf{c}\| = \sqrt{\sum_{d=1}^D (x_d - c_d)^2}$ is called the Euclidean distance.
 where $\mathbf{x} = (x_1, x_2, \dots, x_D)$, $\mathbf{c} = (c_1, c_2, \dots, c_D)$

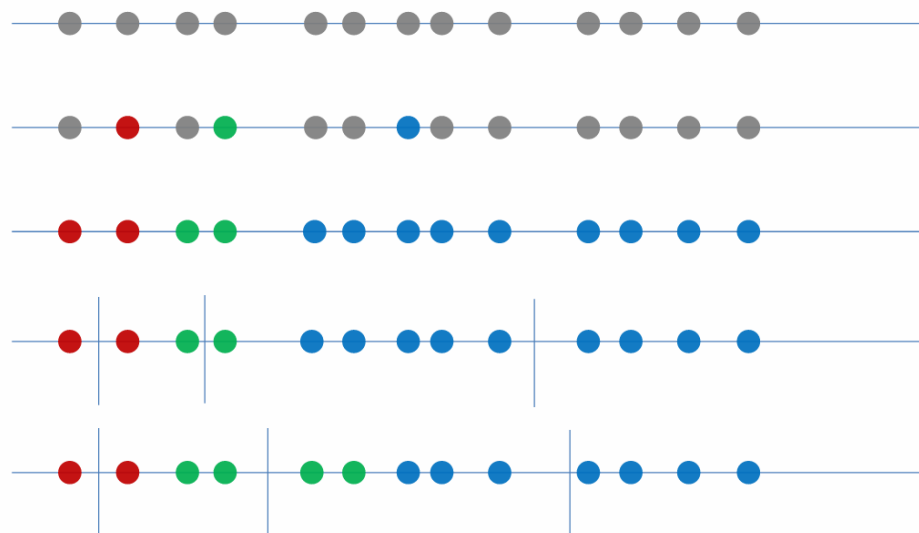
- By repeating these two steps, the total loss J is guaranteed to NOT increase (i.e., remain the same or decrease) until convergence.

Why?

- At Step 2, we compute the new mean, by solving an optimization, i.e., compute the derivative and set to zero, and solve $\mathbf{c}_k$. This means that, the new $\mathbf{c}_k$ is guaranteed to give a smaller J value.

- At Step 1: we only change the assignment, if the distance to the new centroid is smaller! In other words, we either remain in the old group, or change to a new group that is closer (i.e., gives a smaller J)

K-Means Clustering (1D)



^ 1D samples with $K = 3$. The vertical lines are the new centres.

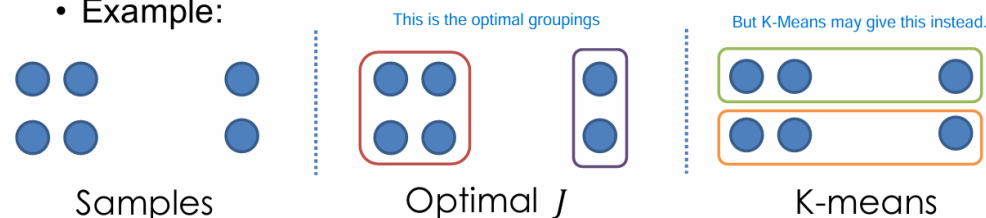


^ Different initializations give different clusters!

K-Means Clustering Issues

- Unfortunately, K-means is not guaranteed to find a global minimum, it finds only local minimum.

• Example:



- Finding the optimal J is NP-hard* (Checking all possible Wik = Computationally Expensive)
- In practice, k-means clustering usually performs well
- It can be very efficient, and its solution can be used as a starting point for other clustering algorithms.

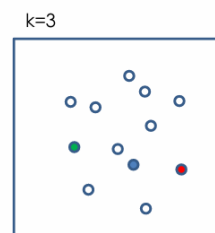
Methods of Initialization for K-Means Clustering

• Initialization

Forgy method:

- Randomly chooses k observations from the dataset and uses these as the initial means.

Initialization by centroid

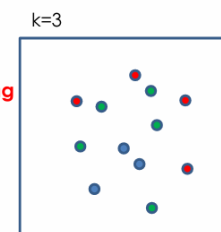


Random partition:

- First randomly assigns a cluster to each observation and then proceeds to the update step, thus computing the initial mean to be the centroid of the cluster's randomly assigned points

Instead of randomly choosing k samples as centers for k groups, we randomly form groups at the start.

Initialization by grouping



- Random Partition: Instead of randomly choosing k samples as centres for k groups, we randomly form groups at the start.

Hard vs Soft Clustering

Hard clustering:

Each data point can **belong to only one** cluster, e.g. K-means

- For example, an apple can be red OR green (hard clustering)

Soft clustering (also known as Fuzzy clustering):

Each data point can **belong to more than one** cluster.

- For example, an apple can be red AND green (fuzzy clustering)
- Here, the apple can be red to a certain degree as well as green to a certain degree.
- Instead of the apple belonging to green [green = 1] and not red [red = 0], the apple can belong to green [green = 0.3] and red [red = 0.5].
- These values are normalized between 0 and 1; however, they DO NOT represent probabilities, so the two values do not need to add up to 1.

Objective Function for Fuzzy C-Means

Objective Function for **Fuzzy C-means**

Minimize J

$$J = \sum_{i=1}^m \sum_{k=1}^C (w_{ik})^r \| \mathbf{x}_i - \mathbf{c}_k \|^2$$

$\mathbf{x}_i$ is the i-th sample and $\mathbf{c}_k$ is the centroid for the k-th group

where $w_{ik} = \frac{1}{\sum_{j=1}^C \left(\frac{\| \mathbf{x}_i - \mathbf{c}_k \|}{\| \mathbf{x}_i - \mathbf{c}_j \|} \right)^{\frac{2}{r-1}}}$

No need to memorize the equation!

W_{ik} no longer 0 or 1 but a range from 0 to 1.

Each element, $w_{ik} \in [0,1]$, tells the degree to which element, $\mathbf{x}_i$, belongs to cluster $\mathbf{c}_k$.

The **fuzzifier $r > 1$** determines the level of cluster fuzziness; usually $1.25 \leq r \leq 2$.

- W_{ik} no longer 0 or 1 but a range from 0 to 1.

Objective Function for Fuzzy C-means

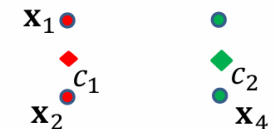
Minimize J

$$J = \sum_{i=1}^m \sum_{k=1}^C (w_{ik})^r \| \mathbf{x}_i - \mathbf{c}_k \|^2$$

where $w_{ik} = \frac{1}{\sum_{j=1}^C \left(\frac{\| \mathbf{x}_i - \mathbf{c}_k \|}{\| \mathbf{x}_i - \mathbf{c}_j \|} \right)^{\frac{2}{r-1}}}$

W₁₁ > W₁₂ as sample 1 is closer to C1 than C2.

$$\begin{aligned} w_{11} &= 0.6 \\ w_{12} &= 0.2 \end{aligned}$$



$$\begin{aligned} w_{41} &= 0.18 \\ w_{42} &= 0.75 \end{aligned}$$

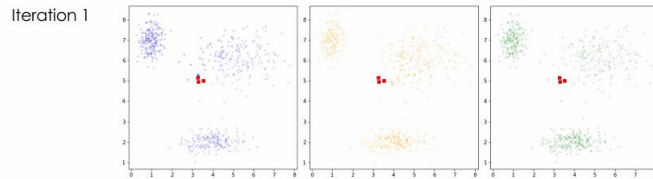
W₄₁ < W₄₂ as sample 4 is closer to C2 than C1.

Each element, $w_{ik} \in [0,1]$, tells the degree to which element, $\mathbf{x}_i$, belongs to cluster $\mathbf{c}_k$.

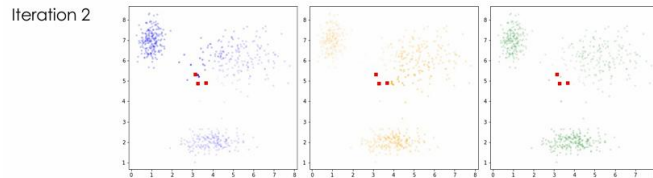
The fuzzifier $r > 1$ determines the level of cluster fuzziness; usually $1.25 \leq r \leq 2$.

Usually $r = 1.5$?

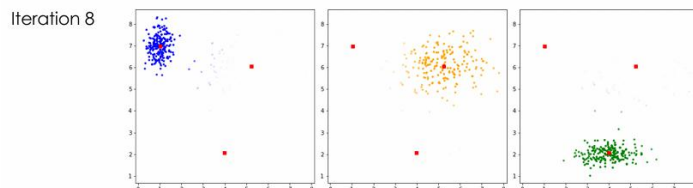
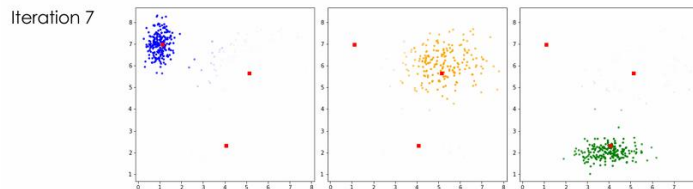
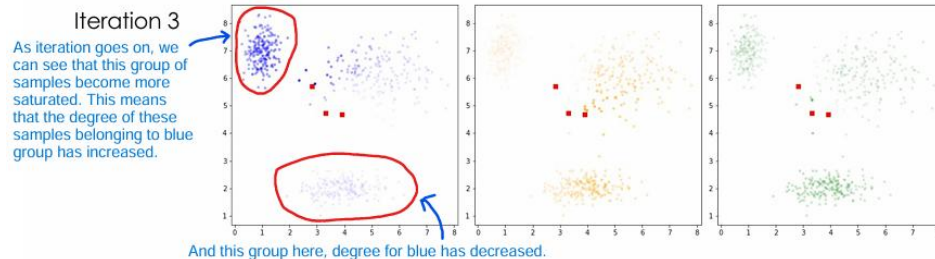
Task: Group the 3 samples into their groups.



Each column is the same dataset just that from left to right, it shows the degree of samples belonging to cluster blue, yellow and green.



Visualization of Fuzzy C-means Iterations

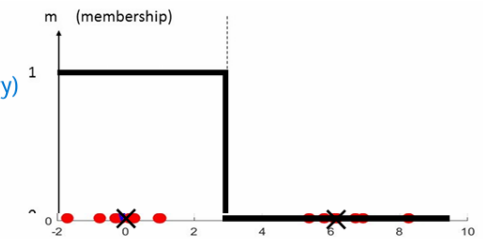


Visualization of Fuzzy C-means Iterations

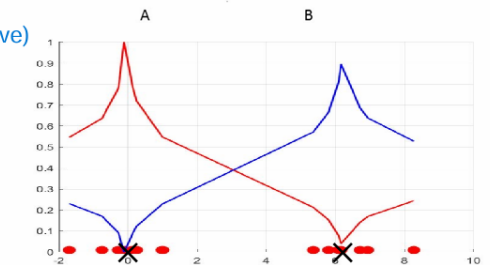
- The saturated groupings have high degree for their respective colours.
- Those light-coloured samples are **due to fuzziness** in which they have a fair degree to be assigned to that colour.

Naive K-Means vs Fuzzy C-Means

Naïve K-means: $w_{ik} \in \{0,1\}$
0 or 1 (Binary)



Fuzzy C-means: $w_{ik} \in [0,1]$
0 to 1 (Graph/Curve)



Ref: https://en.wikipedia.org/wiki/Fuzzy_clustering

Practice Question

We have a collection of 9 foreign coins. We measure their radius in millimeters and summarize them as follows.

Coin ID	01	02	03	04	05	06	07	08	09
Radius (mm)	10	11	12	15	16	17	20	21	22

We'd like to group the coins into three groups according to their radius.

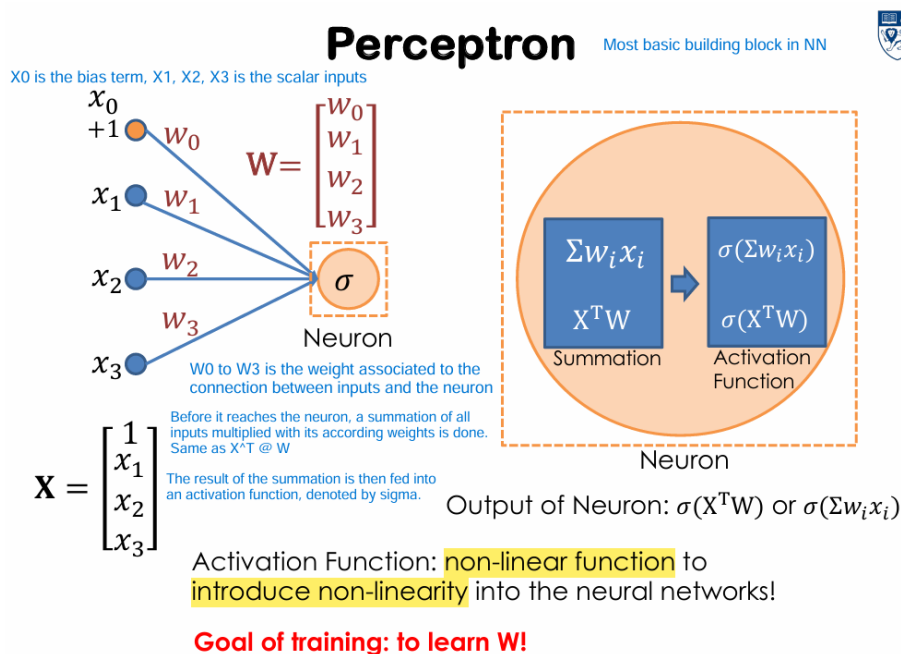
Assume we pick coin 01 as the initial centroid for Group A, coin 04 for Group B, and coin 07 for Group C. We would like to assign the coins to the three groups using Euclidean distance. Before updating the new centroid, we will have _BLANK1_ coins in Group A (please enter an integer here).

Group A: 3 Coins (01, 02 & 03)

Neural Networks

- Introduction to Neural Networks
 - Perceptron
 - Activation Functions
 - Multi-layer Perceptron
- Training and Testing of Neural Networks
 - Training: Forward and Backward
 - Testing: Forward
- Convolutional Neural Networks

Perceptron

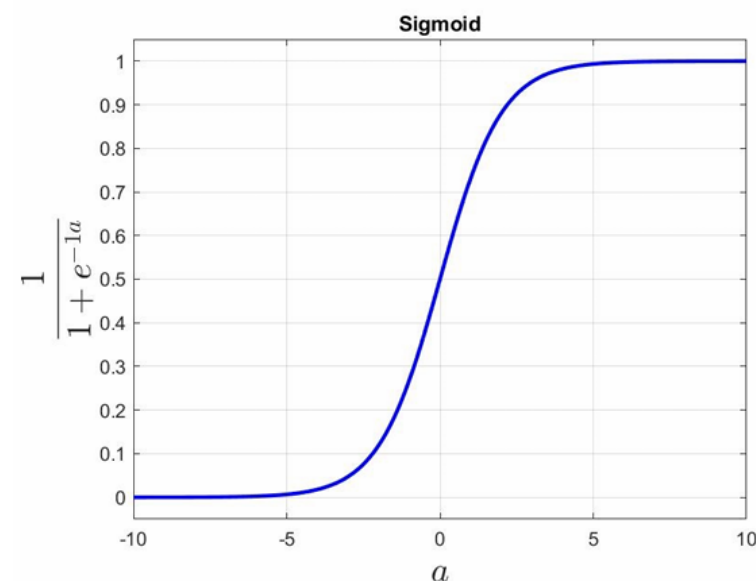


Activation Function: Non-linear function to introduce non-linearity into the neural networks!

- Before the input vector reaches the neuron, a summation of all inputs multiplied with its according weights is done. Same as $X^T W$.
- The result of the summation is then fed into an activation function, denoted by sigma.

Sigmoid Activation Function

$$\sigma(a) = \frac{1}{1 + e^{-\beta a}},$$

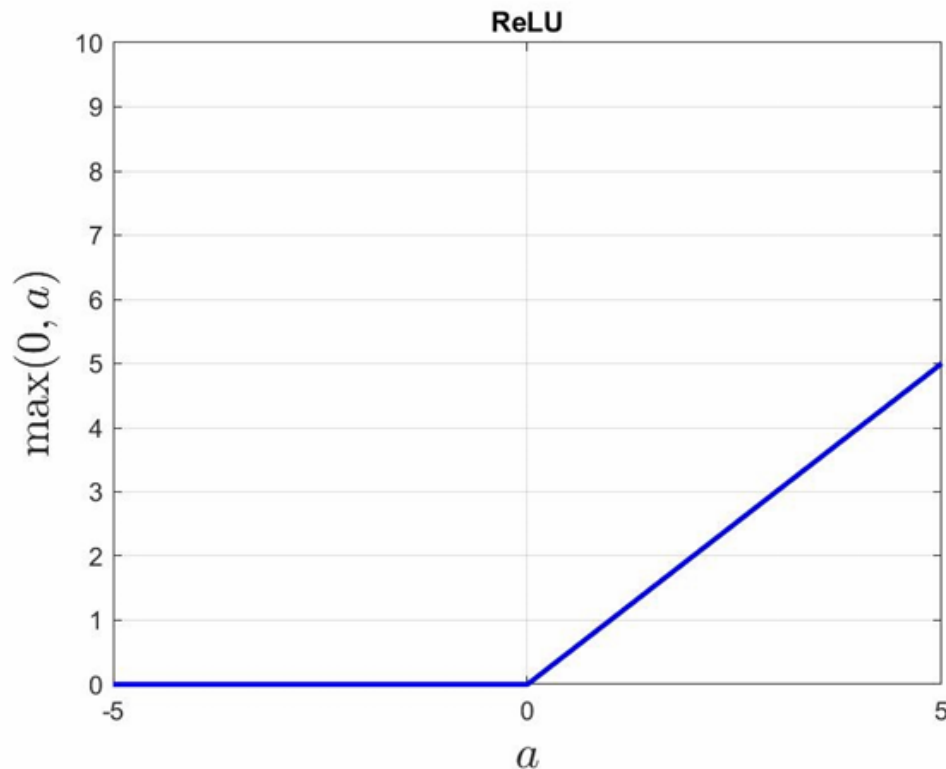


- The graph is when beta = 1.
- Output of this activation function is always between 0 and 1. Useful when you want the output to have a probability-like value.

ReLU Activation Function

$$\sigma(a) = \max(0, a)$$

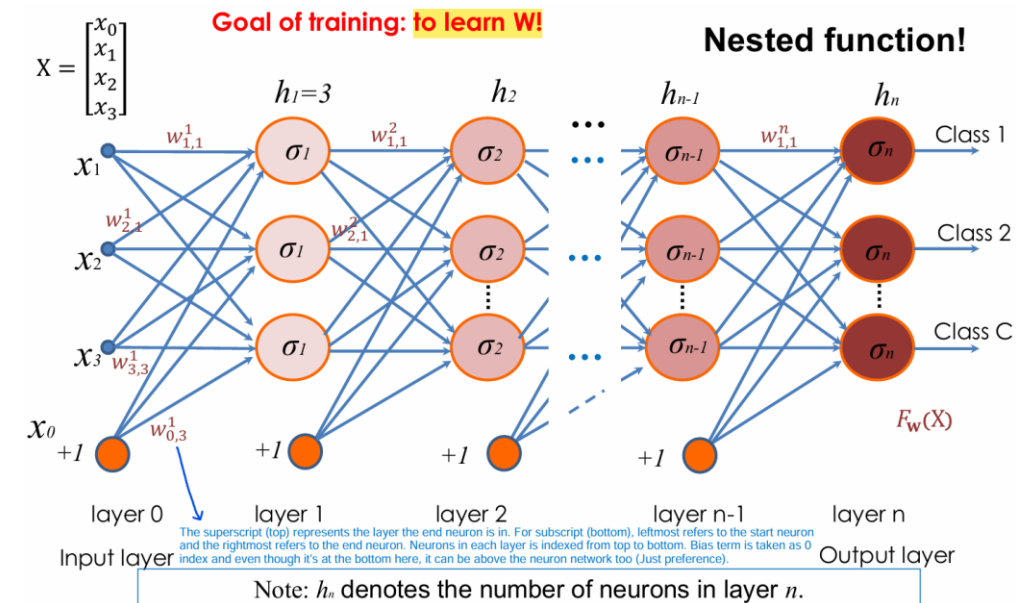
Rectified Linear Unit (ReLU)



^ Outputs the greater between a and 0.

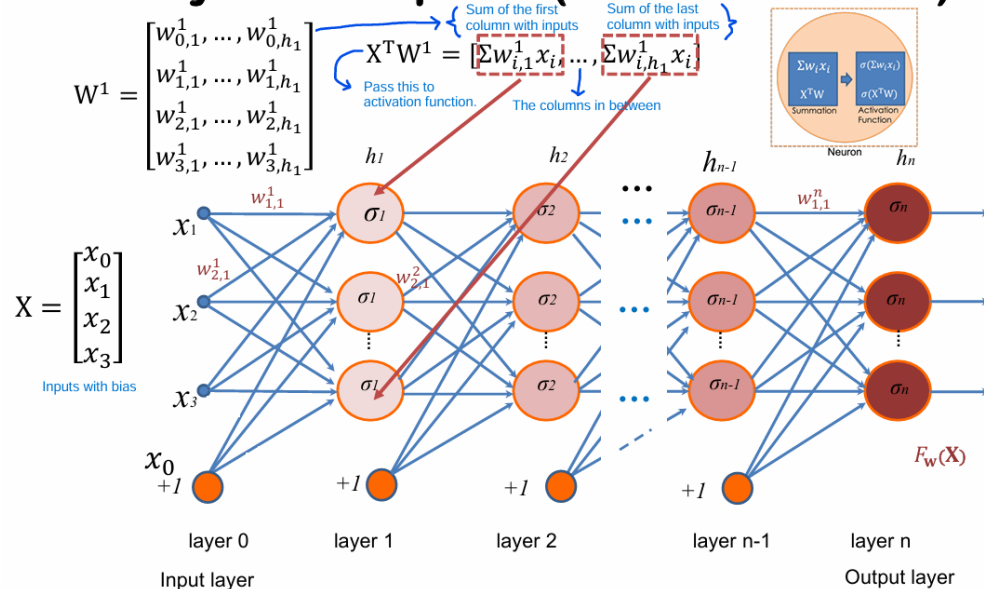
Multilayer Perceptron (Neural Network)

Goal of training: To learn W !



- Stack the neurons vertically and horizontally. Each vertical stack is called a layer. Each layer can have different numbers of neurons, and this is defined by user. Number of layers is also determined by user.
- The number of neurons in different layers may differ, i.e., h_1 don't have to be equal to h_2 .
- For **classification** task, the number of neurons in the last layer equals to the number of classes.
- We can treat the whole network as a function $F_w(X)$ where w is to be learned.
- Also, notice that the input has bias included.

Multilayer Perceptron (Neural Network)

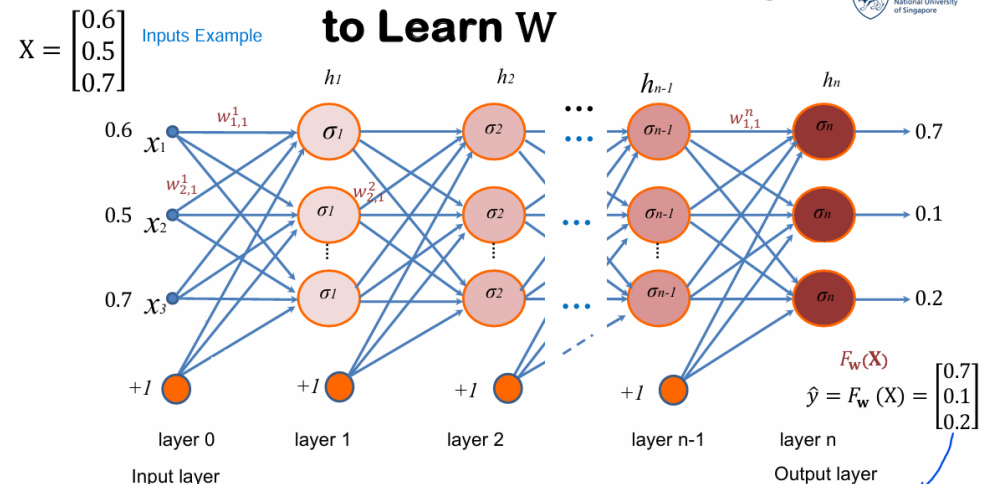


- W^1 is the weight vector for the neurons in layer 1 only.
- In W^1 , each column represents the weights for the connections going into the neurons in the next layer.
- Leftmost column is all the weights for the corresponding inputs going into the FIRST neuron in layer 1, second column represents all the weights for the corresponding inputs going into the second neuron in layer 1, so on and forth until all neurons in the next layer are covered. h_1 means the total number of neurons in layer 1.
- A neural network is essentially a **nested function**. Output of a neuron in layer x becomes the input for another neuron in layer $x + 1$.

$$F_W(X) = \sigma([1, \dots \sigma([1, \sigma(X^T W^1)] \ W^2) \dots] \ W^n)$$

How to learn W ?

Goal of Neural Network Training: to Learn W



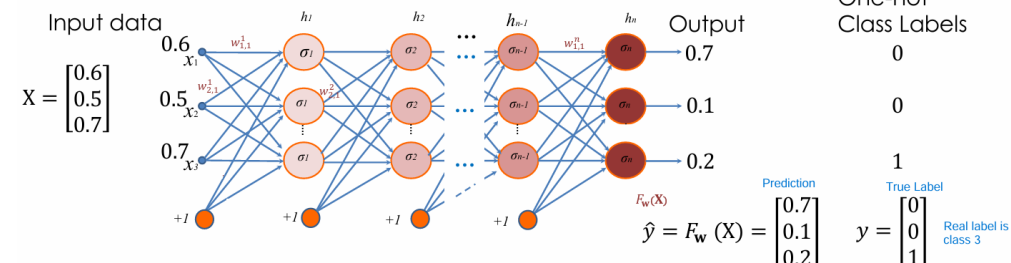
This means that the input X has a probability of 0.7 to belong in class 1, 0.1 to class 2 and 0.2 to class 3.

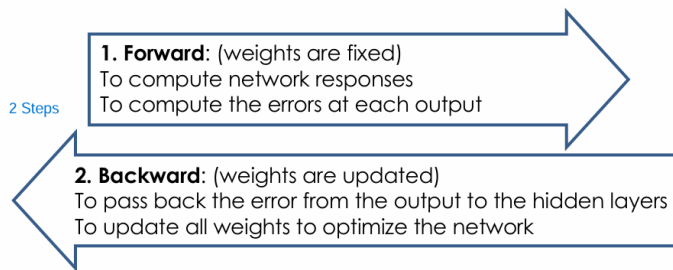
Specifically, W is learned through

1. Random initialization The weights are randomly initialized.
2. Backpropagation

Neural Network Training: Backpropagation

Assume we train a NN for 3-class classification





A loss function for a single sample:
 $\min_{\mathbf{w}} \sum_{i=1}^C (\hat{y}_i - y_i)^2$
In this case, C = 3
 or
 $\min_{\mathbf{w}} ||\hat{y} - y||^2$
Update W!

- The very first iteration, the w are initialized randomly, we get the network response, update the weights and this repeat.
- IMPORTANT: There are 2 steps in backpropagation! **Forward pass** (compute outputs and loss, with weights fixed). **Backward pass** (compute gradients and update weights).
- Squared errors is used for the loss function here.

Backpropagation (Loss Function)

- Recall that the parameters W are randomly initialized.
- We use Backpropagation to update W .
- In essence, Backpropagation is gradient descent!
- Assume we have N samples, each sample denoted by X^j and the output of NN by $\hat{y}^j$, loss function is then

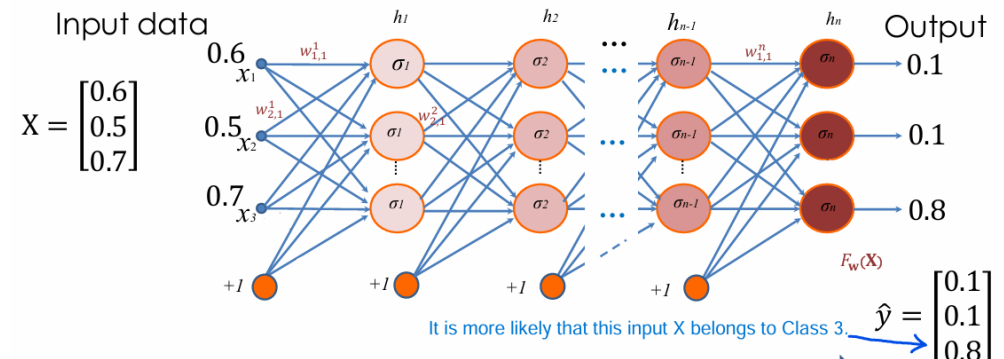
$$J = \sum_{j=1}^N ||\hat{y}^j - y^j||^2, \quad \min_{\mathbf{w}} J$$

Recall gradient descent in Lec 8: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} J$

- We would therefore like to compute $\nabla_{\mathbf{w}} J$!
 - J is a function of $\hat{y}$, and $\hat{y}$ is a function of $\mathbf{w}$, i.e., $\hat{y} = F_{\mathbf{w}}(X)$
 - Use gradient descent and chain rule!

Neural Network Testing

- Once all network is trained and parameters are updated, we may conduct testing. (1 step)

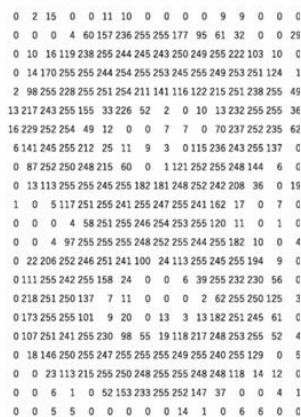
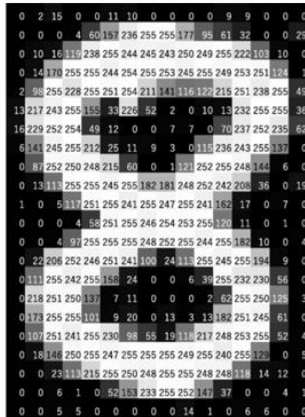


1. Forward: (weights are fixed)
To estimate compute network responses
To predict the output labels given novel inputs

- Weights are now fixed as computation is complete. Feed inputs in the input layer to get a prediction output.

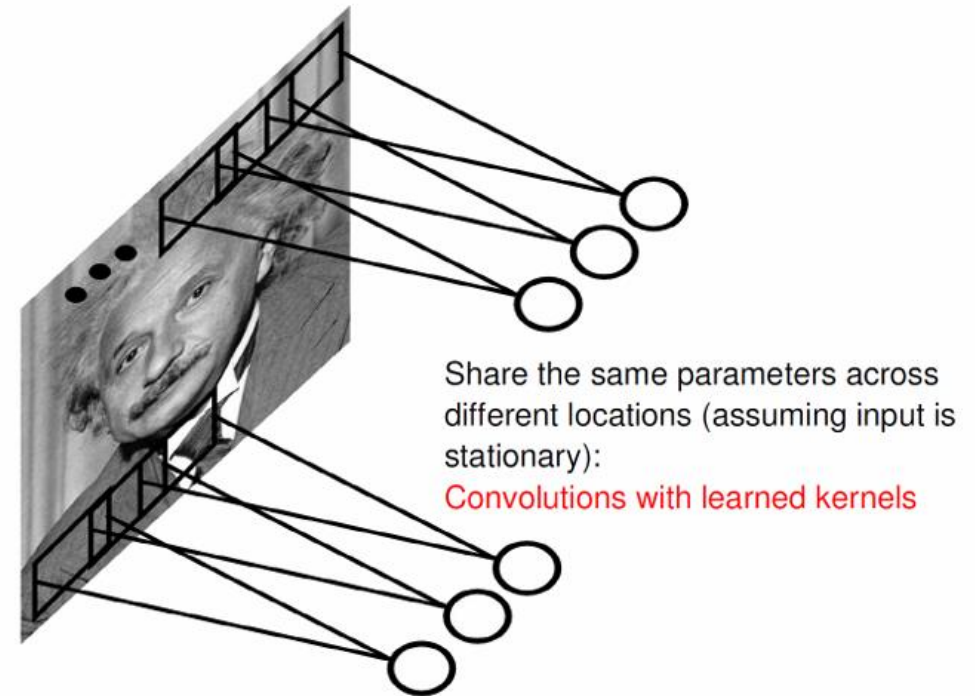
Convolutional Neural Network (CNN)

- A convolutional neural network (CNN) is a special type of neural network that **significantly reduces the number of parameters** in a deep neural network.
- Very popular in image-related applications
- Each image is stored as a matrix in a computer

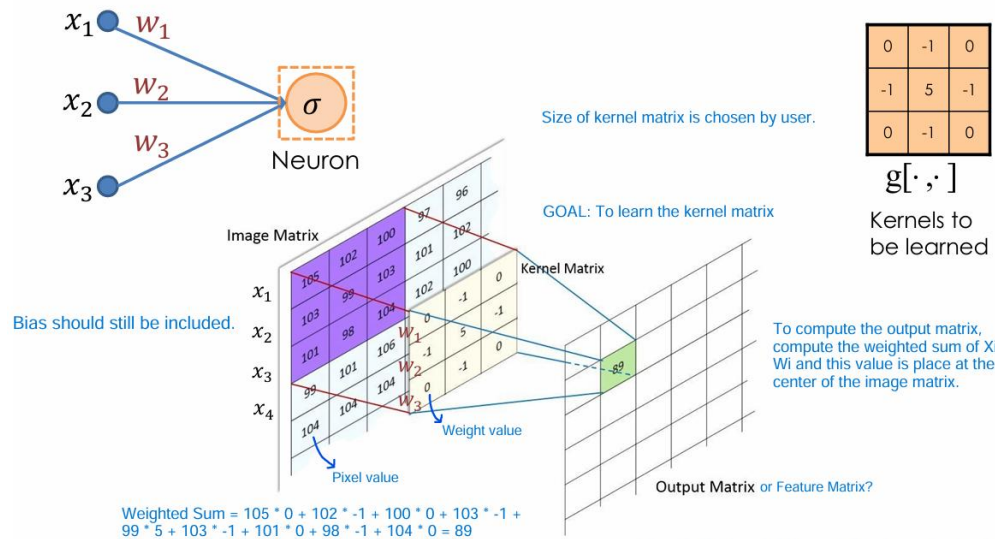


Example 1

- If we model all matrix entries as inputs all at once
 - Assume we have an image/matrix size of 200x200
 - Assume we have 10,000 neurons in the first layer
 - Imagine the input is an image broken down into 200×200 pixels, then converted to greyscale and then into a matrix, we will have $200 \times 200 = 40,000$ inputs for one neuron. Multiplied with 10,000 neurons, we will have 400 million parameters to learn!
- Hence, we introduce CNN to reduce the number of parameters.
- Works in a **sliding-window** manner!
 - Instead of linking all 200×200 pixels to one neuron, we provide a small local neighbourhood of pixels instead. This neighbourhood is called a kernel. To ensure all pixels in the image are captured, we do a sliding-window of the kernel by scanning the whole image/matrix.

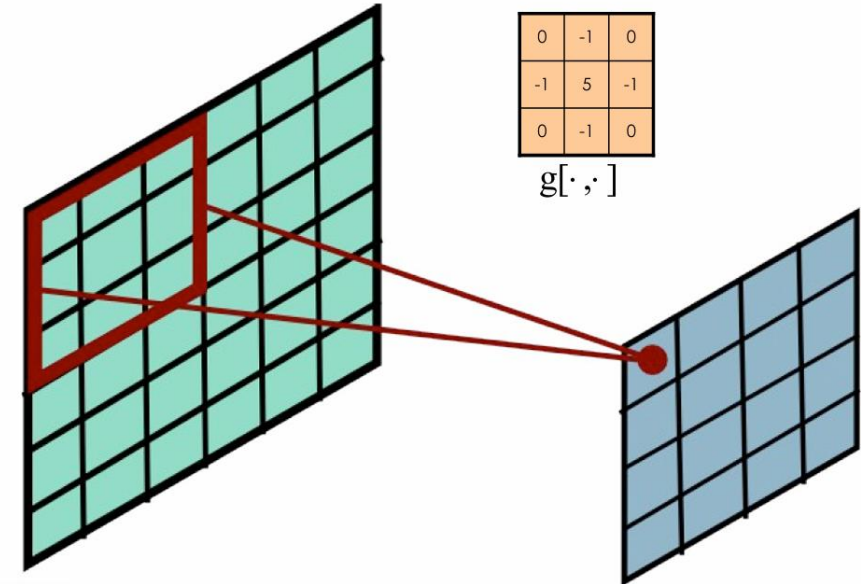


- You can have **multiple kernels**. Each Kernel will have its own feature map/output matrix. From there, you can do pooling (max or average to get smaller feature matrix but same number of feature matrix).



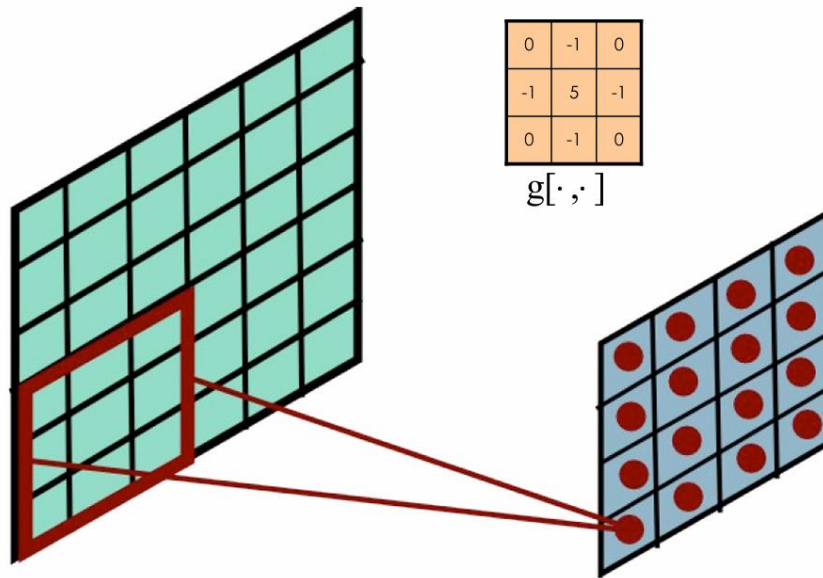
- Size of kernel matrix is chosen by user. Usually a square matrix with odd length.
- Goal is to learn the weights for the kernel matrix.
- In **convolution**, you slide a small **filter** (or **kernel**) over the input image.
- At each position, you compute a **dot product** between the filter and the overlapping patch of the image.
- If you use a **3x3** filter on an input without padding:
You **cannot** center the filter at the edges without going "out of bounds". Thus, the output is **smaller**.

Convolutional Neural Network (CNN)



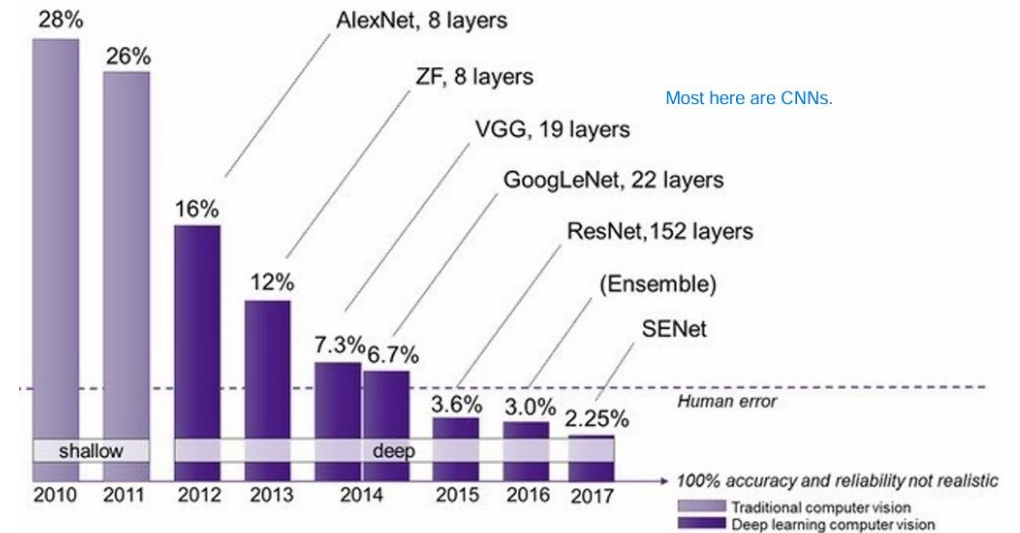
- Need to learn what the value in each box of the kernel matrix should be and this is done the same way as NN (Backpropagation).

Convolutional Neural Network (CNN)



- Kernel matrix stays the same when scanning.
- Notice the output matrix is of smaller length.

Neural Networks are Effective



- 100% accuracy and reliability is not realistic.

Practice Question

- Which of the following statements is correct?
 - A) Sigmoid serves as a linear activation function in neural networks. *Should be non-linear.*
 - B) Compared to dense connection (connecting each node in previous layer to the current layer), convolutional neural networks (CNNs) yield more computational complexity. *Less computational complexity as lesser parameters to learn.*
 - C) When training neural networks, its network weights W are fixed. *Weights are updated in training and fixed in testing.*
 - D) None of the others is correct. **D**